

—

Занятие №11.

Графы в рекомендациях

Лашинин Олег

01.04.26

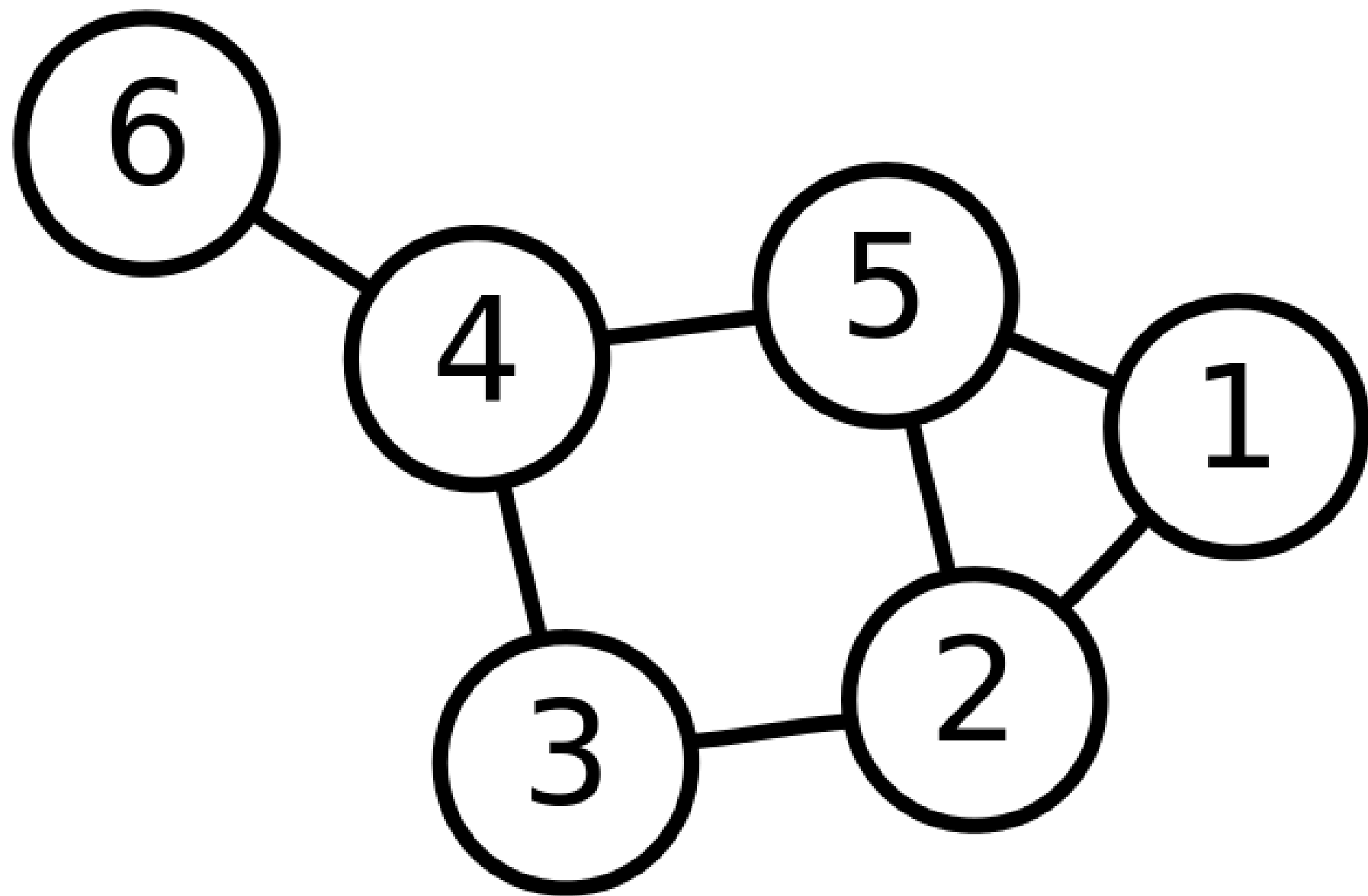
План занятия

- Графы
- Примеры задач на графах
- Графовые свертки и примеры
- Двудольные графы в рекомендациях и модели на них
- Графы знаний и recsys модели на них
- Про сравнение с трансформерами

Почему сегодня эта тема?

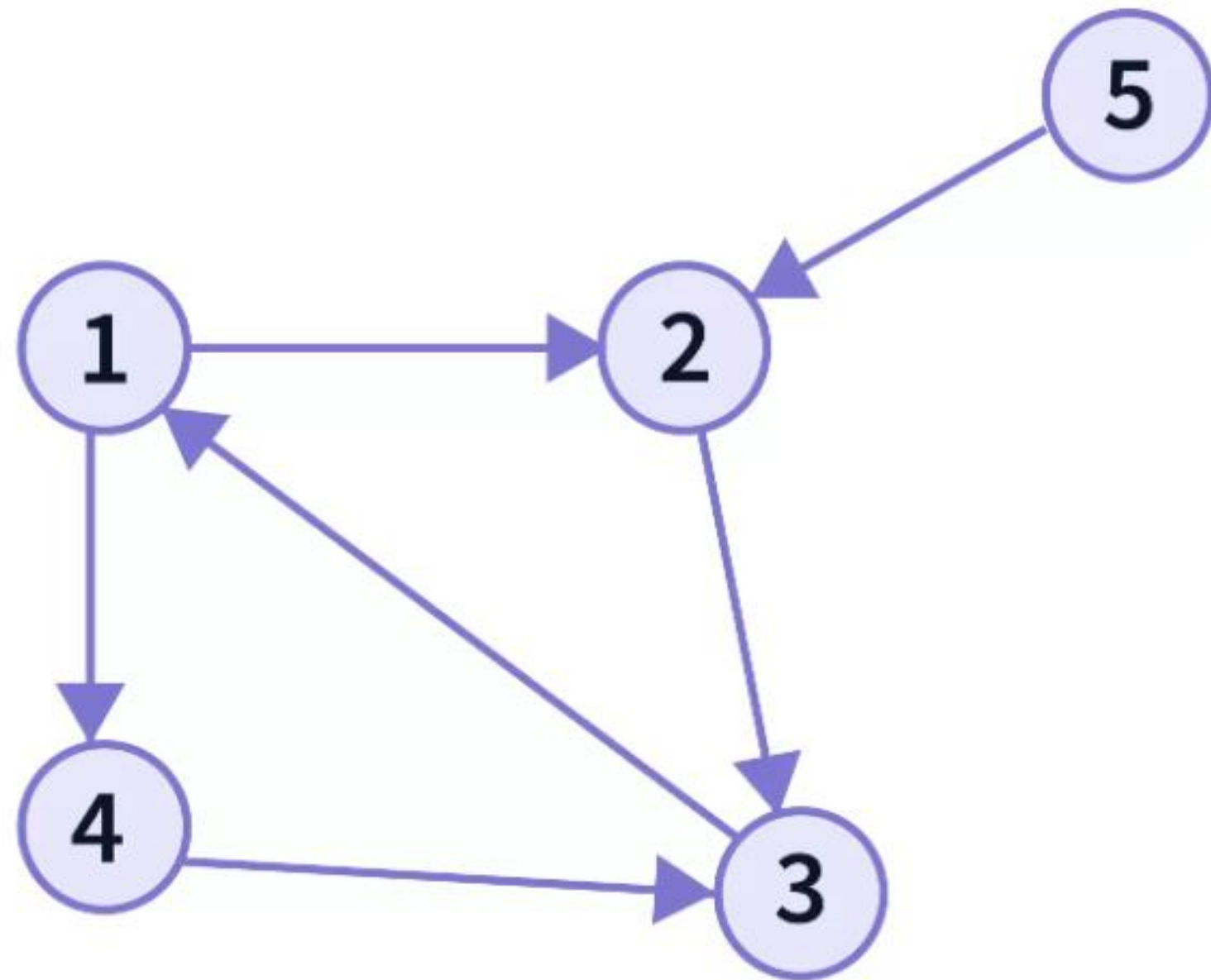
Графы – конкурирующий подход с трансформерами по возможностям учета различной информации, с явным моделированием связей высокого порядка

Графы



Граф $G(V, E)$ - объект, задающийся двумя множествами **вершин** и **ребер**

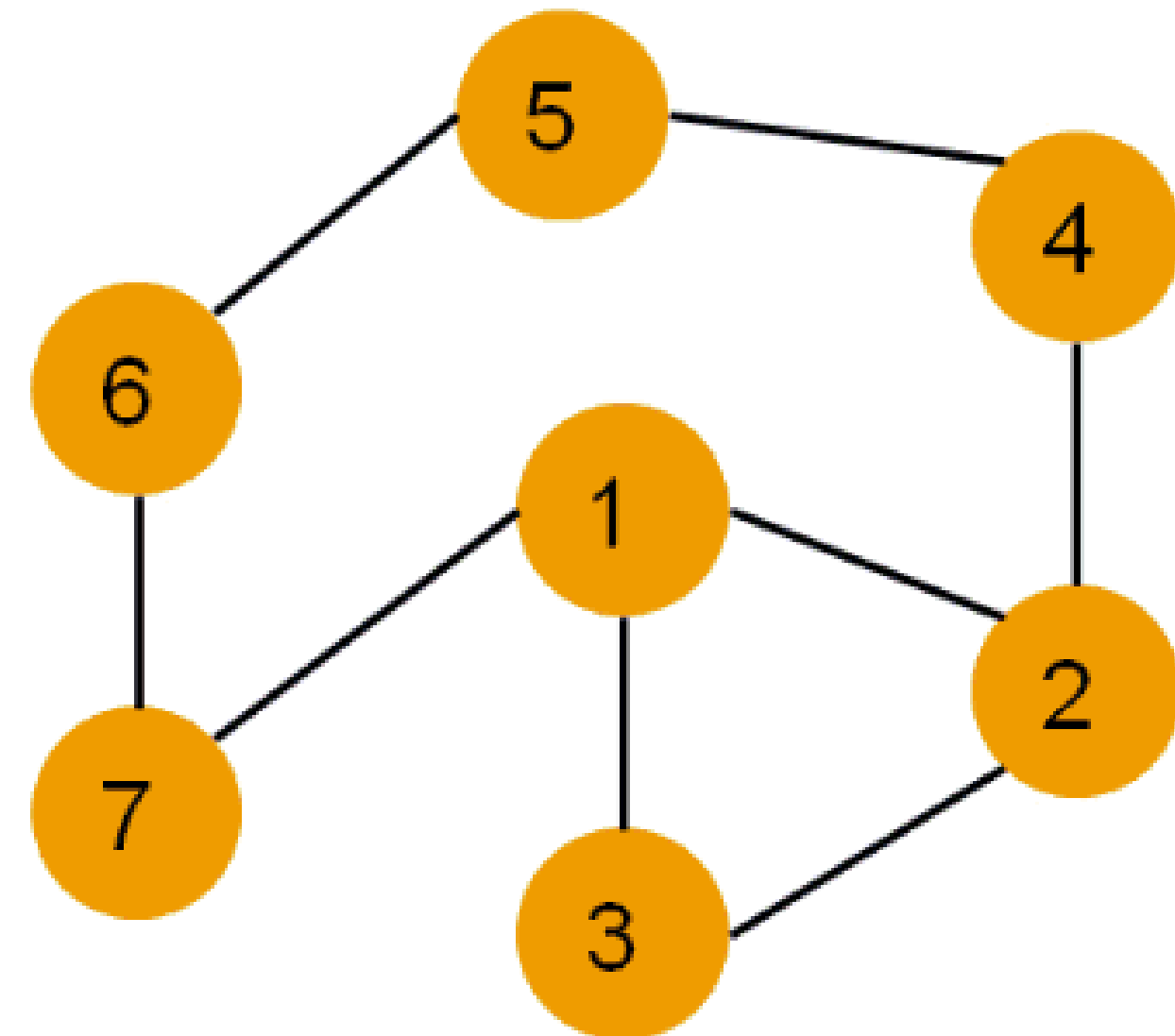
Графы



Ориентированный

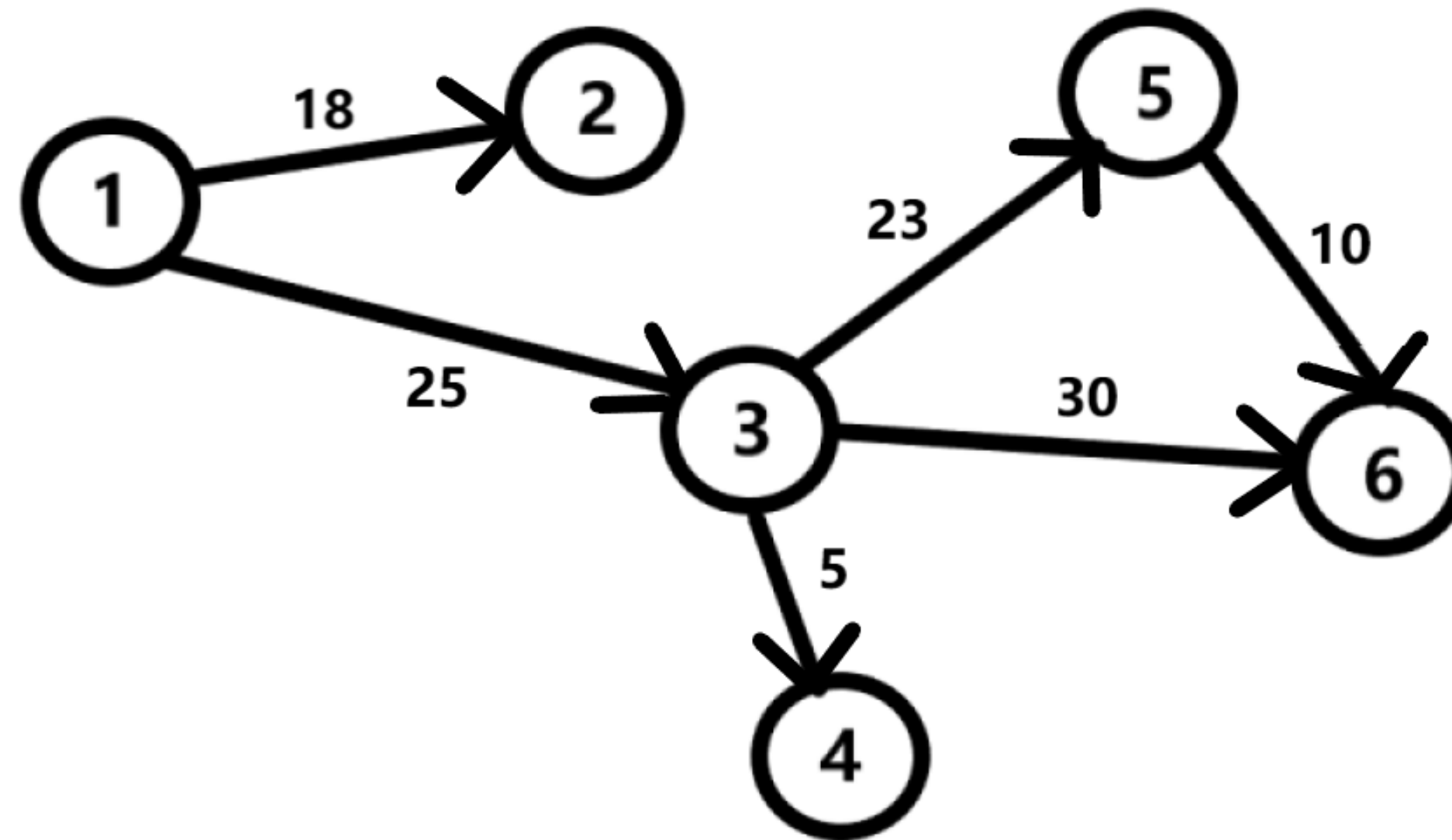


Придумайте примеры



Неориентированный

Графы

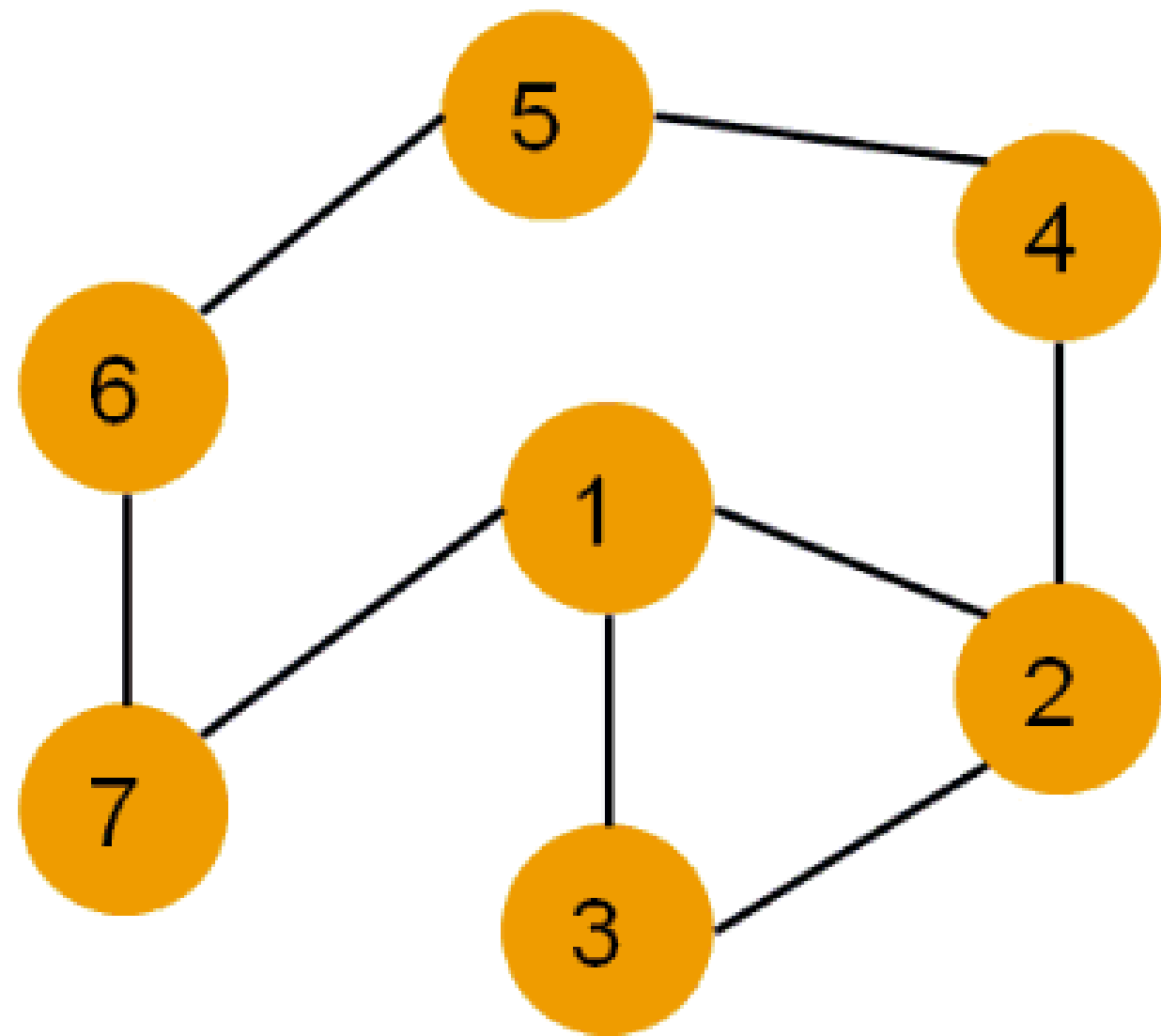


Взвешанный

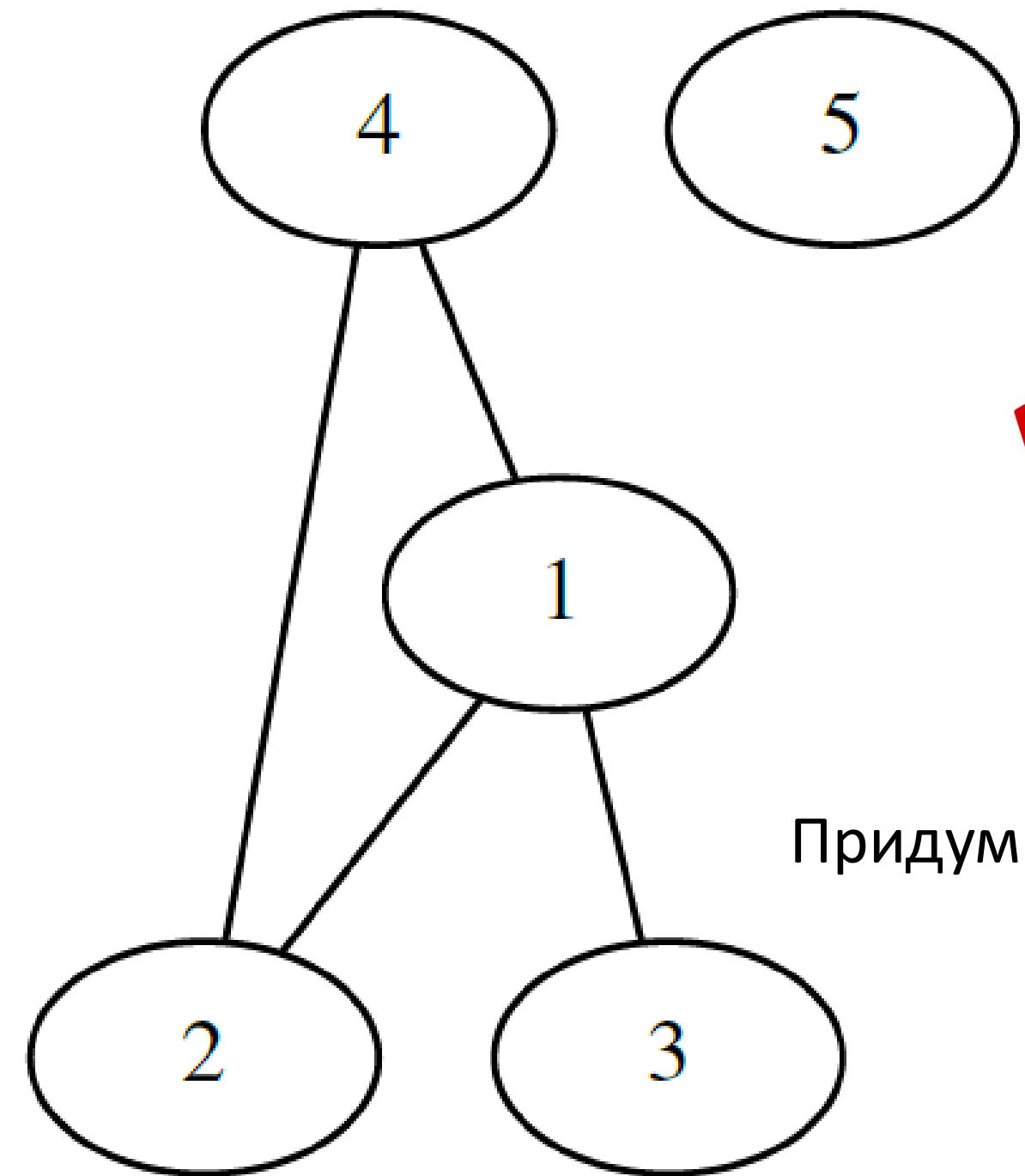


Придумайте примеры

Графы



СВЯЗНЫЙ



Несвязный

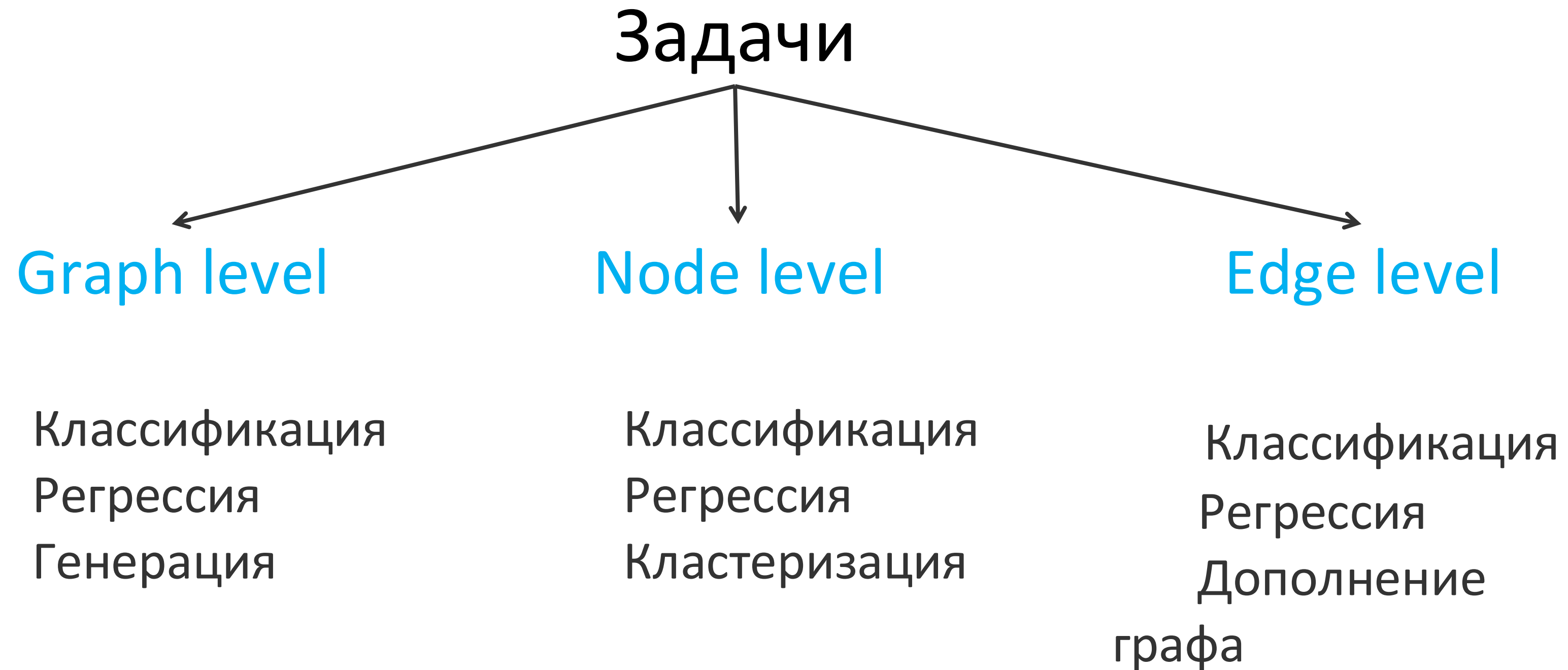


Придумайте примеры

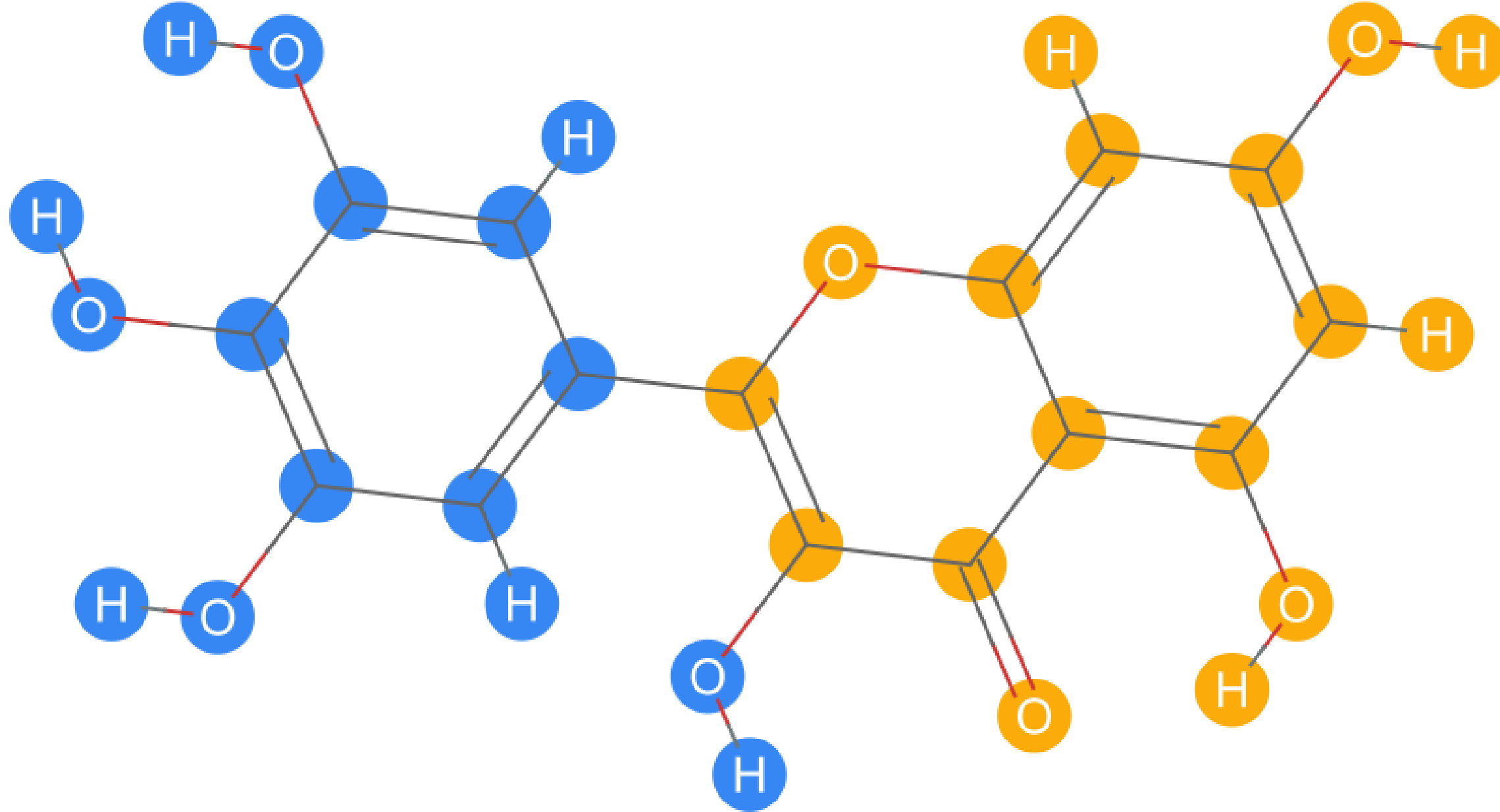
Московский
Университет



Задачи на графах



Graph level



Классификация: является антибактериальным/токсичным

Регрессия: Характеристика липофильности молекул

Генерация: Сгенерировать молекулу обладающую нужными свойствами

Node level

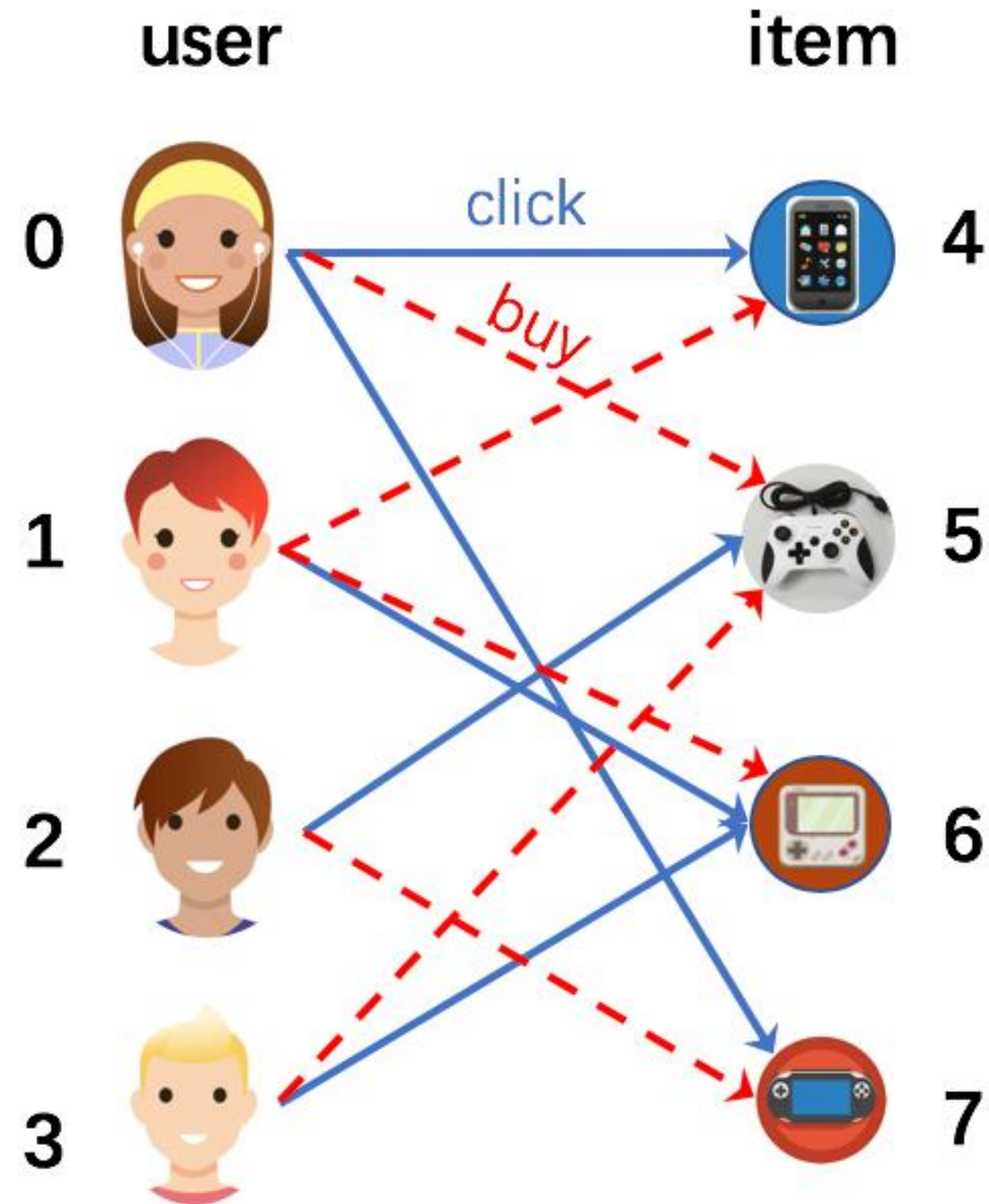


Классификация: Выдавать/не выдавать кредитку

Регрессия: Степень интереса в тематике

Кластеризация: Выделение КОМЬЮНИТИ

Edge level



Двудольный граф

Классификация: Определение типа взаимодействия

Регрессия: Степень интереса к товару

Дополнение графа: Предсказания взаимодействия

user-user graph

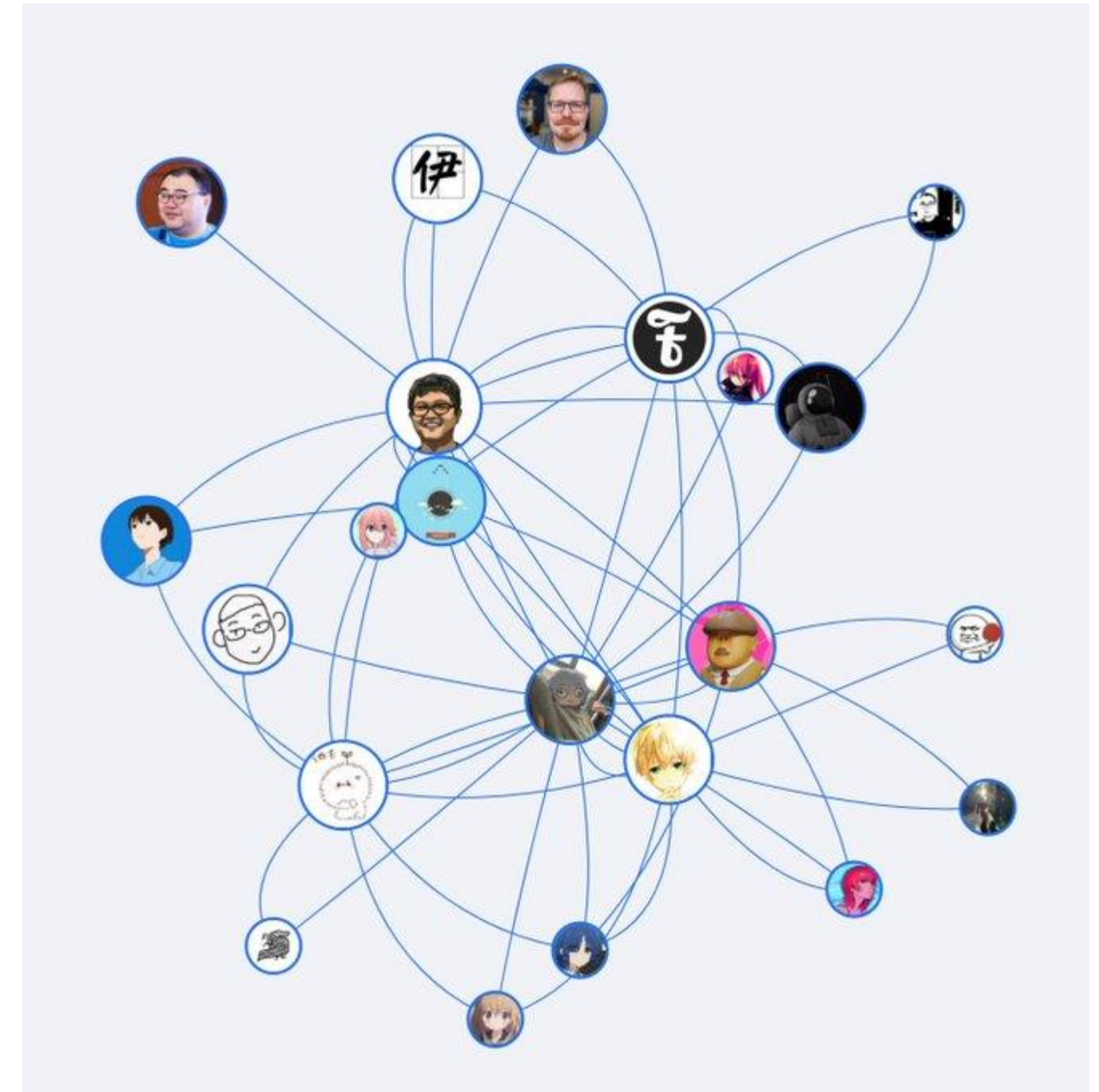
$G = (U, E)$, где

U - множество пользователей

E - множество ребер, отображающих взаимодействия либо свойства похожести между пользователями

Такой граф называется социальным.

Пример задачи: рекомендация новых друзей.



Graph/Edge/node level задача?

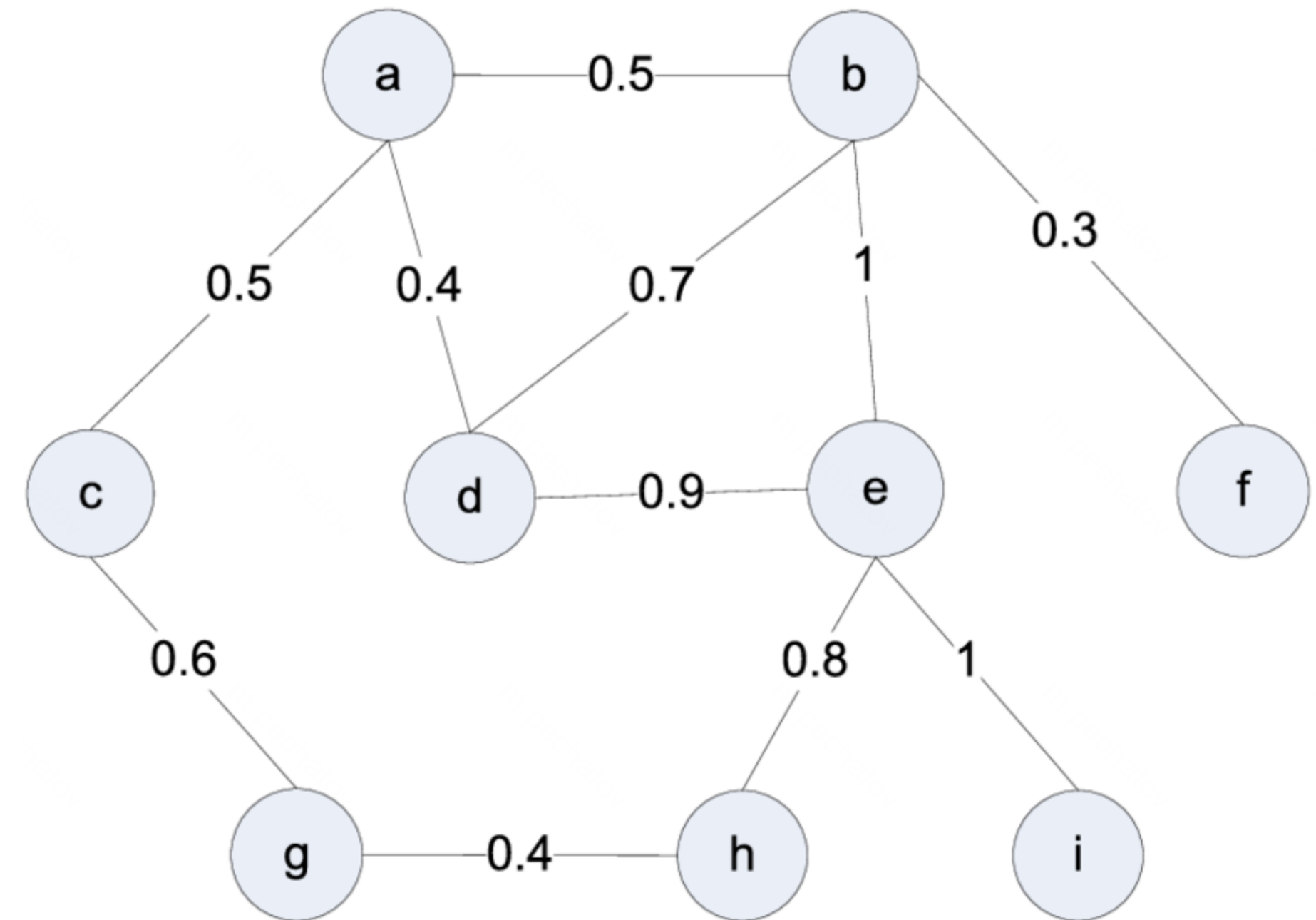
item-item graph

$G = (I, E)$, где

I - множество айтемов

E - множество ребер, отображающих взаимодействия либо свойства похожести между айтемами

Пример задачи: рекомендация похожих/сопутствующих товаров.



user-item graph

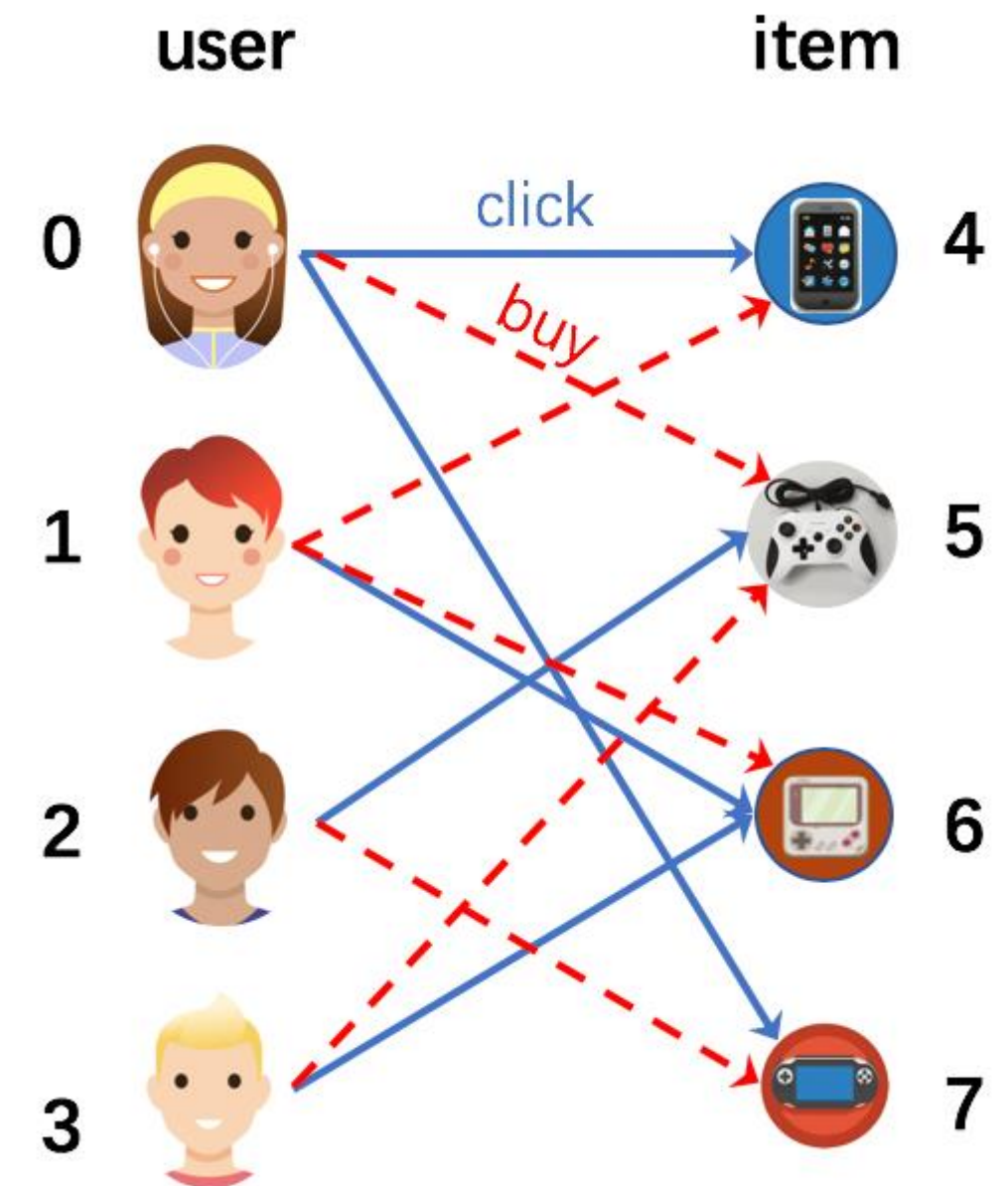
$G = (V, E)$, где

V - множество юзеров и айтемов

E - множество ребер, отображающих взаимодействия между пользователями и айтемами

Граф, как правило, является двудольным

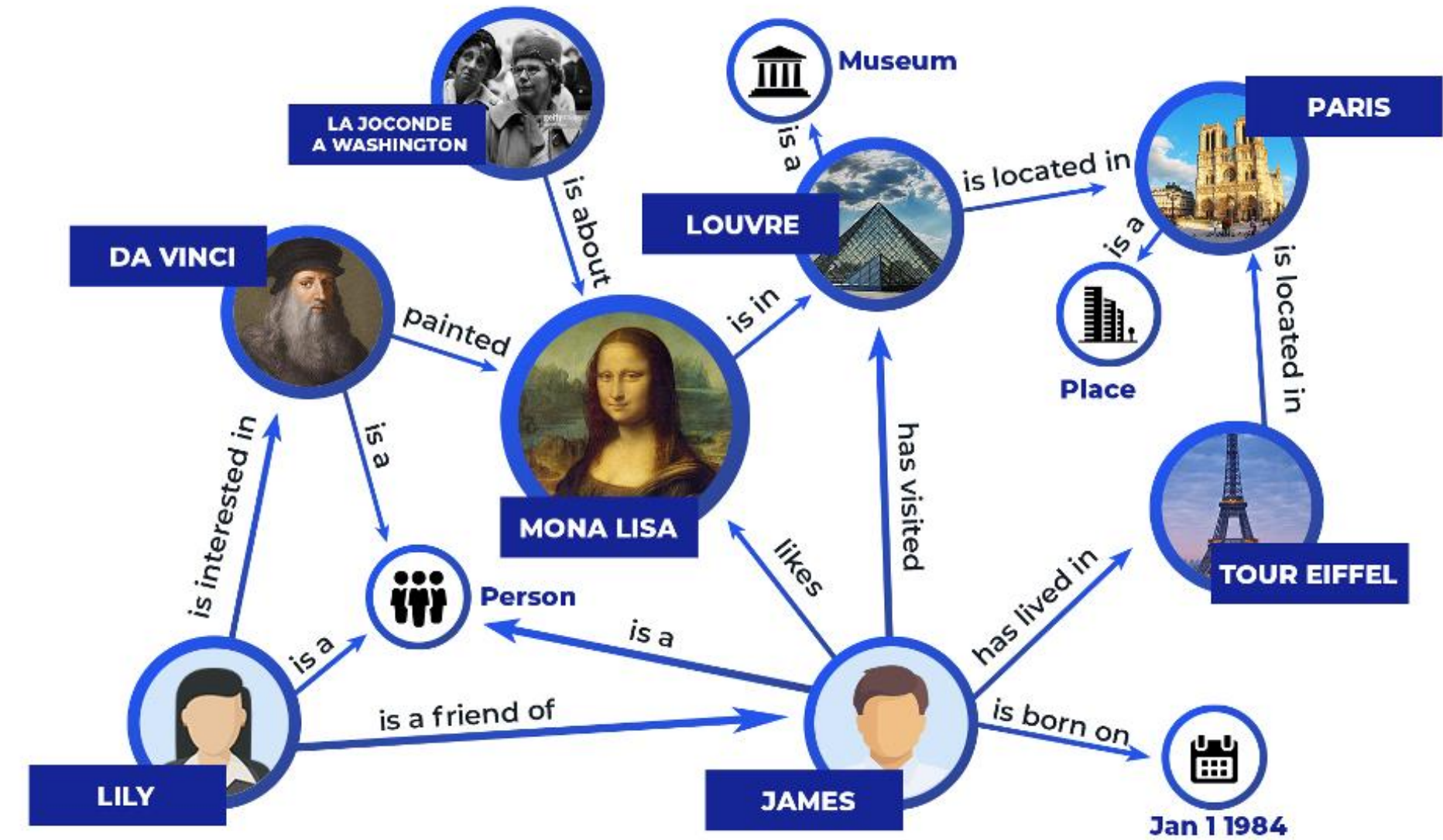
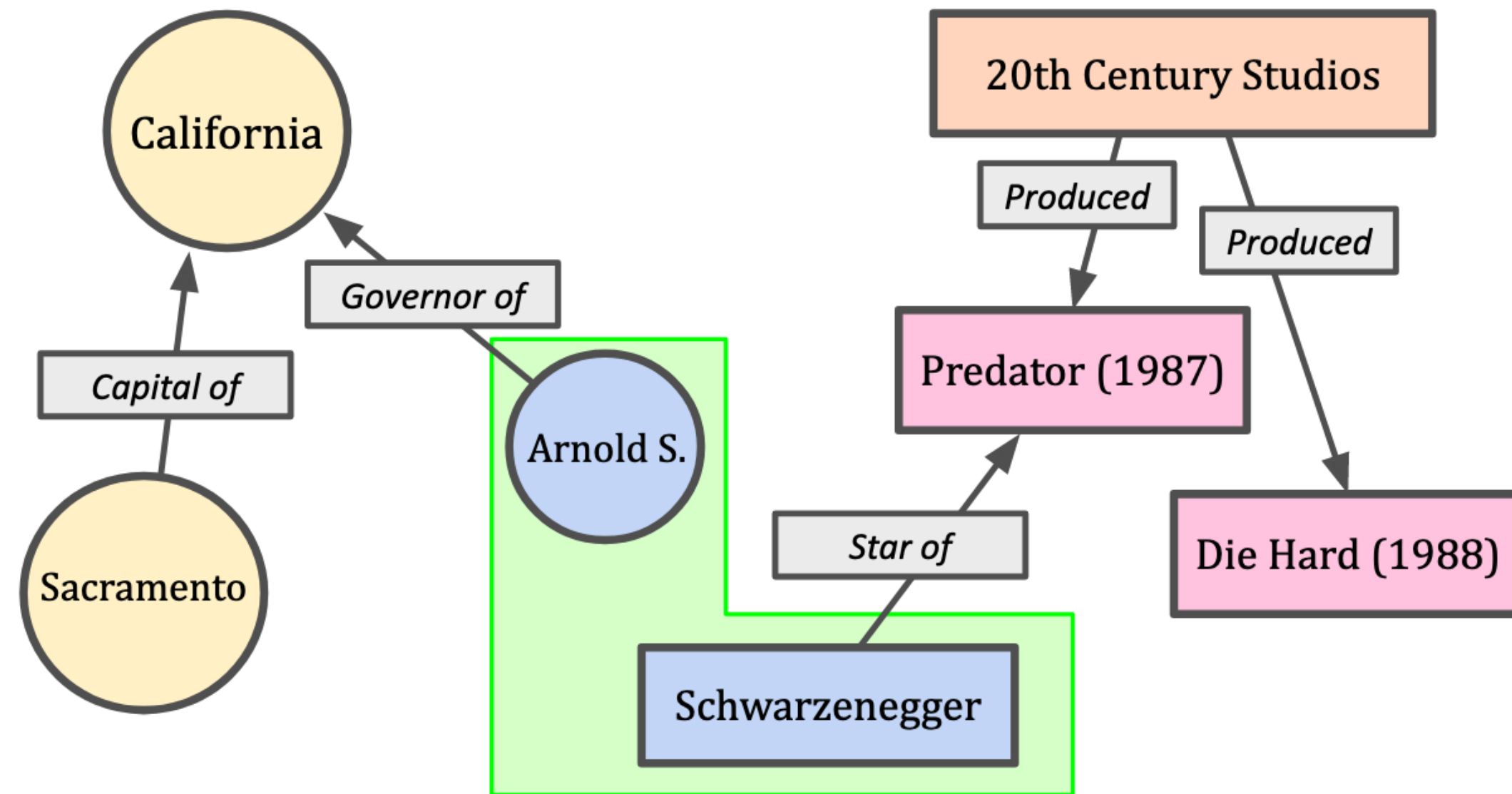
Пример задачи: рекомендации фильмов/товаров



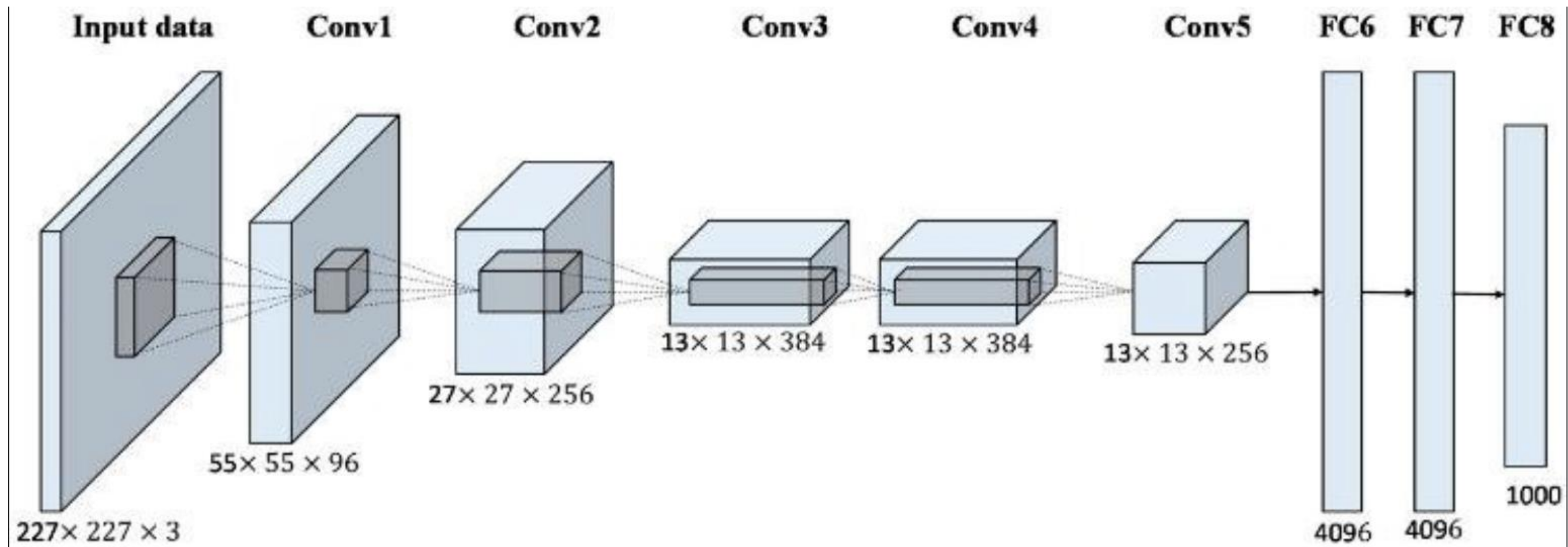
Что будет если объединить user-user-item-item граф?



knowledge graph

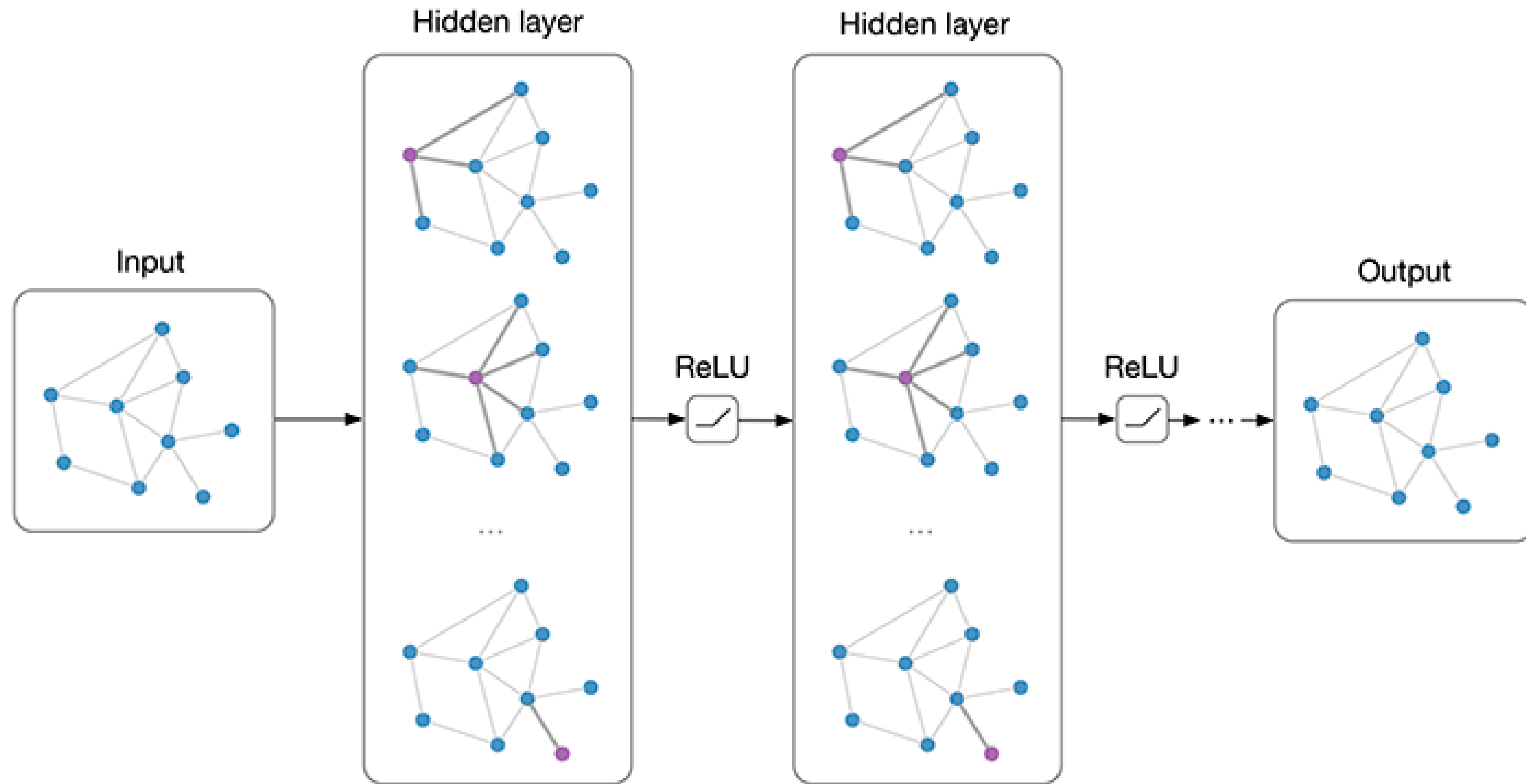


Графовые свертки



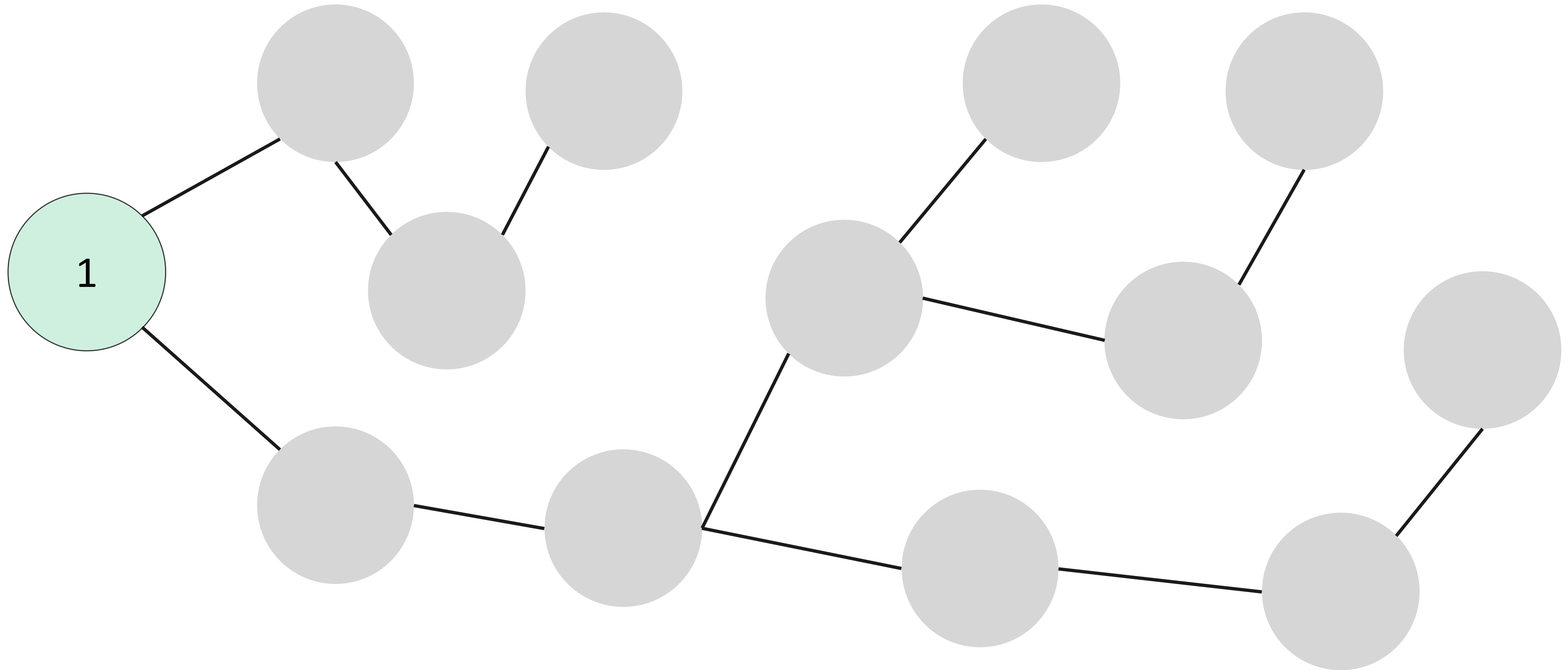
AlexNet

Графовые свертки

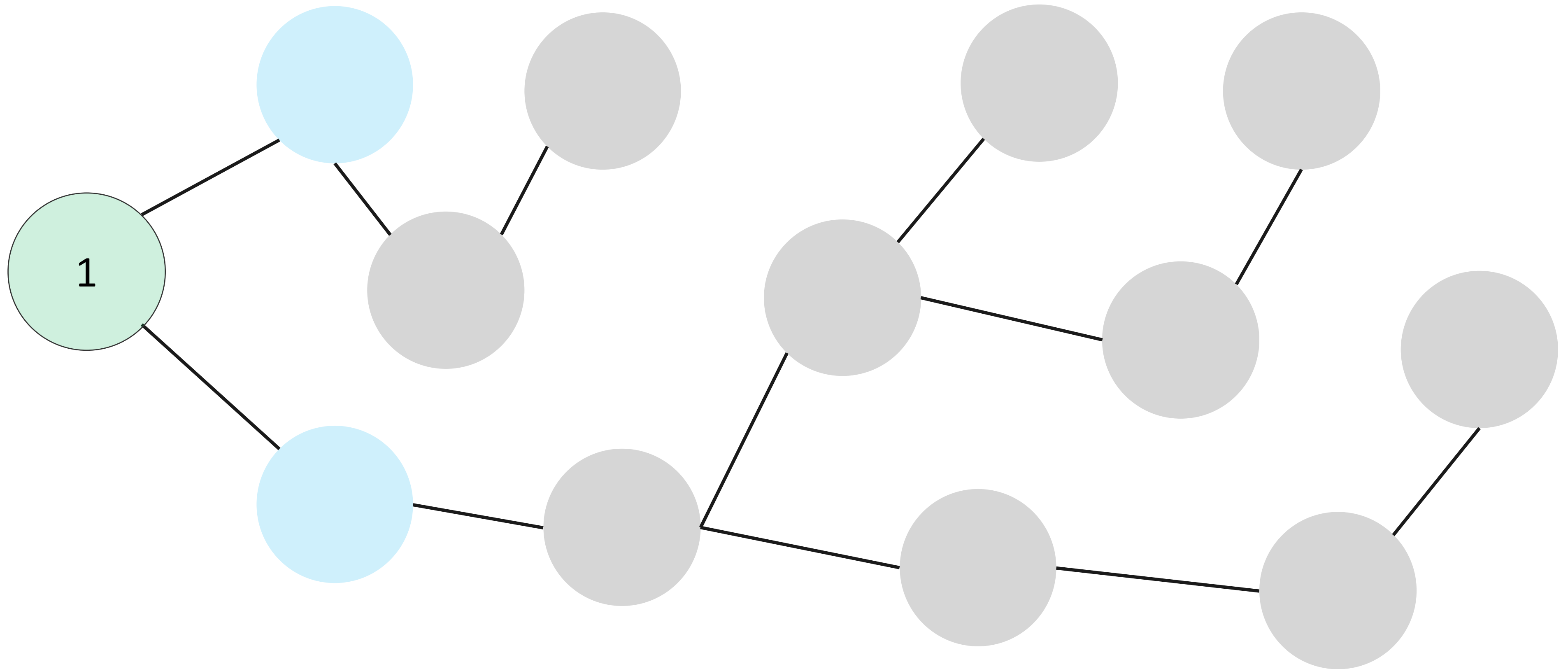


графовая свертка

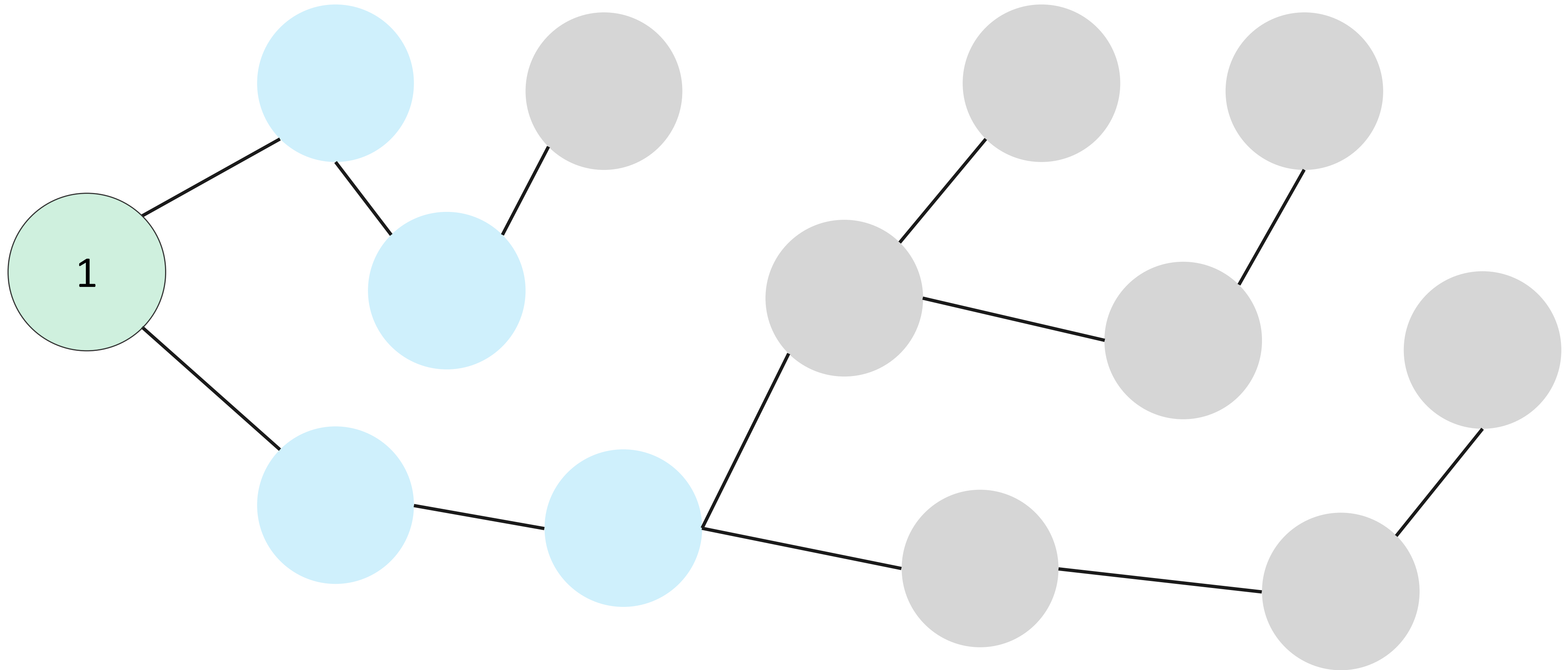
Графовые свертки. Receptive field



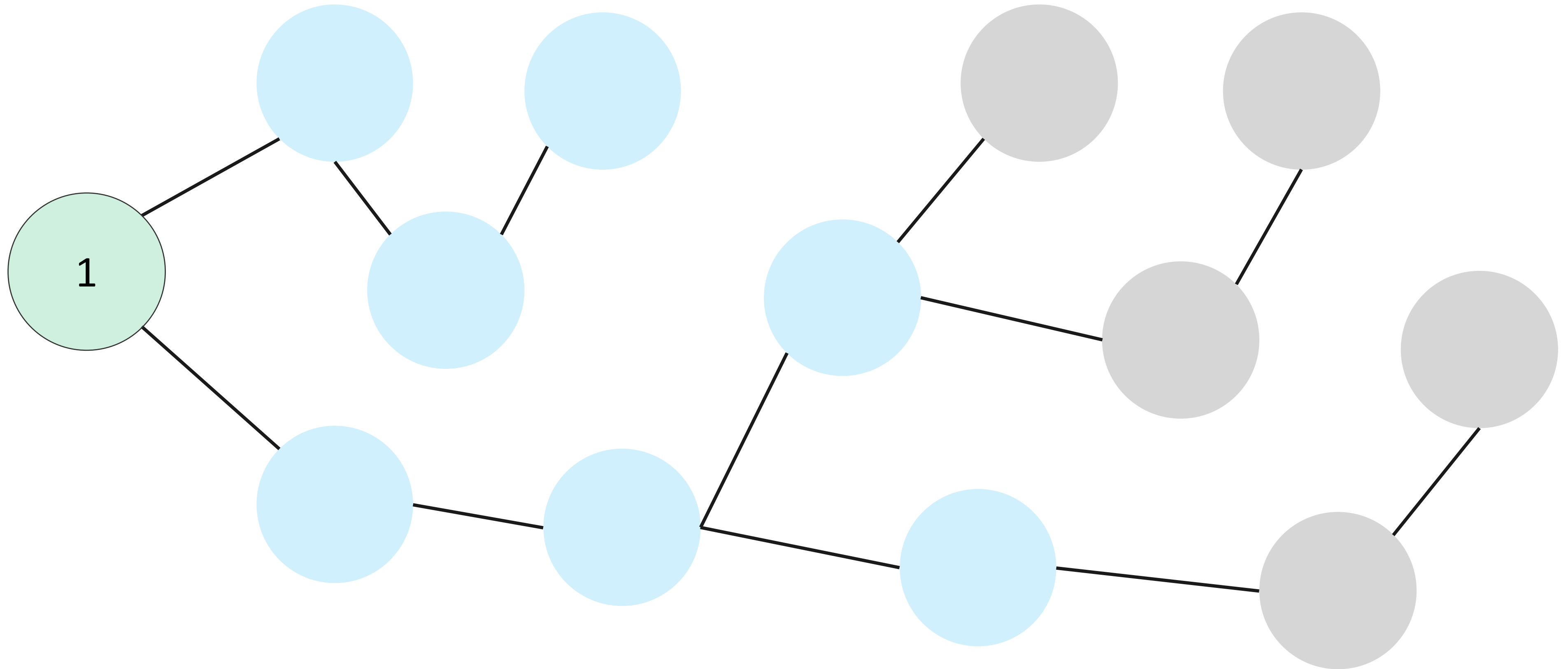
Графовые свертки. Receptive field



Графовые свертки. Receptive field

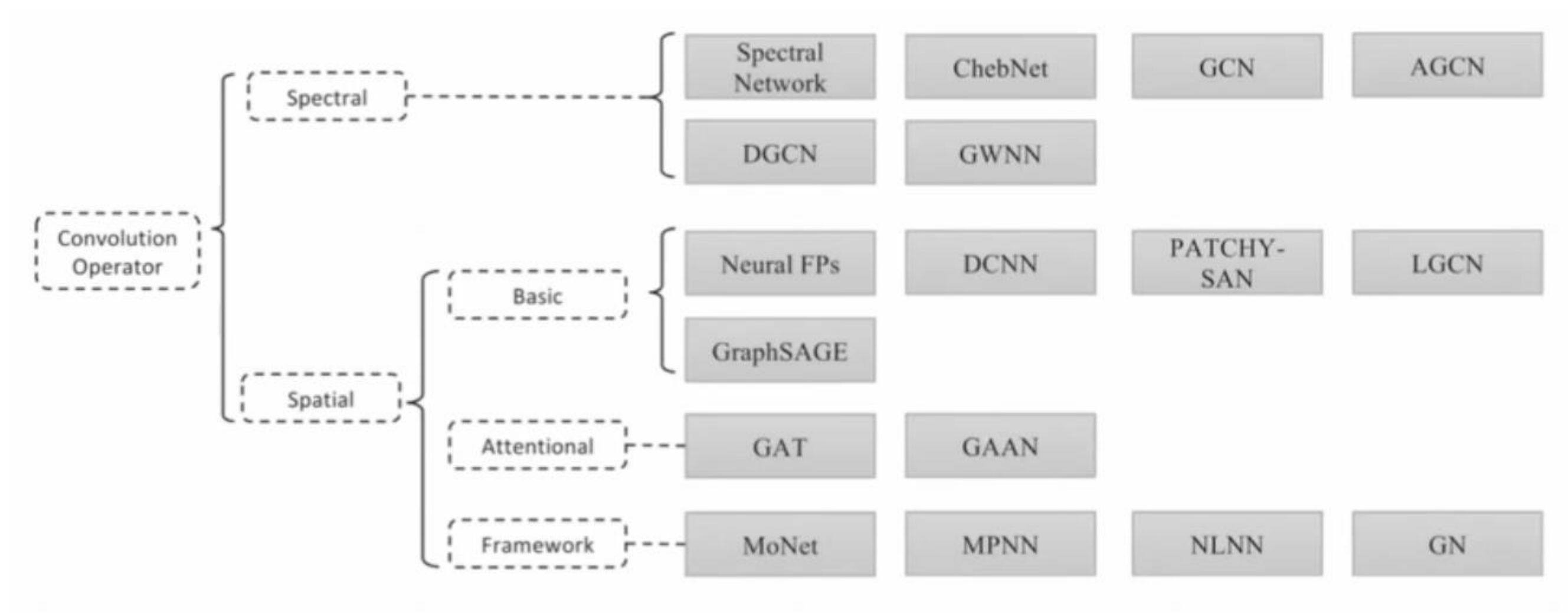


Графовые свертки. Receptive field



N сверток собирают информацию с соседей, до которых можно дойти не больше чем за N шагов

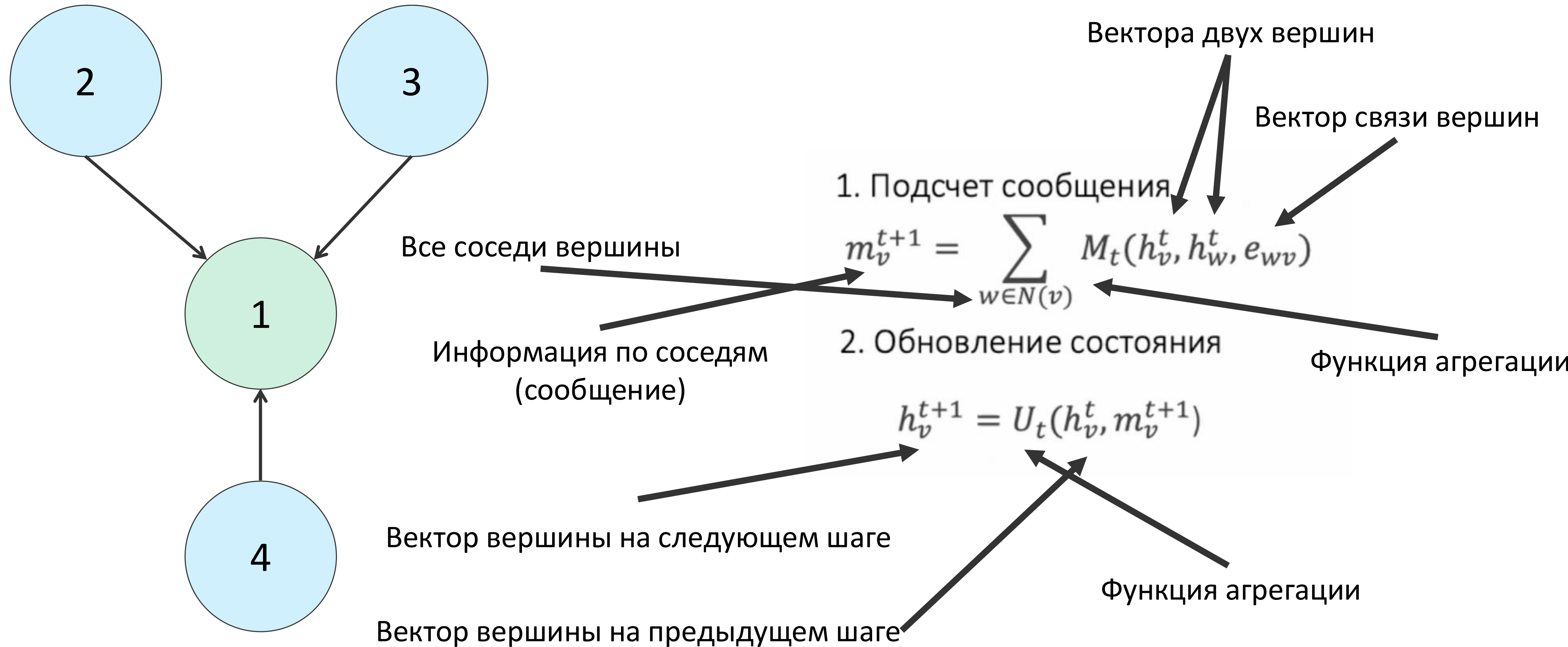
Графовые свертки. Парадигмы



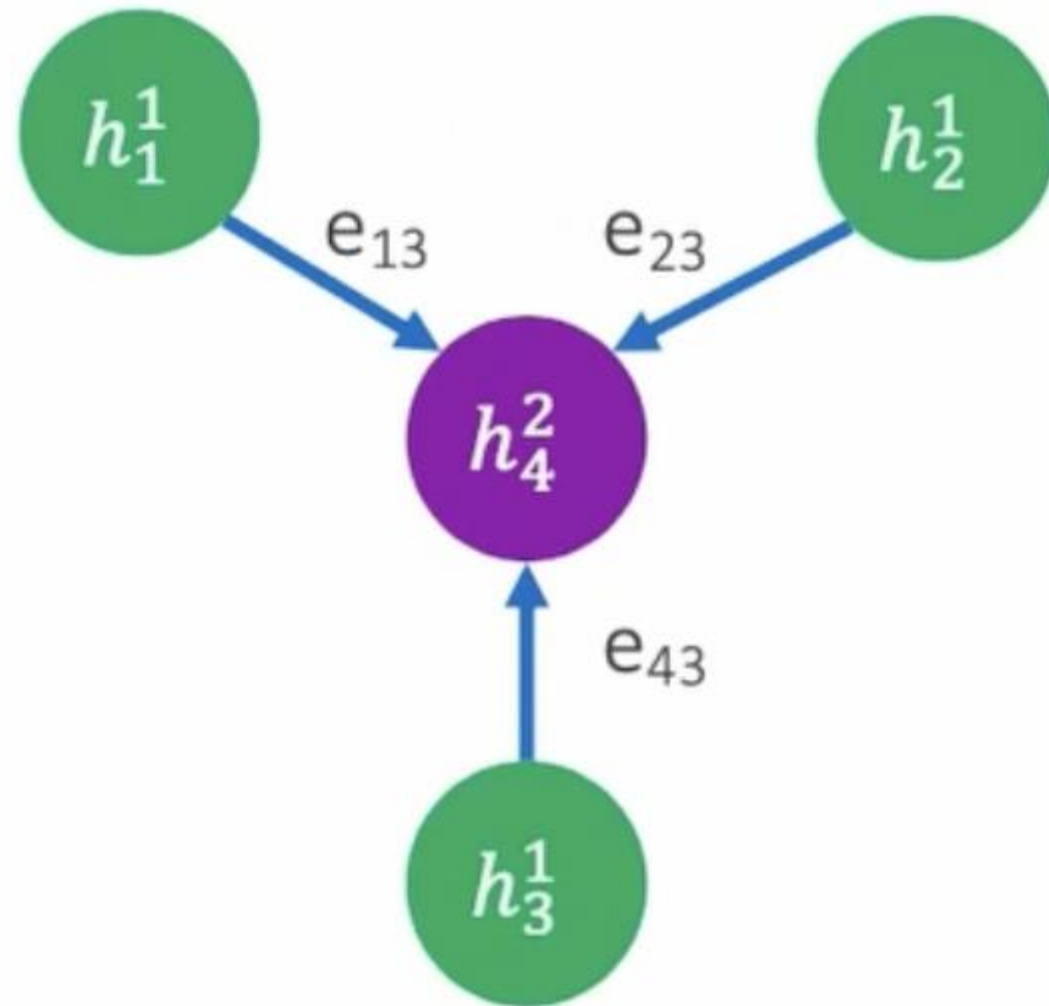
Парадигмы построения сверток:

- **передача сообщений**(локальная)
 - использует локальную окрестность вершины
- **спектральная парадигма**(глобальная)
 - использует глобальное положение вершины в графе

Графовые свертки. Передача сообщений



Графовые свертки. Передача сообщений



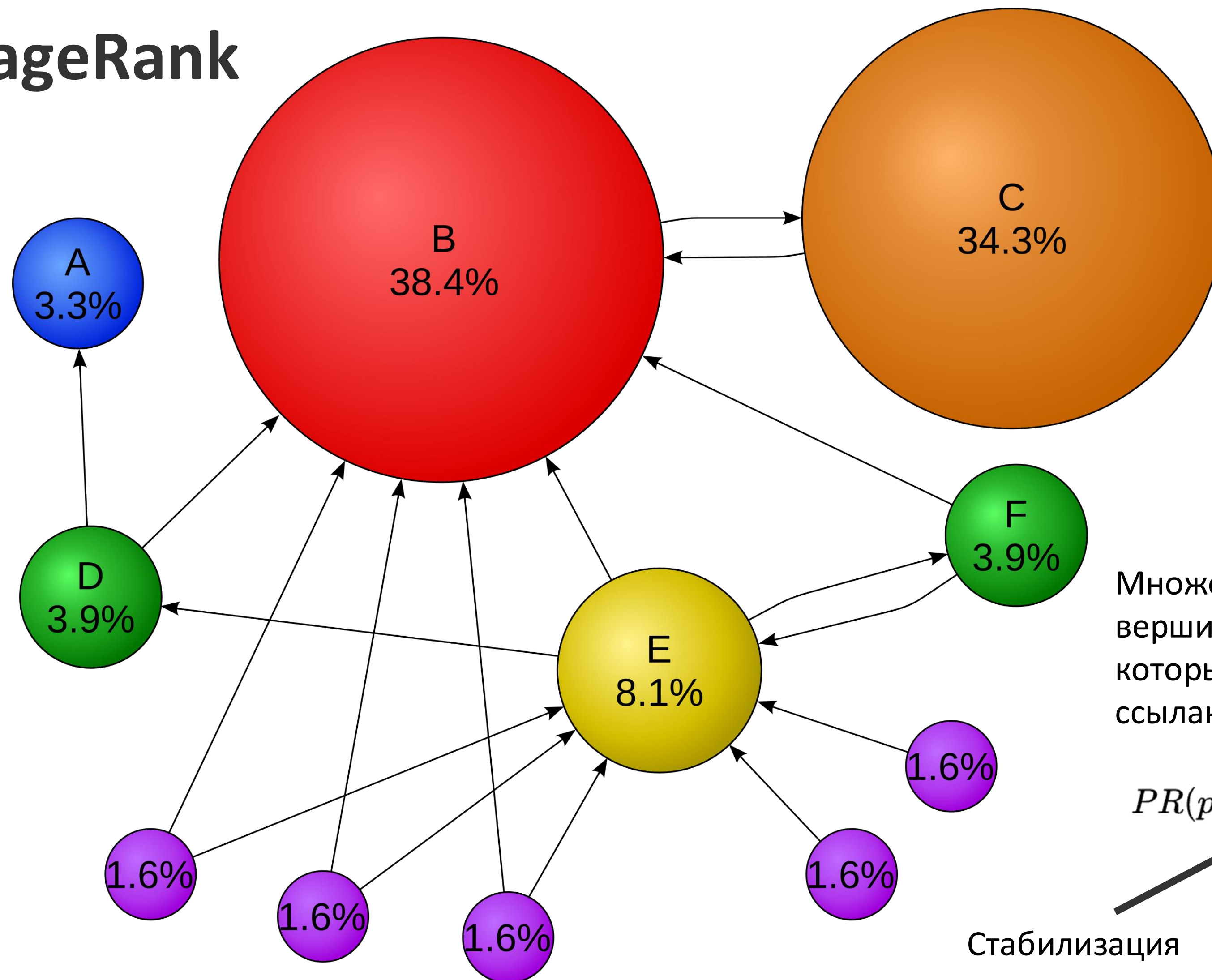
1. Подсчет сообщения

$$m_4^2 = M_t(h_1^1, h_4^1, e_{14}) + M_t(h_2^1, h_4^1, e_{24}) + M_t(h_3^1, h_4^1, e_{34})$$

2. Обновление состояния

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

PageRank



PR входящей
страницы

$$PR(u) = \sum_{v \in B_u} \frac{PR(v)}{L(v)},$$

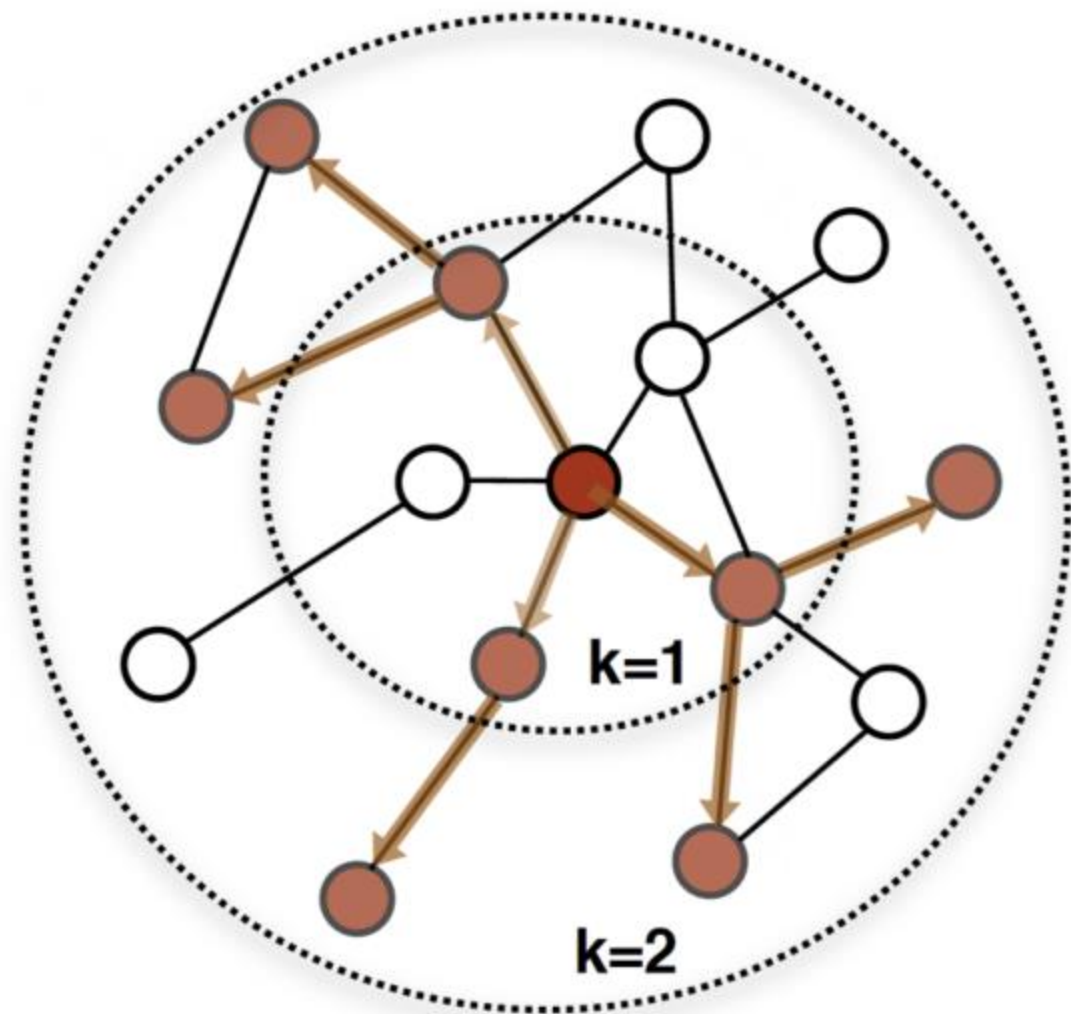
Множество
вершин,
которые
ссылаются на u

Количество
исходящих
ссылок

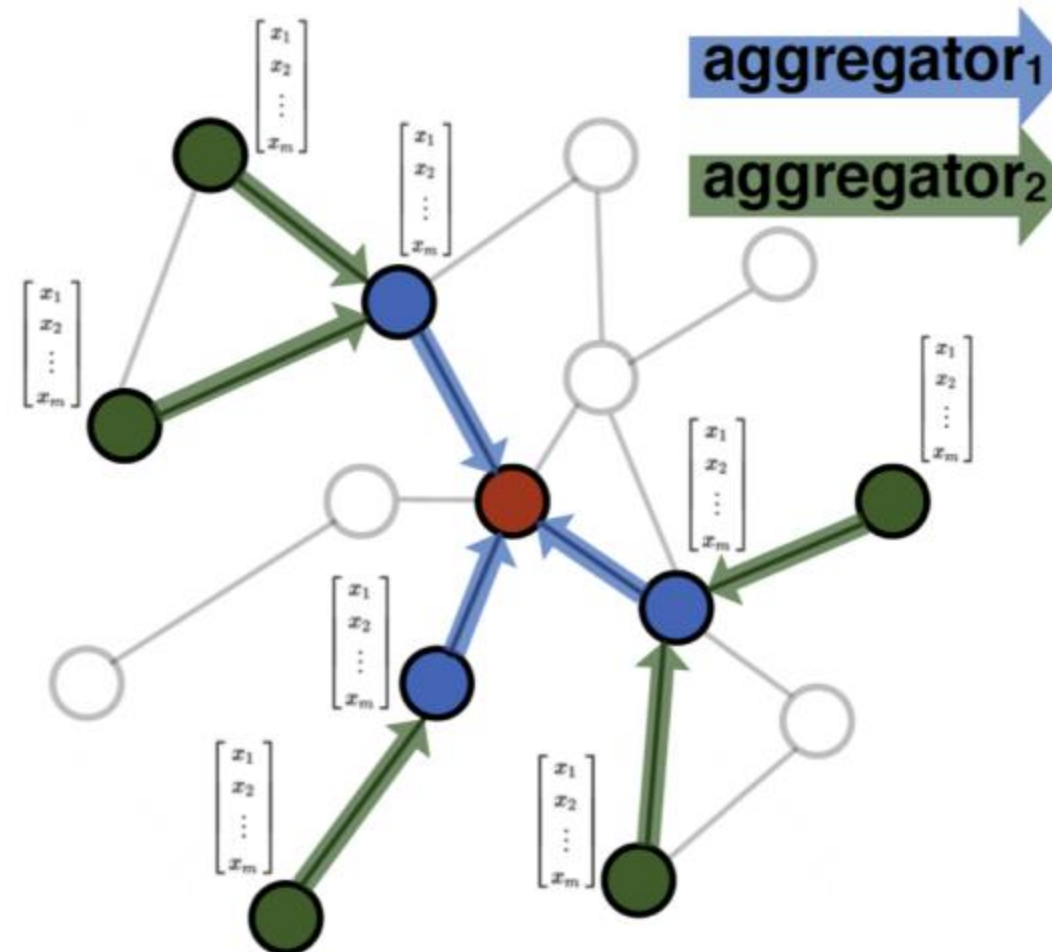
$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

Стабилизация Число вершин

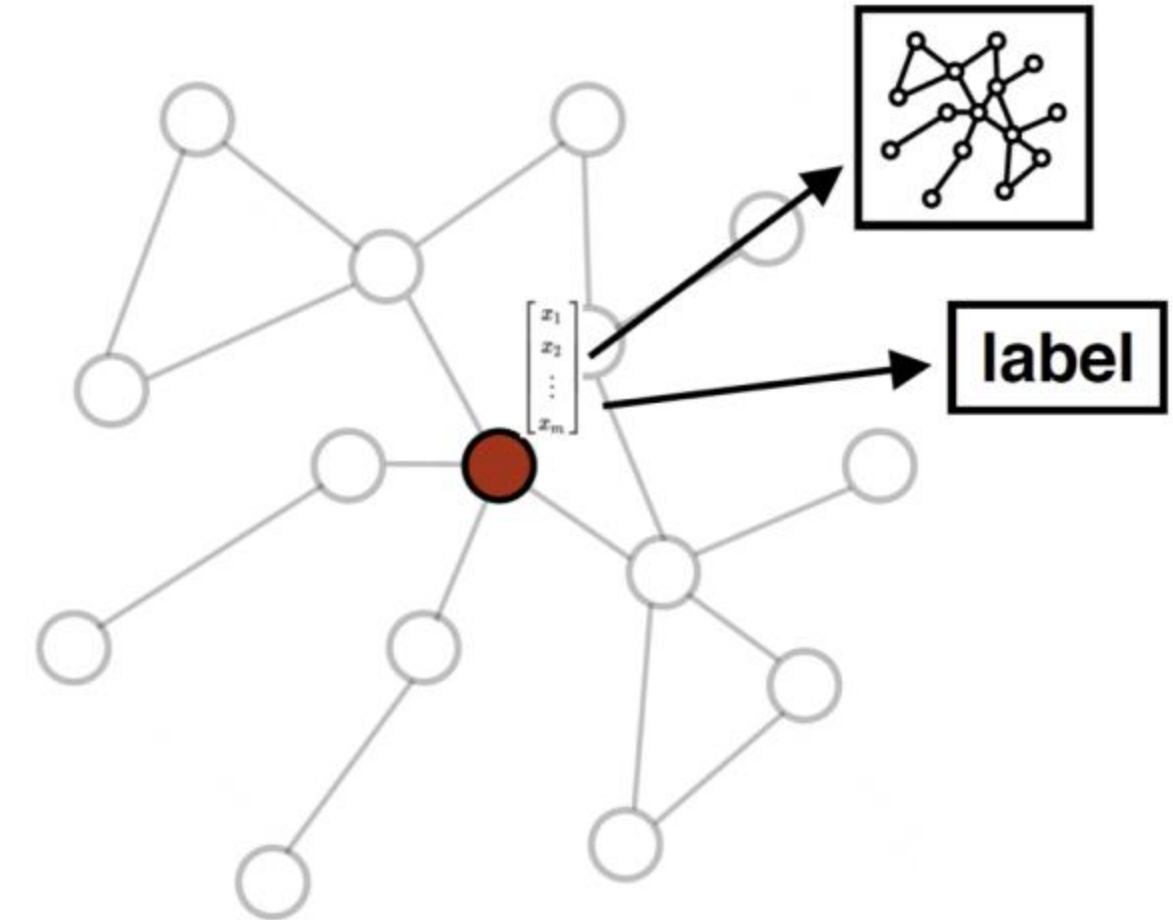
GraphSage



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

Сообщение

Функция агрегации соседей

Предыдущий
вектор ноды

$$\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\});$$

Соседи

$$\mathbf{h}_v^k \leftarrow \sigma(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k))$$

Обучаемая матрица

Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K ; weight matrices $\mathbf{W}^k, \forall k \in \{1, \dots, K\}$; non-linearity σ ; differentiable aggregator functions $\text{AGGREGATE}_k, \forall k \in \{1, \dots, K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

```
1  $\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$  ;
2 for  $k = 1 \dots K$  do
3   for  $v \in \mathcal{V}$  do
4      $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ ;
5      $\mathbf{h}_v^k \leftarrow \sigma \left( \mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k) \right)$ 
6   end
7    $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2, \forall v \in \mathcal{V}$ 
8 end
9  $\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$ 
```

Как обучается GraphSage?

$$J_G(\mathbf{z}_u) = -\log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - Q \cdot \mathbb{E}_{v_n \sim P_n(v)} \log(\sigma(-\mathbf{z}_u^\top \mathbf{z}_{v_n}))$$

Эмбеддинг вершины

Эмбеддинг соседа v к u

Насэмплированные негативы

Эмбеддинг негативного соседа

Число негативных сэмплов

Unsupervised сетап: выучили эмбеддинги, потом можем классифицировать

GraphSage



Table 1: Prediction results for the three datasets (micro-averaged F1 scores). Results for unsupervised and fully supervised GraphSAGE are shown. Analogous trends hold for macro-averaged scores.

Name	Citation		Reddit		PPI	
	Unsup. F1	Sup. F1	Unsup. F1	Sup. F1	Unsup. F1	Sup. F1
Random	0.206	0.206	0.043	0.042	0.396	0.396
Raw features	0.575	0.575	0.585	0.585	0.422	0.422
DeepWalk	0.565	0.565	0.324	0.324	—	—
DeepWalk + features	0.701	0.701	0.691	0.691	—	—
GraphSAGE-GCN	0.742	0.772	0.908	0.930	0.465	0.500
GraphSAGE-mean	0.778	0.820	0.897	0.950	0.486	0.598
GraphSAGE-LSTM	0.788	0.832	0.907	0.954	0.482	0.612
GraphSAGE-pool	0.798	0.839	0.892	0.948	0.502	0.600
% gain over feat.	39%	46%	55%	63%	19%	45%

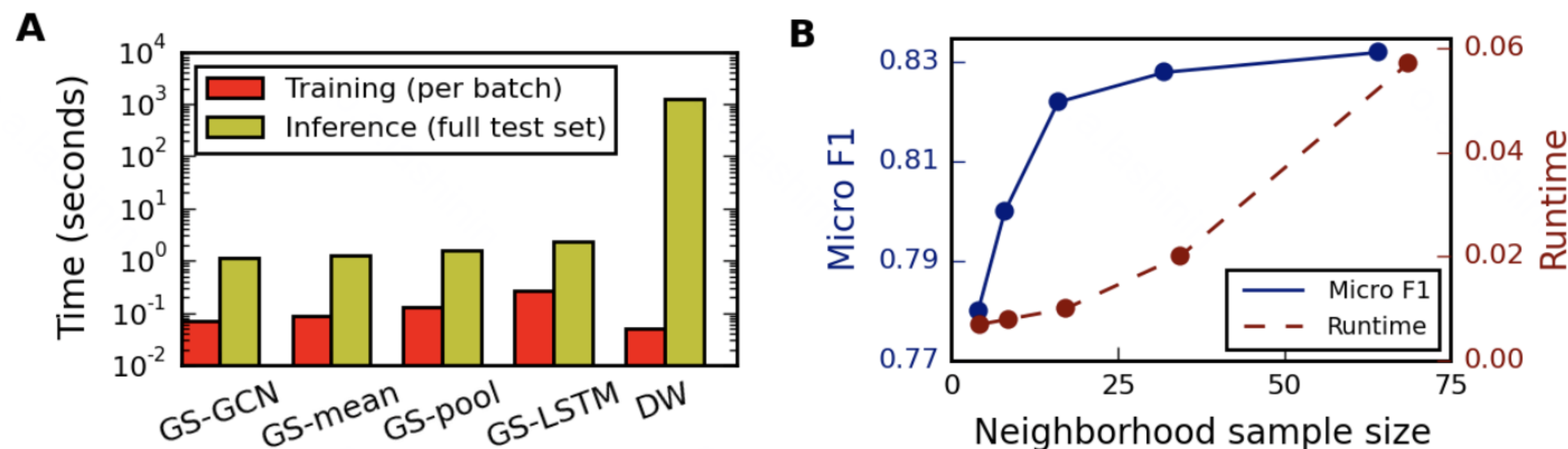
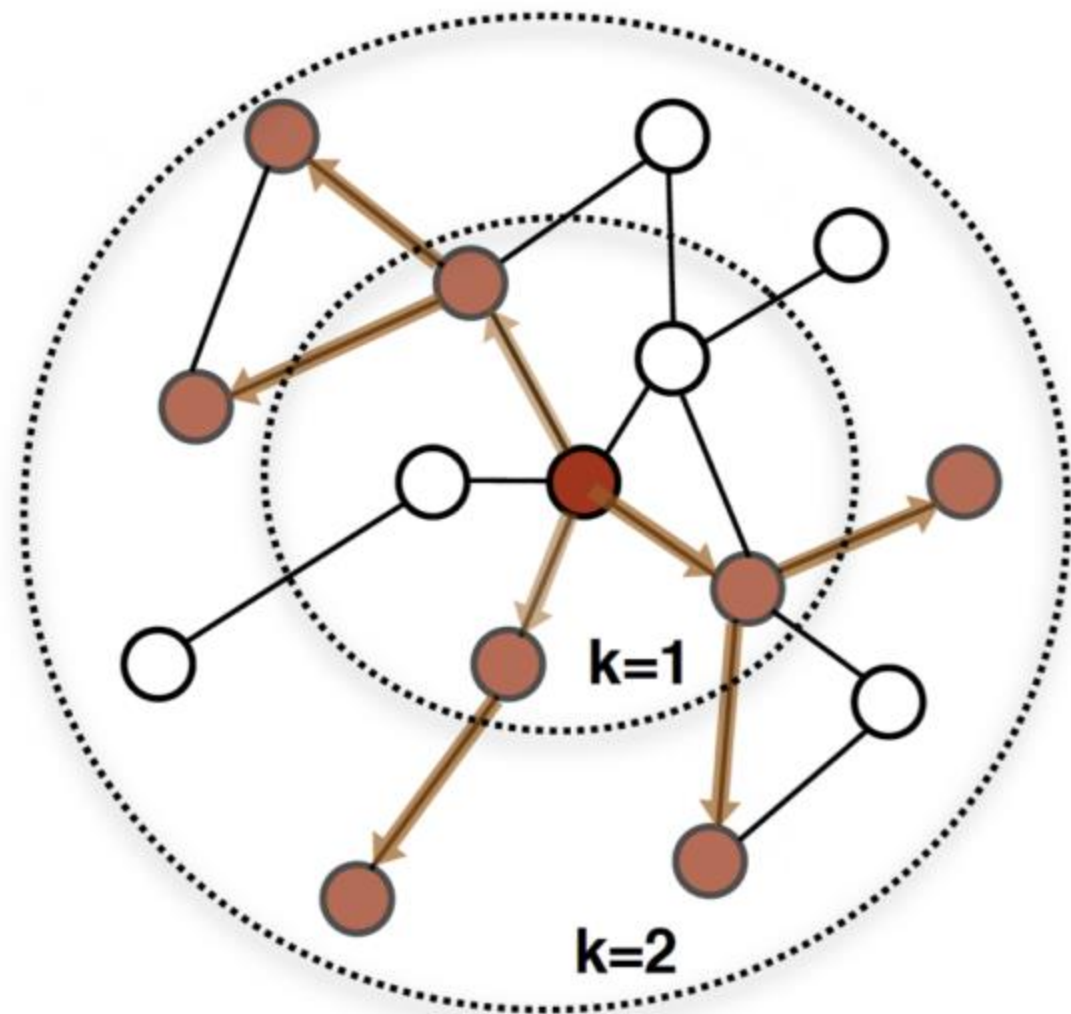
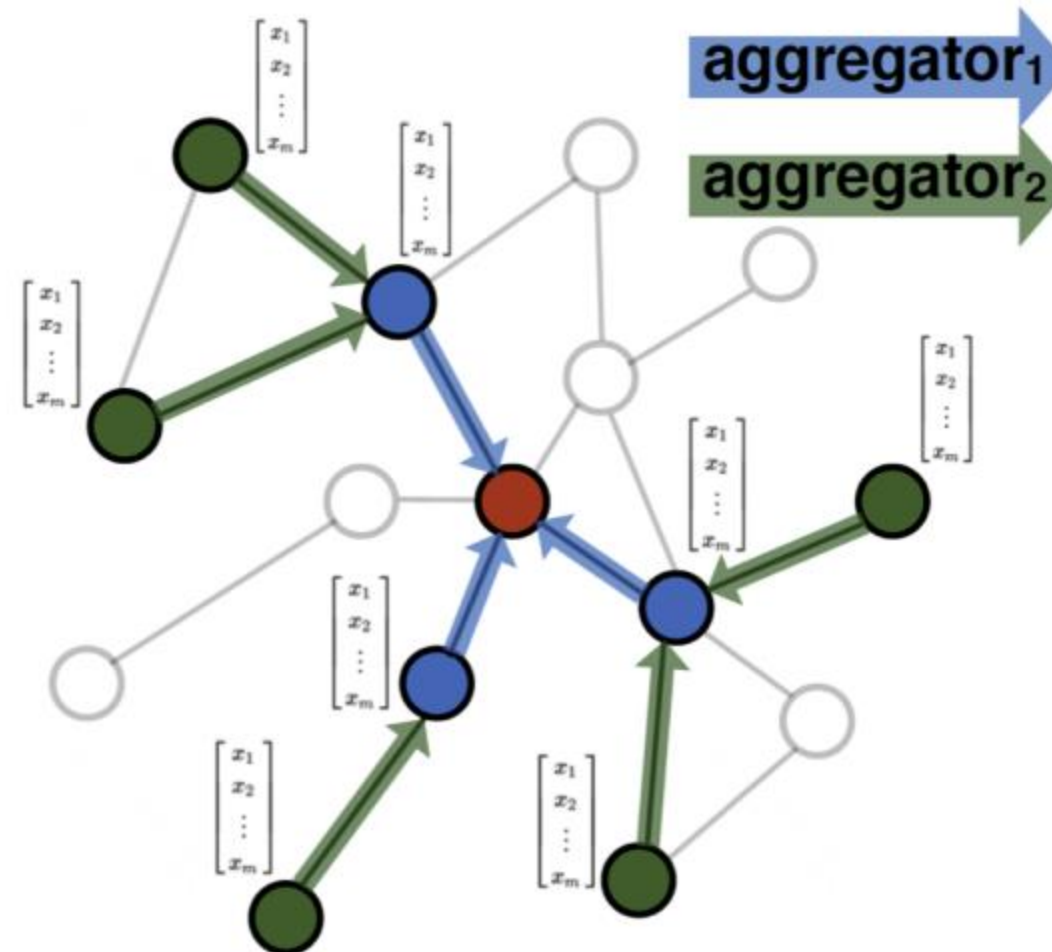


Figure 2: **A:** Timing experiments on Reddit data, with training batches of size 512 and inference on the full test set (79,534 nodes). **B:** Model performance with respect to the size of the sampled neighborhood, where the “neighborhood sample size” refers to the number of neighbors sampled at each depth for $K = 2$ with $S_1 = S_2$ (on the citation data using GraphSAGE-mean).

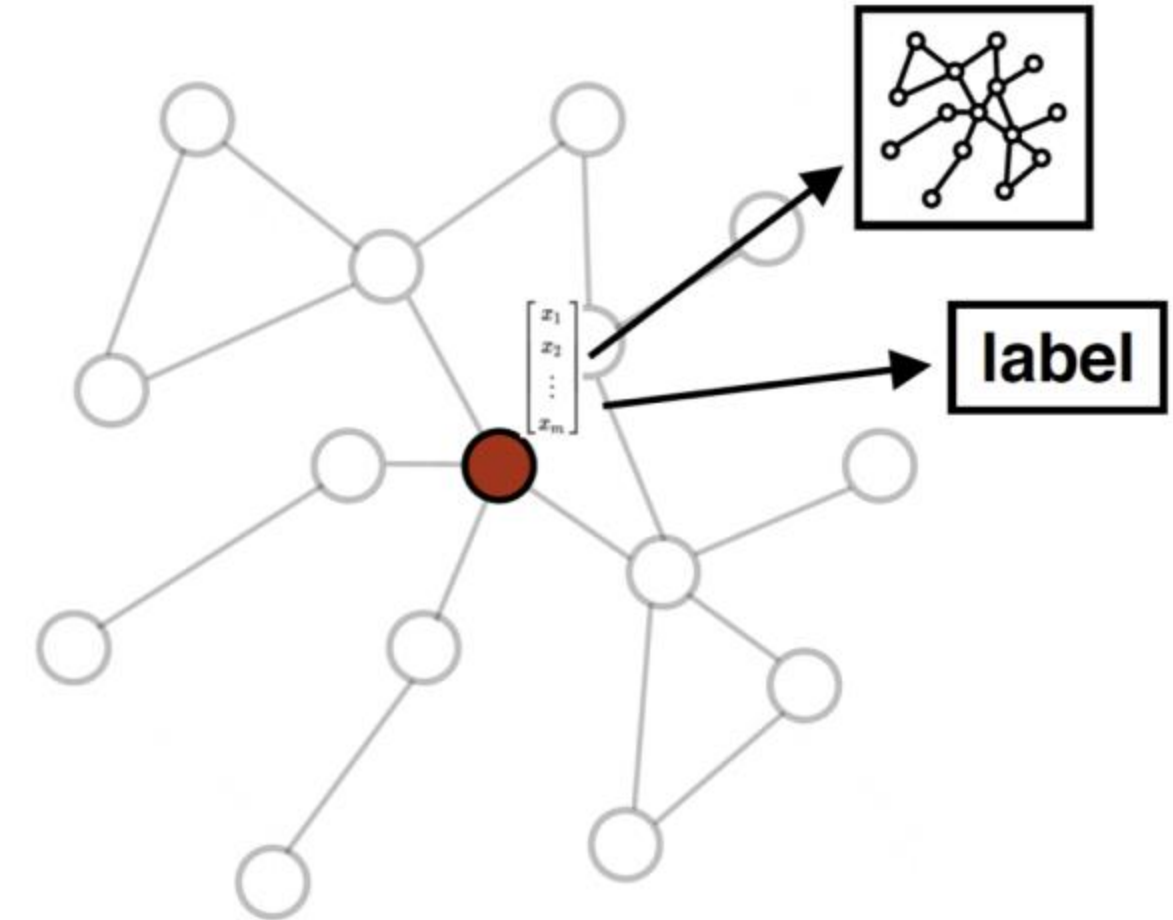
GraphSage: как можно еще агрегировать?



1. Sample neighborhood



2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

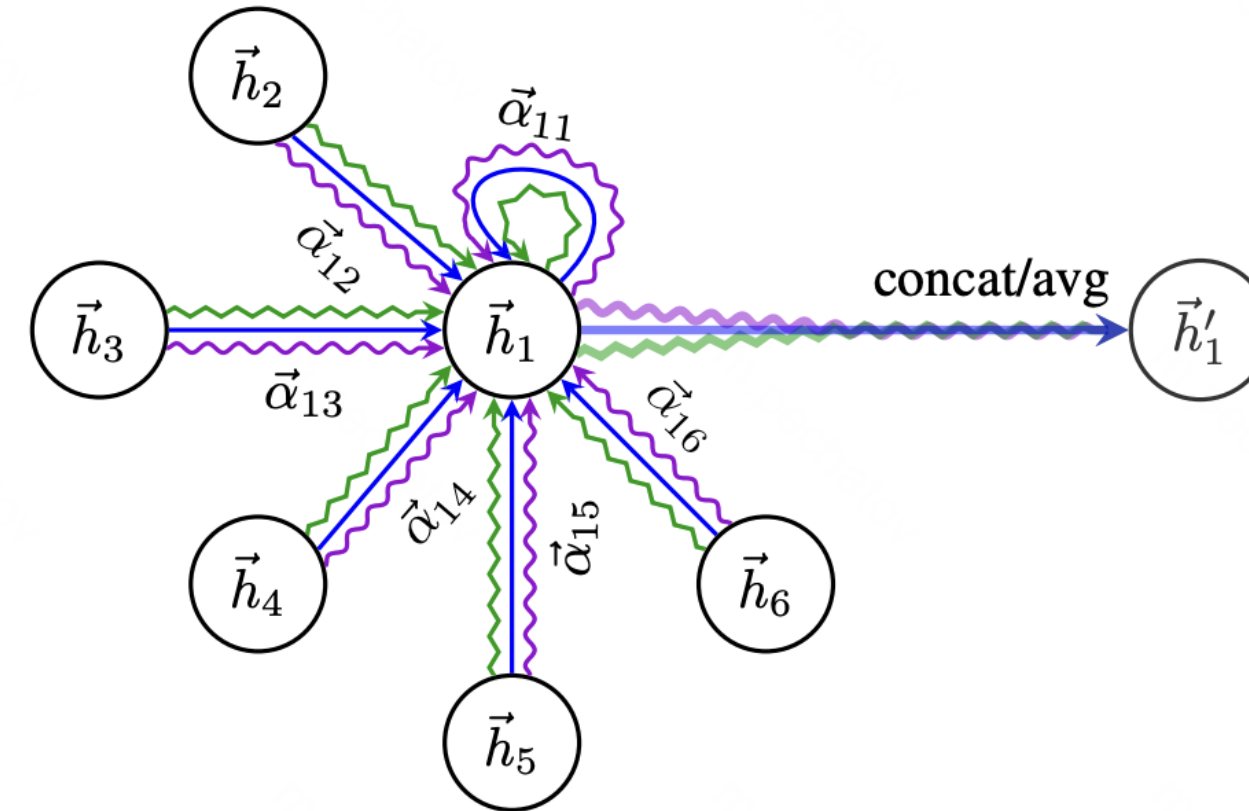
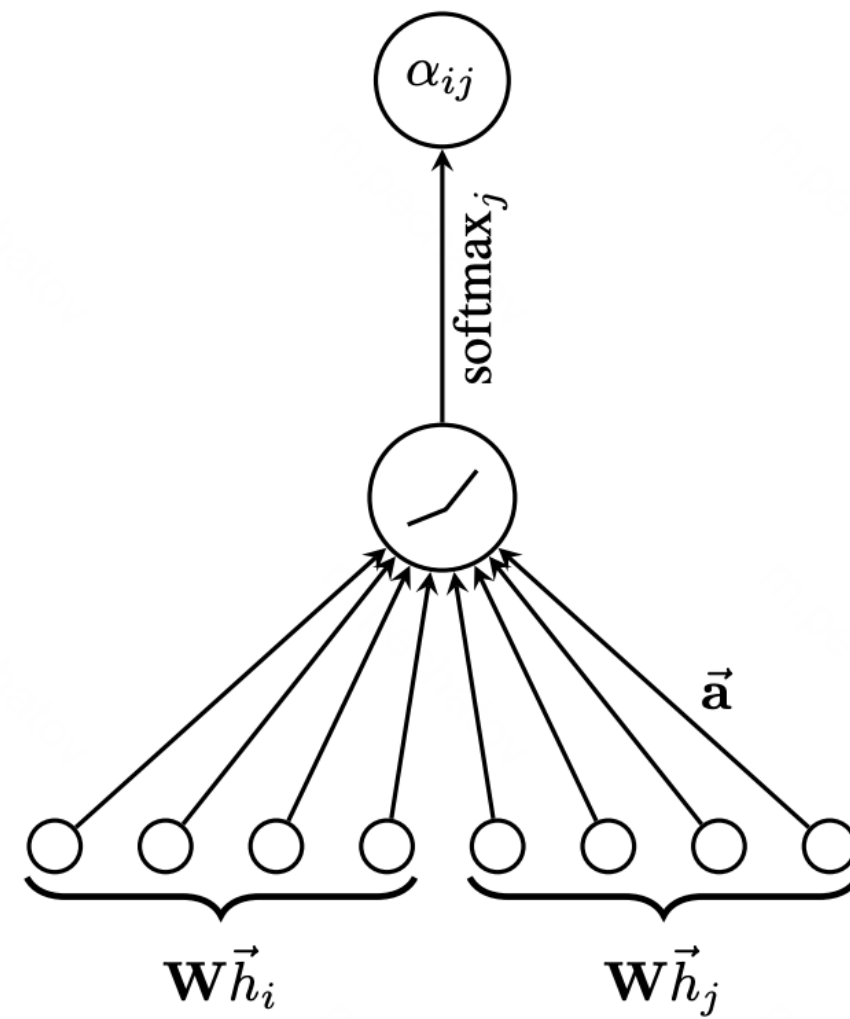
сообщение

$$\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\});$$

$$\mathbf{h}_v^k \leftarrow \sigma \left(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k) \right)$$

обучаемая матрица

GAT



Эмбеддинг якорной ноды

Эмбеддинг соседней ноды

$$e_{ij} = a(\mathbf{W}\vec{h}_i, \mathbf{W}\vec{h}_j) \quad \text{Простая нейросеть}$$

$$a : \mathbb{R}^{F'} \times \mathbb{R}^{F'} \rightarrow \mathbb{R}$$

$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

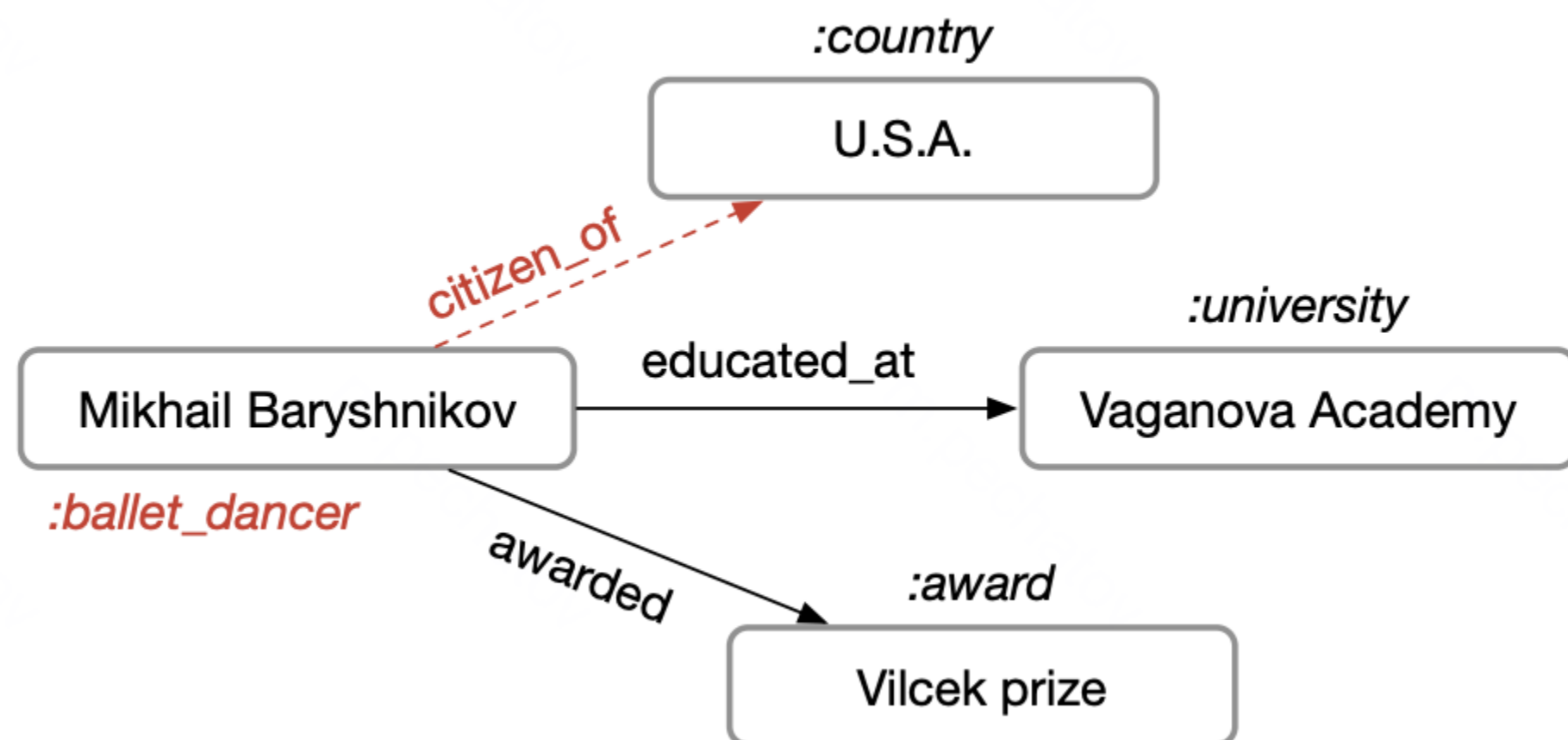
Взвешенная на attention сумма соседей

$$\vec{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W} \vec{h}_j \right).$$

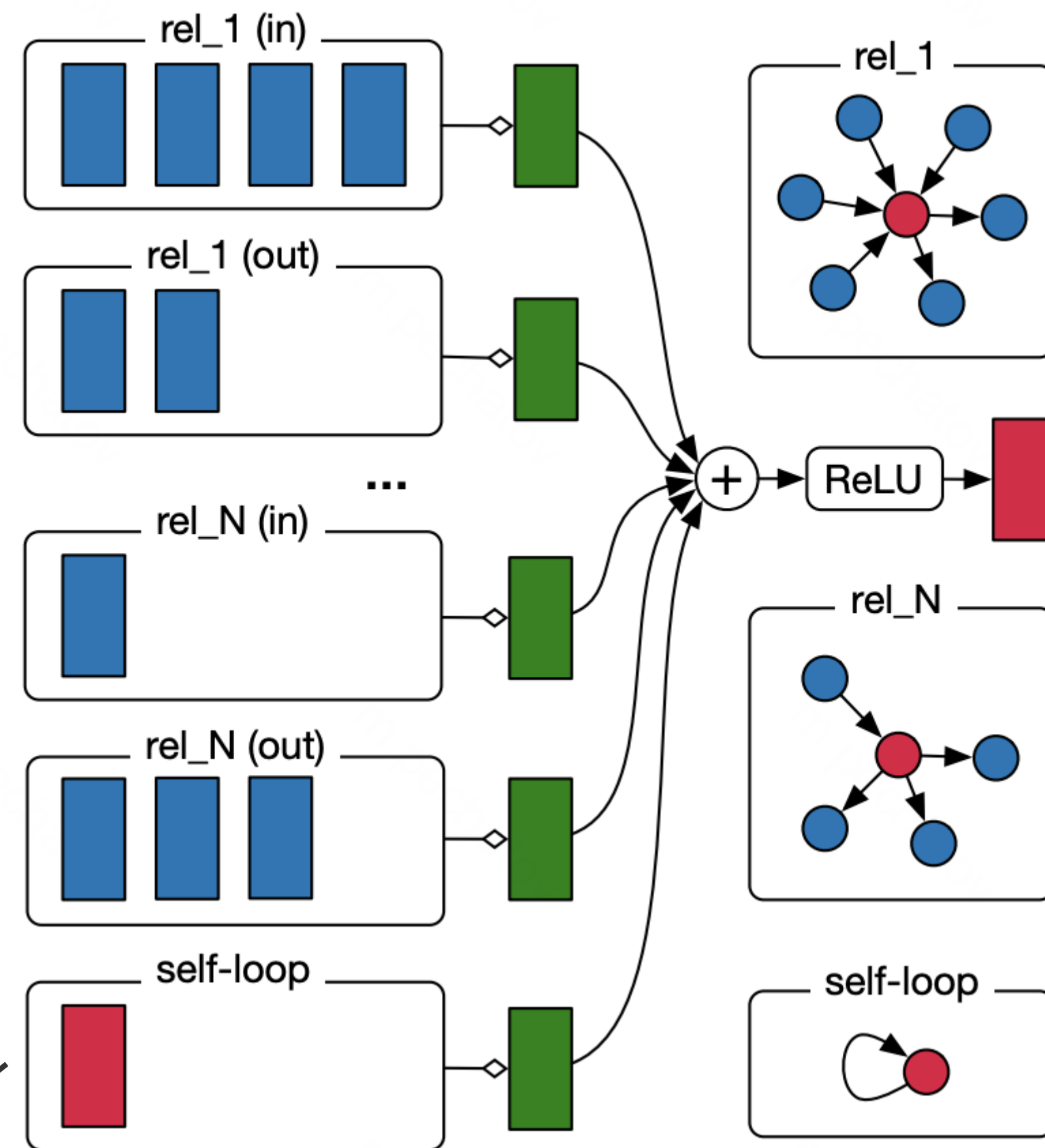
Несколько «голов» attention

$$\vec{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right)$$

RGCN

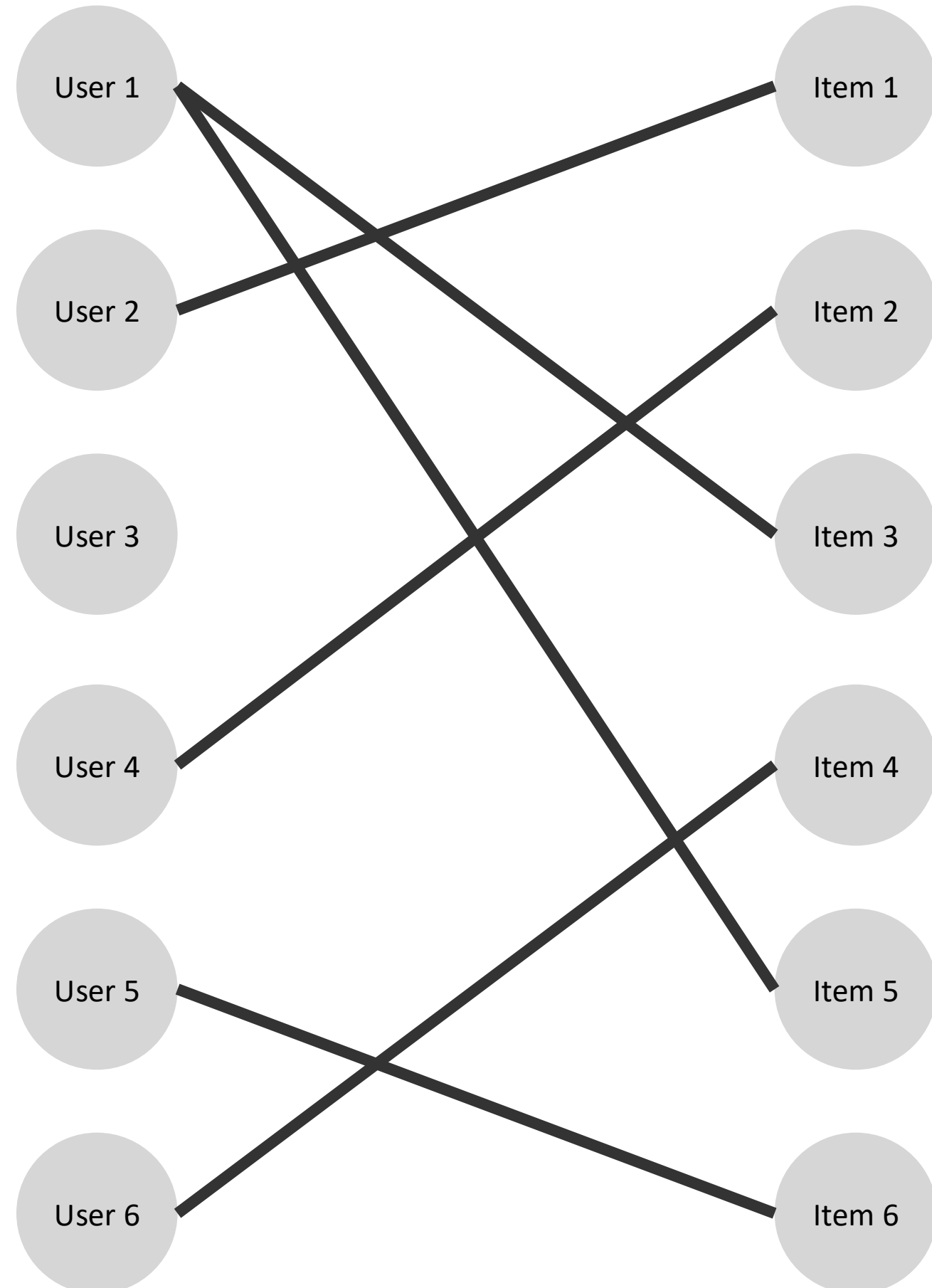


Разные типы ребер r



$$h_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

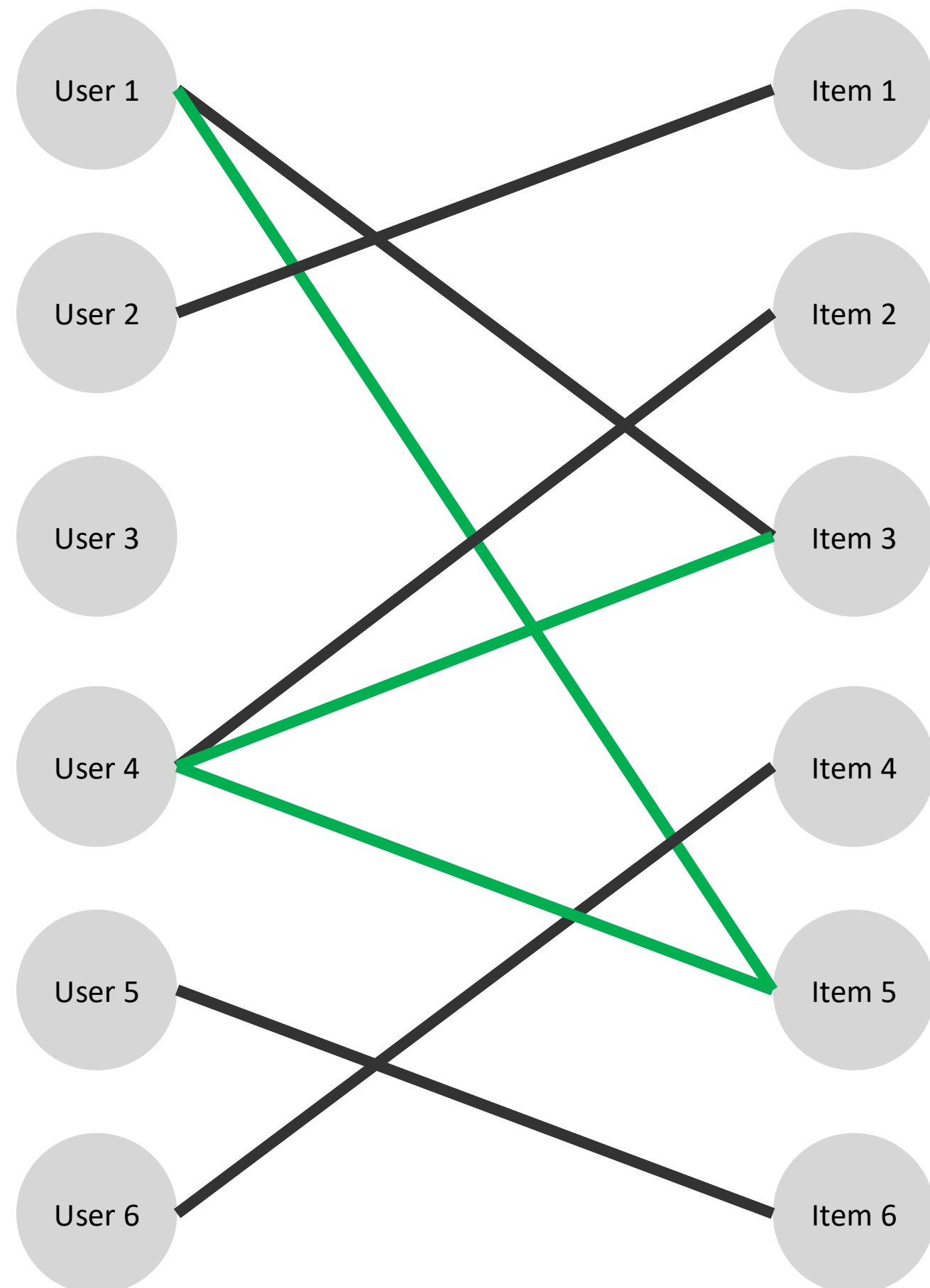
Двудольные графы и модели



Ключевые RecSys модели:

- P3alpha
- P3beta
- LigthGCN
- GFCF
- И другие модификации

P3Alpha



$$p_{ui} = (r_{ui} / N_u)^\alpha$$

Гиперпараметр

$$p_{iu} = (r_{ui} / N_i)^\alpha$$

Число ребер

Было ли взаимодействие u-i?

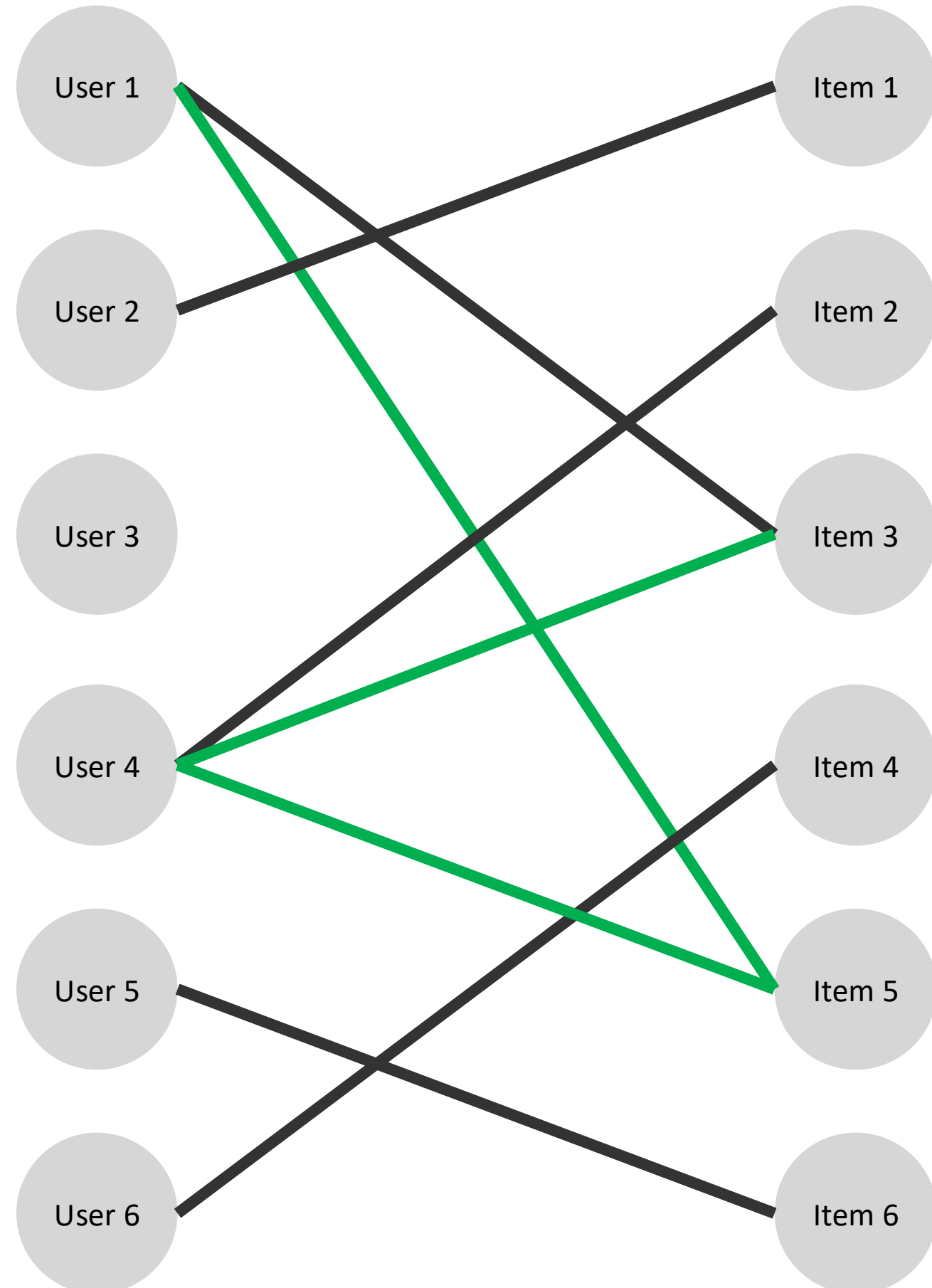
$$s_{ij} = \sum_v p_{jv} p_{vi}$$

Все юзеры в датасете

ItemKNN матрица похожести

По факту идет сортировка по числу возможных путей $u \rightarrow I$, но немного сглаженная

P3beta



$$p_{ui} = (r_{ui}/N_u)^\alpha$$

Гиперпараметр

$$p_{iu} = (r_{ui}/N_i)^\alpha$$

Число ребер

Было ли взаимодействие u-i?

$$\tilde{p}_{ui}^3 = \frac{p_{ui}^3}{d_{ii}^\beta}, \text{ where } \beta \in \mathbb{R} \text{ and } \beta > 0.0.$$

Популярность айтема

Это нужно, чтобы сделать debiasing модели от рекомендаций популярных товаров

Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches

Maurizio Ferrari Dacrema
Politecnico di Milano, Italy
maurizio.ferrari@polimi.it

Paolo Cremonesi
Politecnico di Milano, Italy
paolo.cremonesi@polimi.it

Dietmar Jannach
University of Klagenfurt, Austria
dietmar.jannach@aau.at

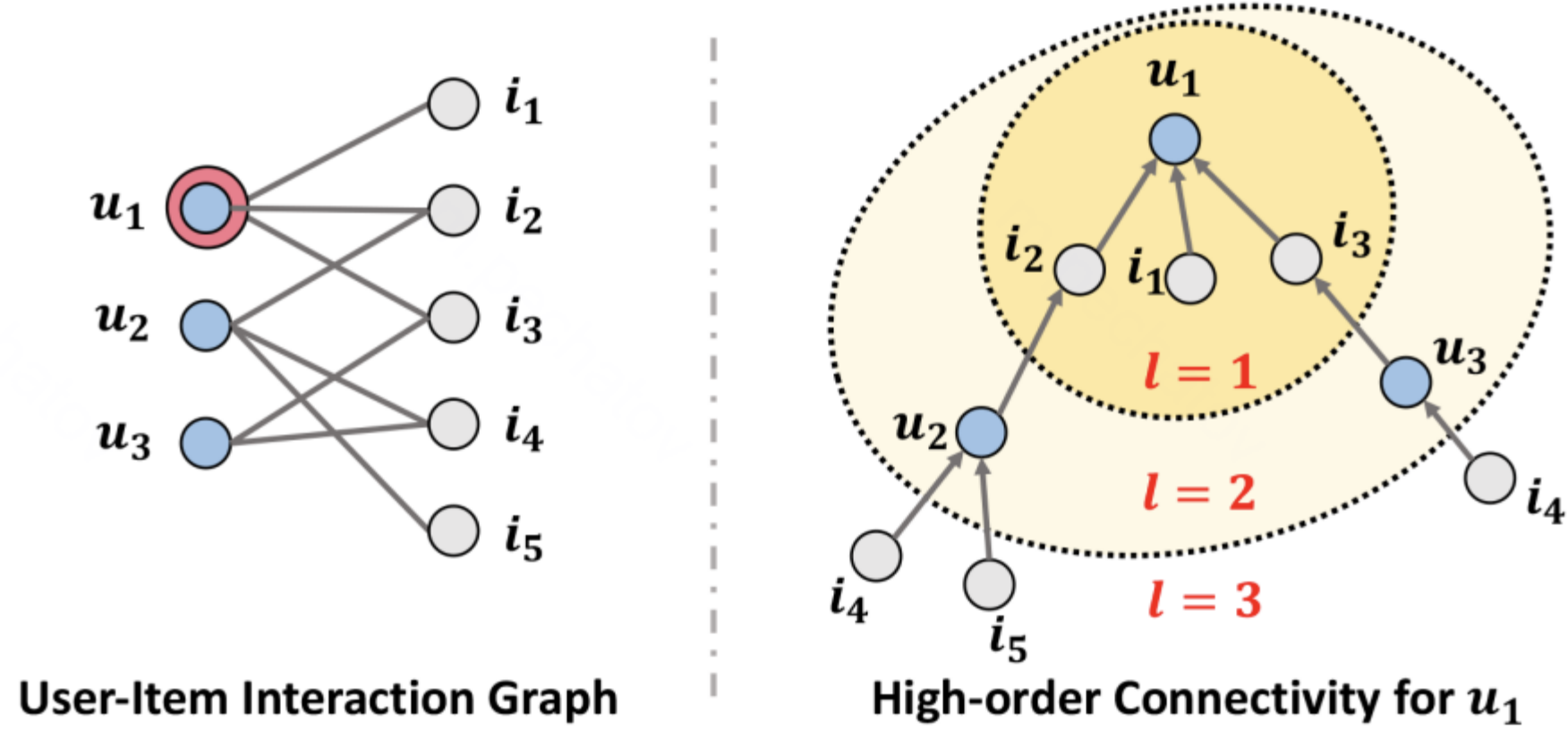
Table 6: Experimental results for NCF.

	Pinterest			
	HR@5	NDCG@5	HR@10	NDCG@10
TopPopular	0.1663	0.1065	0.2744	0.1412
UserKNN	0.7001	0.5033	0.8610	0.5557
ItemKNN	0.7100	0.5092	0.8744	0.5629
P ³ α	0.7008	0.5018	0.8667	0.5559
RP ³ β	0.7105	0.5116	0.8740	0.5650
NeuMF	0.7024	0.4983	0.8719	0.5536
	Movielens 1M			
	HR@5	NDCG@5	HR@10	NDCG@10
TopPopular	0.3043	0.2062	0.4531	0.2542
UserKNN	0.4916	0.3328	0.6705	0.3908
ItemKNN	0.4829	0.3328	0.6596	0.3900
P ³ α	0.4811	0.3331	0.6464	0.3867
RP ³ β	0.4922	0.3409	0.6715	0.3991
NeuMF	0.5486	0.3840	0.7120	0.4369
SLIM	0.5589	0.3961	0.7161	0.4470

Table 8: Experimental results for Mult-VAE (Netflix data), using metrics and cutoffs reported in the original paper.

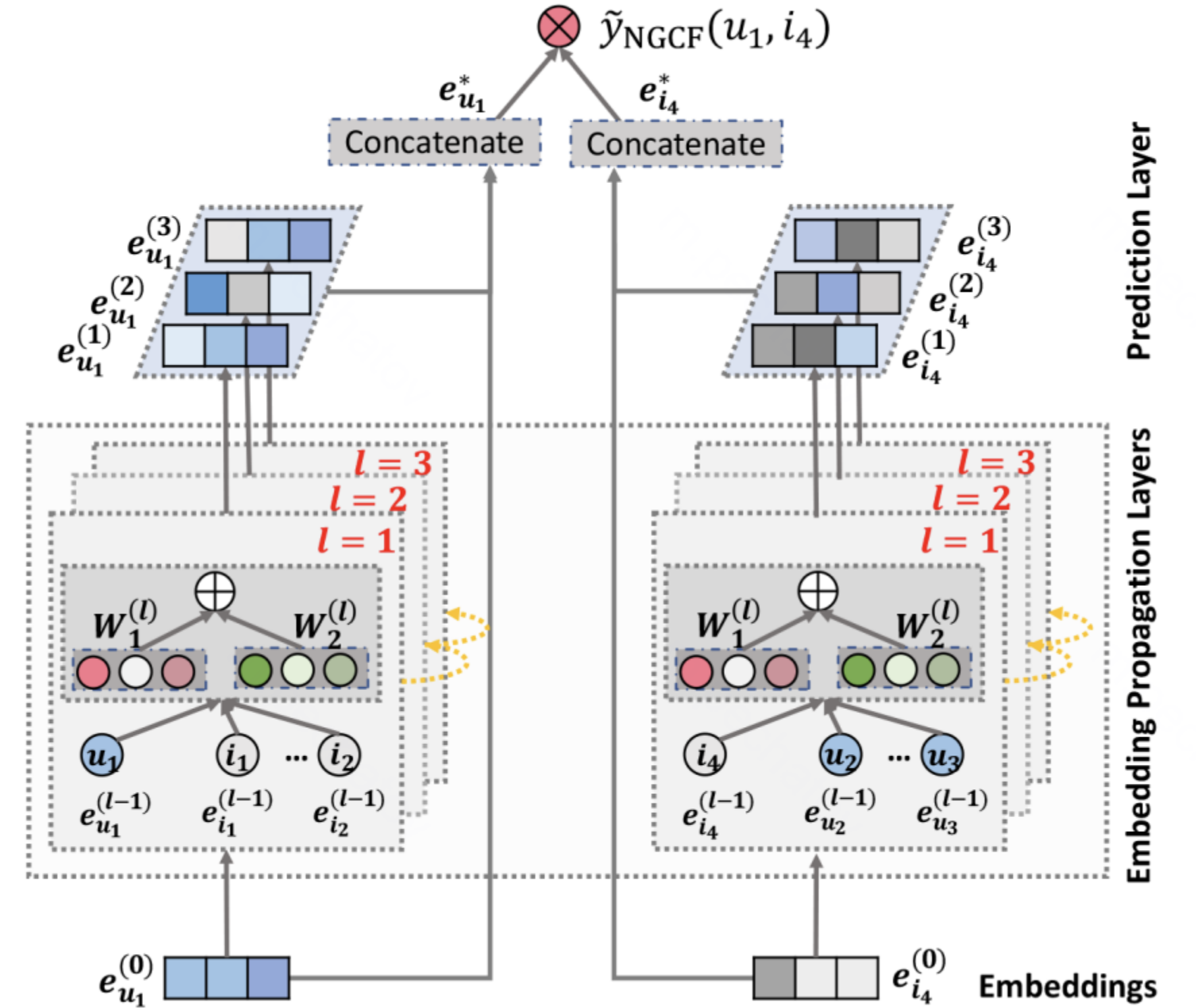
	REC@20	REC@50	NDCG@100
TopPop	0.0782	0.1643	0.1570
ItemKNN CF	0.2088	0.3386	0.3086
P ³ α	0.1977	0.3346	0.2967
RP ³ β	0.2196	0.3560	0.3246
SLIM	0.2551	0.3995	0.3745
Mult-VAE	0.2626	0.4138	0.3756

NGCF

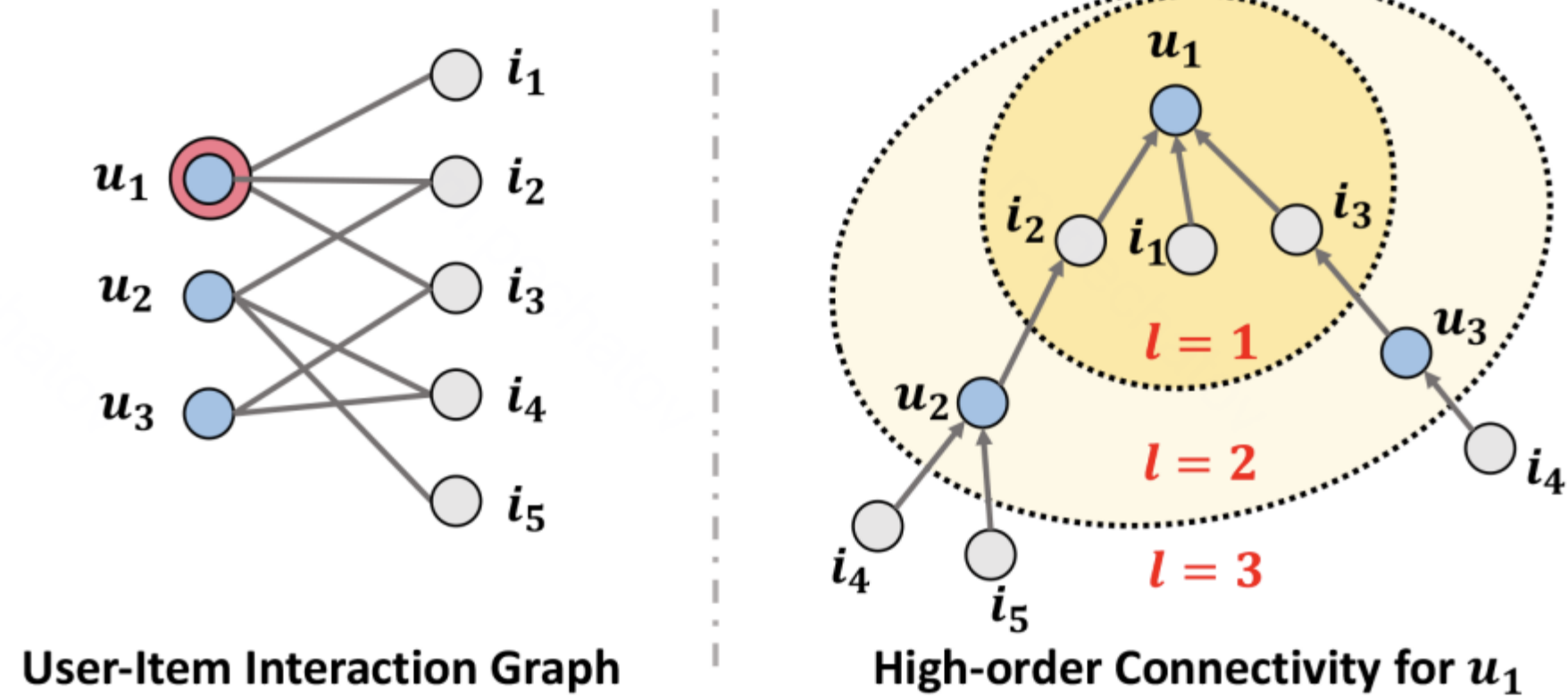


$$\mathbf{m}_{u \leftarrow i} = \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \left(\mathbf{W}_1 \mathbf{e}_i + \mathbf{W}_2 (\mathbf{e}_i \odot \mathbf{e}_u) \right),$$

$$\mathbf{e}_u^{(1)} = \text{LeakyReLU} \left(\mathbf{m}_{u \leftarrow u} + \sum_{i \in \mathcal{N}_u} \mathbf{m}_{u \leftarrow i} \right),$$



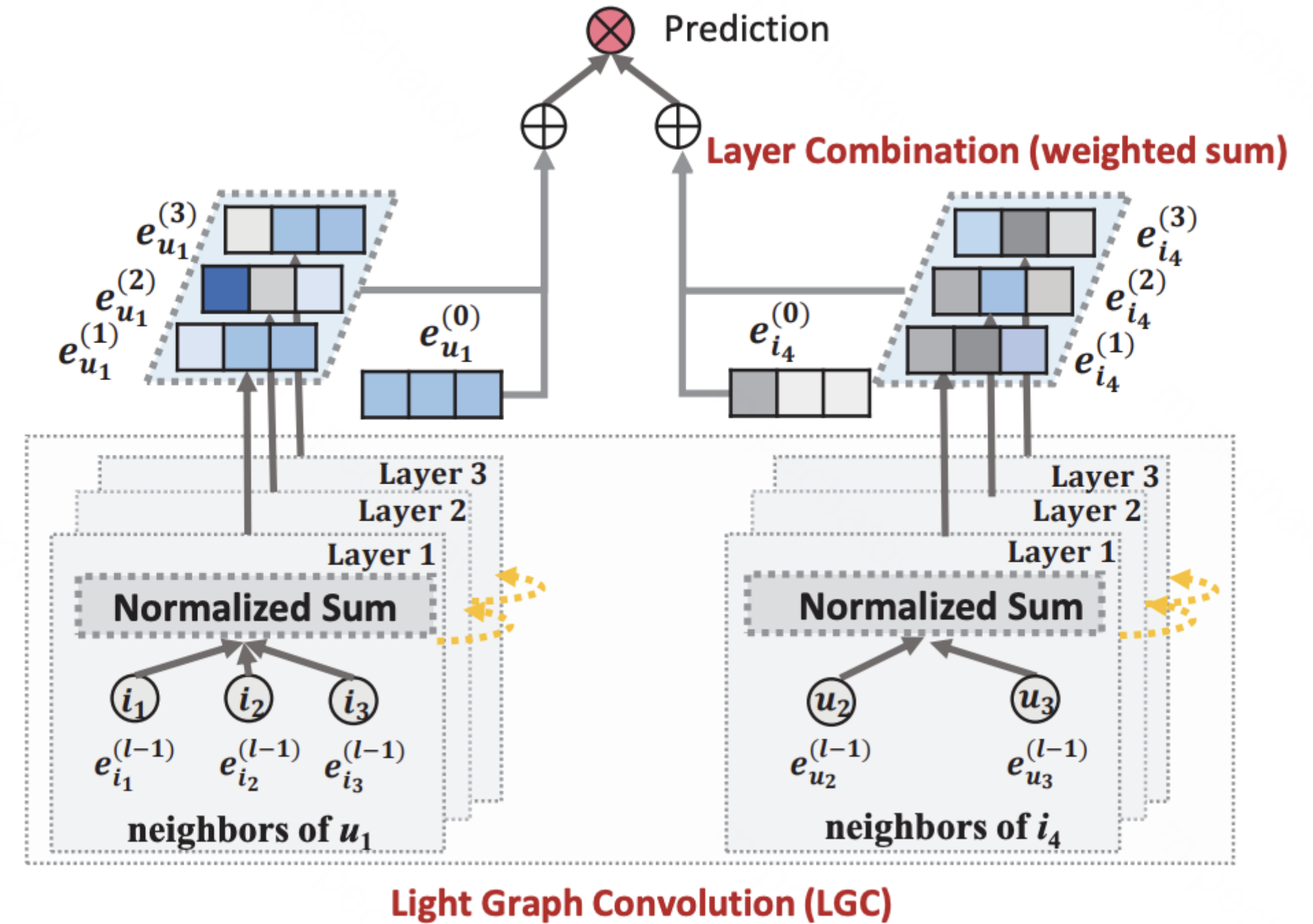
LightGCN



$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k)},$$

$$\mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i|} \sqrt{|\mathcal{N}_u|}} \mathbf{e}_u^{(k)}.$$

$$L_{BPR} = - \sum_{u=1}^M \sum_{i \in \mathcal{N}_u} \sum_{j \notin \mathcal{N}_u} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda ||\mathbf{E}^{(0)}||^2$$



$$\mathbf{e}_u = \sum_{k=0}^K \alpha_k \mathbf{e}_u^{(k)}; \quad \mathbf{e}_i = \sum_{k=0}^K \alpha_k \mathbf{e}_i^{(k)},$$

LightGCN можно не тренировать!

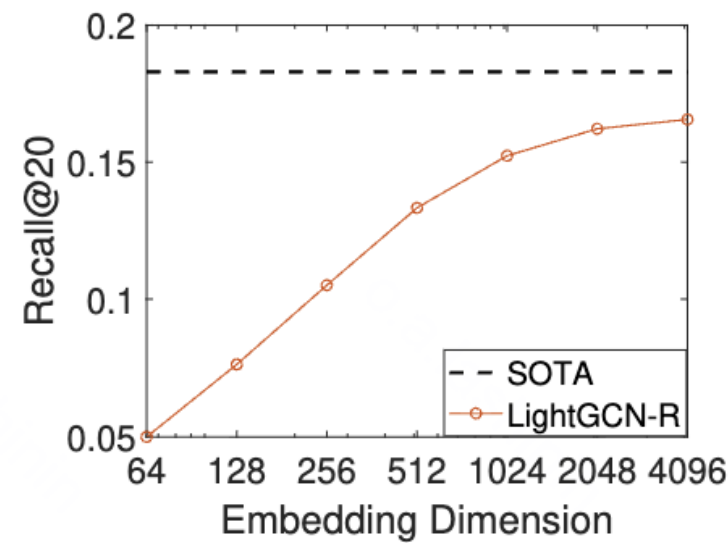


How Powerful is Graph Convolution for Recommendation?

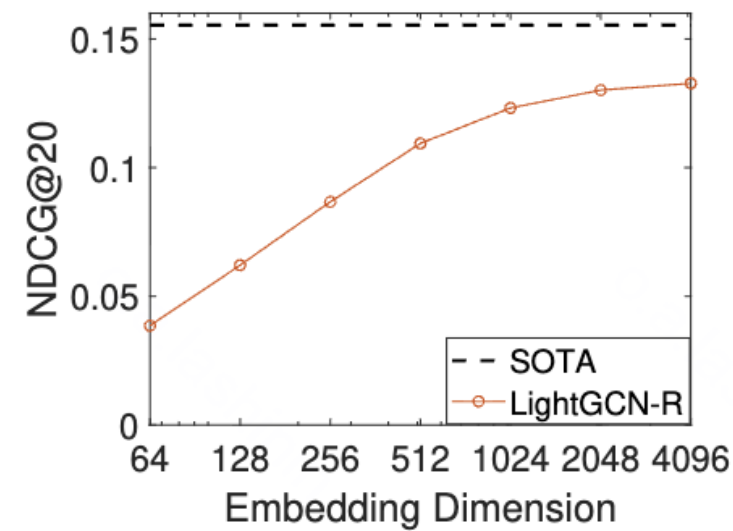
Yifei Shen¹, Yongji Wu², Yao Zhang³, Caihua Shan⁴, Jun Zhang¹, Khaled B. Letaief¹, Dongsheng Li⁴

¹HKUST, ²Duke University, ³Fudan University, ⁴Microsoft Research Asia

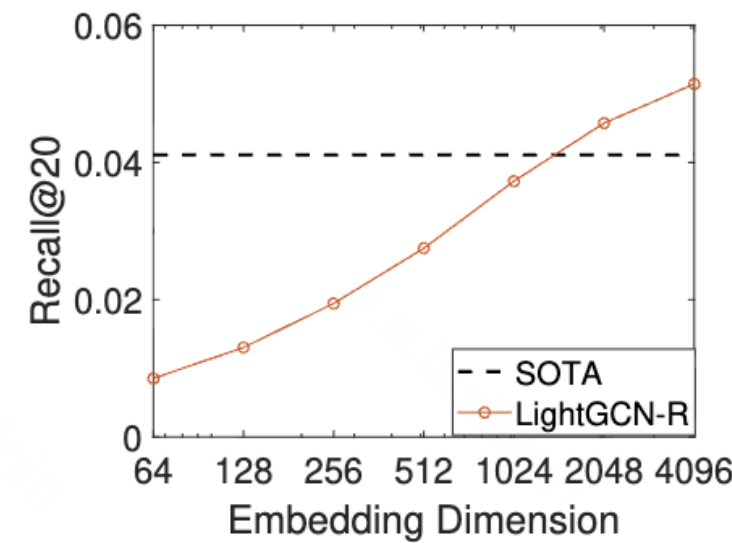
yshenaw@connect.ust.hk, wuyongji317@gmail.com, {yaozhang,dongshengli}@fudan.edu.cn,
{eejzhang, eekhaled}@ust.hk, caihuashan@microsoft.com



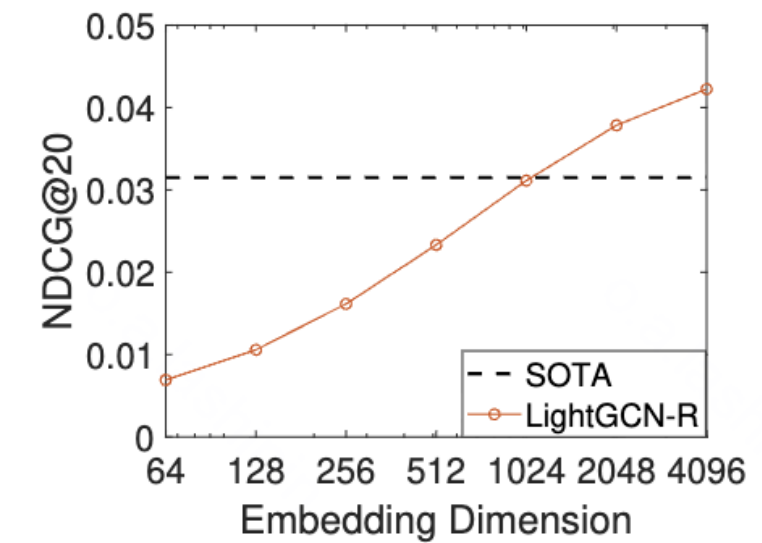
(a) Recall on Gowalla dataset.



(b) NDCG on Gowalla dataset.



(c) Recall on Amazon-book dataset.



(d) NDCG on Amazon-book dataset.

Figure 1: Performance of *untrained* LightGCN versus SOTA, where the SOTA line is LightGCN's performance reported in [13] and LightGCN-R denotes the untrained LightGCN with different embedding dimensions.

GFCF (из той же статьи)

Предсказания по юзеру $\mathbf{s}_u = \mathbf{r}_u \left(\tilde{\mathbf{R}}^T \tilde{\mathbf{R}} + \alpha \mathbf{D}_I^{-\frac{1}{2}} \bar{\mathbf{U}} \bar{\mathbf{U}}^T \mathbf{D}_I^{\frac{1}{2}} \right)$

Вектор юзера через интеракции

item-item similarity matrix

top-K singular vectors of $\tilde{\mathbf{R}}$

Сколько раз айтем встречается в данных

Инференс как у ItemKNN!

Table 3: The comparison of overall performance among GF-CF and competing methods. The performance of benchmarks is reproduced from [13].

Dataset	Gowalla		Yelp2018		Amazon-book	
Method	recall	ndcg	recall	ndcg	recall	ndcg
NGCF	0.1570	0.1327	0.0579	0.0477	0.0344	0.0263
Mult-VAE	0.1641	0.1335	0.0584	0.0450	0.0407	0.0315
GRMF	0.1477	0.1205	0.0571	0.0462	0.0354	0.0270
GRMF-norm	0.1557	0.1261	0.0561	0.0454	0.0352	0.0269
LightGCN	0.1830	0.1554	0.0649	0.0530	0.0411	0.0315
LGCN-IDE	0.1682	0.1347	0.0609	0.0505	0.0612	0.0514
GF-CF	0.1849	0.1518	0.0697	0.0571	0.0710	0.0584

Еще один подход 23 года (GTE)

```
# process the adjacency matrix
adj = scipy_sparse_mat_to_torch_sparse_tensor(train).coalesce().to(device)

# iterative representation propagation on graph
for i in range(k):
    print("Running layer", i)
    user_rep_temp = torch.sparse.mm(adj, item_rep) + user_rep
    item_rep_temp = torch.sparse.mm(adj.transpose(0,1), user_rep) + item_rep
    user_rep = user_rep_temp
    item_rep = item_rep_temp

# evaluation
pred = user_rep.cpu().numpy()
```

How Expressive are Graph Neural Networks in Recommendation?

Xuheng Cai
The University of Hong Kong
Hong Kong, China
rickcai@connect.hku.hk

Xubin Ren
The University of Hong Kong
Hong Kong, China
xubinrengs@gmail.com

Lianghao Xia
The University of Hong Kong
Hong Kong, China
aka_xia@foxmail.com

Chao Huang*
The University of Hong Kong
Hong Kong, China
chaohuang75@gmail.com

Еще один подход 23 года (GTE)

How Expressive are Graph Neural Networks in Recommendation?

Xuheng Cai
The University of Hong Kong
Hong Kong, China
rickcai@connect.hku.hk

Lianghao Xia
The University of Hong Kong
Hong Kong, China
aka_xia@foxmail.com

Xubin Ren
The University of Hong Kong
Hong Kong, China
xubinrengs@gmail.com

Chao Huang*
The University of Hong Kong
Hong Kong, China
chao Huang75@gmail.com

$$w \in \mathcal{N}^{(k)}(u) \cup \{u\}$$

By induction, the expression holds for every iteration k . $\square$

With this Lemma, in the simplified paradigm described above, for a user node u and two item nodes v_1 and v_2 , if $L\text{-TC}(u, v_1) > L\text{-TC}(u, v_2)$, we explore the relationship between their respective predicted scores $\hat{y}_{u,v_1}$ and $\hat{y}_{u,v_2}$ after L GNN layers:

$$\begin{aligned} \hat{y}_{u,v_1} - \hat{y}_{u,v_2} &= \mathbf{e}_u^{(L)} \cdot \mathbf{e}_{v_1}^{(L)} - \mathbf{e}_u^{(L)} \cdot \mathbf{e}_{v_2}^{(L)} \\ &= \left(\sum_{w \in \mathcal{N}^{(L)}(u) \cup \{u\}} L\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)} \right) \cdot \left(\sum_{w' \in \mathcal{N}^{(L)}(v_1) \cup \{v_1\}} L\text{-TC}(v_1, w') \cdot \mathbf{e}_{w'}^{(0)} \right) \\ &\quad - \left(\sum_{w \in \mathcal{N}^{(L)}(u) \cup \{u\}} L\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)} \right) \cdot \left(\sum_{w'' \in \mathcal{N}^{(L)}(v_2) \cup \{v_2\}} L\text{-TC}(v_2, w'') \cdot \mathbf{e}_{w''}^{(0)} \right) \\ &= \left(k_1 + L\text{-TC}(u, v_1) \cdot \mathbf{e}_{v_1}^{(0)} \right) \cdot \left(k_3 + L\text{-TC}(u, v_1) \cdot \mathbf{e}_u^{(0)} \right) \\ &\quad - \left(k_2 + L\text{-TC}(u, v_2) \cdot \mathbf{e}_{v_2}^{(0)} \right) \cdot \left(k_4 + L\text{-TC}(u, v_2) \cdot \mathbf{e}_u^{(0)} \right) \\ &= (k_1 k_3 - k_2 k_4) \\ &\quad + \left((k_1 \mathbf{e}_u^{(0)} + k_3 \mathbf{e}_{v_1}^{(0)}) \underline{L\text{-TC}(u, v_1)} - (k_2 \mathbf{e}_u^{(0)} + k_4 \mathbf{e}_{v_2}^{(0)}) \underline{L\text{-TC}(u, v_2)} \right) \\ &\quad + \left(\mathbf{e}_u^{(0)} \mathbf{e}_{v_1}^{(0)} \left(\underline{L\text{-TC}(u, v_1)} \right)^2 - \mathbf{e}_u^{(0)} \mathbf{e}_{v_2}^{(0)} \left(\underline{L\text{-TC}(u, v_2)} \right)^2 \right) \end{aligned}$$

where

$$\begin{aligned} k_1 &= \sum_{w \in \mathcal{N}^{(L)}(u) \cup \{u\} - \{v_1\}} L\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)}, & k_2 &= \sum_{w \in \mathcal{N}^{(L)}(u) \cup \{u\} - \{v_2\}} L\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)}, \\ k_3 &= \sum_{w' \in \mathcal{N}^{(L)}(v_1) \cup \{v_1\} - \{u\}} L\text{-TC}(v_1, w') \cdot \mathbf{e}_{w'}^{(0)}, & k_4 &= \sum_{w'' \in \mathcal{N}^{(L)}(v_2) \cup \{v_2\} - \{u\}} L\text{-TC}(v_2, w'') \cdot \mathbf{e}_{w''}^{(0)} \end{aligned}$$

$$y_{u,v} = \mathbf{e}_u^{\text{---}} \cdot \mathbf{e}_v^{\text{---}} \quad (10)$$

With this simplified paradigm, we show that when an item v_1 has higher topological closeness to a user u than another item v_2 , the prediction score $\hat{y}_{u,v_1}$ is not necessarily higher than $\hat{y}_{u,v_2}$.

Lemma 6. *With the simplified paradigm described above, in layer k , the embedding of a user u can be written as follows:*

$$\mathbf{e}_u^{(k)} = \sum_{w \in \mathcal{N}^{(k)}(u) \cup \{u\}} k\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)}$$

where $\mathcal{N}^{(k)}(u)$ is the set of k -hop neighbors of u (i.e., all nodes whose distance to u is smaller than or equal to k). Specially, $\mathcal{N}(u) = \mathcal{N}^{(1)}(u)$ is the set of (1-hop) neighbors of u .

Proof. This is obviously true when $k = 0$, because

$$\sum_{w \in \mathcal{N}^{(0)}(u) \cup \{u\}} 0\text{-TC}(u, w) \cdot \mathbf{e}_w^{(0)} = 0\text{-TC}(u, u) \cdot \mathbf{e}_u^{(0)} = \mathbf{e}_u^{(0)}$$

If it holds for $k = l - 1$, then in layer $k = l$, the embedding of u is:

$$\mathbf{e}_u^{(l)} = \sum_{v \in \mathcal{N}(u)} \mathbf{e}_v^{(l-1)} + \mathbf{e}_u^{(l-1)} \quad (11)$$

$$= \sum_{v \in \mathcal{N}(u) \cup \{u\}} \left(\sum_{w \in \mathcal{N}^{(l-1)}(v) \cup \{v\}} (l-1)\text{-TC}(v, w) \cdot \mathbf{e}_w^{(0)} \right) \quad (12)$$

$$= \sum_{v \in \mathcal{N}(u) \cup \{u\}} \left(\sum_{w \in \mathcal{N}^{(l-1)}(v) \cup \{v\}} |\mathcal{P}_{v,w}^{(l-1)}| \cdot \mathbf{e}_w^{(0)} \right) \quad (13)$$

$$= \sum_{w \in \mathcal{N}^{(l)}(u) \cup \{u\}} \left(\sum_{v \in \mathcal{N}(u) \cup \{u\}} |\mathcal{P}_{v,w}^{(l-1)}| \right) \cdot \mathbf{e}_w^{(0)} \quad (14)$$

Еще один подход 23 года (GTE)



Table 1: Performance of GTE and baselines on datasets of different data density, in terms of Recall (R) and NDCG (N).

Dataset	Density	Metric	BiasMF	NGCF	GCCF	LightGCN	SimGRACE	SAIL	HCCF	LightGCL	<u>GTE</u>	<i>p-val</i>	<i>impr.</i>
<i>Denser datasets</i>													
Yelp	$2.1e^{-3}$	R@20	0.0190	0.0294	0.0462	0.0482	0.0603	0.0471	0.0626	0.0793	0.0647	$3e^{-13}$	-18%
		N@20	0.0161	0.0243	0.0398	0.0409	0.0435	0.0405	0.0527	0.0668	0.0556	$3e^{-14}$	-16%
		R@40	0.0371	0.0522	0.0760	0.0803	0.0989	0.0733	0.1040	0.1292	0.0556	$2e^{-14}$	-18%
		N@40	0.0227	0.0330	0.0508	0.0527	0.0656	0.0516	0.681	0.0852	0.0704	$4e^{-15}$	-17%
Douban	$1.4e^{-3}$	R@20	0.0926	0.0999	0.0986	0.1006	0.0827	0.1211	0.1083	0.1216	0.1269	$3e^{-7}$	4%
		N@20	0.0687	0.0739	0.0730	0.0759	0.0603	0.0910	0.0828	0.0927	0.1029	$3e^{-9}$	11%
		R@40	0.1424	0.1505	0.1482	0.1530	0.1251	0.1778	0.1593	0.1708	0.1777	$2e^{-6}$	4%
		N@40	0.0845	0.0897	0.0887	0.0923	0.0735	0.1090	0.0988	0.1077	0.1182	$6e^{-9}$	10%
Tmall	$1.2e^{-3}$	R@20	0.0103	0.0180	0.0209	0.0225	0.0222	0.0254	0.0314	0.0528	0.0578	$1e^{-11}$	9%
		N@20	0.0072	0.0123	0.0141	0.0154	0.0152	0.0177	0.0213	0.0361	0.0290	$1e^{-13}$	-20%
		R@40	0.0170	0.0310	0.0356	0.0378	0.0367	0.0424	0.0519	0.0852	0.0752	$5e^{-11}$	-12%
		N@40	0.0095	0.0168	0.0196	0.0208	0.0203	0.0236	0.0284	0.0473	0.0326	$5e^{-15}$	-31%
<i>Sparser datasets</i>													
Amazon-beauty	$7.3e^{-4}$	R@20	0.0607	0.0734	0.0782	0.0797	0.0539	0.0834	0.0813	0.0896	0.0976	$1e^{-8}$	9%
		N@20	0.0249	0.0290	0.0315	0.0326	0.0212	0.0334	0.0339	0.0369	0.0440	$1e^{-11}$	19%
		R@40	0.0898	0.1078	0.1155	0.1161	0.0836	0.1196	0.1178	0.1286	0.1322	$1e^{-4}$	3%
		N@40	0.0308	0.0360	0.0391	0.0400	0.0272	0.0408	0.0413	0.0447	0.0511	$1e^{-11}$	14%
Gowalla	$4.0e^{-4}$	R@20	0.0196	0.0552	0.0951	0.0985	0.0869	0.0999	0.1070	0.1578	0.1706	$2e^{-10}$	8%
		N@20	0.0105	0.0298	0.0535	0.0593	0.0528	0.0602	0.0644	0.0935	0.1001	$8e^{-11}$	7%
		R@40	0.0346	0.0810	0.1392	0.1431	0.1276	0.1472	0.1535	0.2245	0.2400	$5e^{-11}$	7%
		N@40	0.0145	0.0367	0.0684	0.0710	0.0637	0.0725	0.0767	0.1108	0.1181	$1e^{-10}$	7%
Sparses-Tmall	$8.6e^{-5}$	R@20	0.0328	0.0395	0.0543	0.0542	0.0174	0.0521	0.0501	0.0518	0.0588	$1e^{-10}$	14%
		N@20	0.0169	0.0196	0.0290	0.0288	0.0084	0.0282	0.0270	0.0300	0.0368	$2e^{-9}$	23%
		R@40	0.0439	0.0552	0.0717	0.0708	0.0274	0.0685	0.0655	0.0653	0.0706	$7e^{-8}$	8%
		N@40	0.0195	0.0233	0.0330	0.0327	0.0108	0.0320	0.0305	0.0332	0.0395	$3e^{-9}$	19%

Обзорная статья про графы и прогресс

Challenging the Myth of Graph Collaborative Filtering: a Reasoned and Reproducibility-driven Analysis

Vito Walter Anelli*
vitowalter.aneli@poliba.it
Politecnico di Bari
Italy

Daniele Malitesta*
daniele.malitesta@poliba.it
Politecnico di Bari
Italy

Claudio Pomo*
claudio.pomo@poliba.it
Politecnico di Bari, Italy
Italy

Alejandro Bellogín
alejandro.bellogin@uam.es
Universidad Autónoma de Madrid
Spain

Eugenio Di Sciascio
eugenio.disciascio@poliba.it
Politecnico di Bari
Italy

Tommaso Di Noia
tommaso.dinoia@poliba.it
Politecnico di Bari
Italy

Families	Models	Gowalla		Yelp 2018		Amazon Book	
		Recall	nDCG	Recall	nDCG	Recall	nDCG
<i>Reference</i>	MostPop	0.0416	0.0316	0.0125	0.0101	0.0051	0.0044
	Random	0.0005	0.0003	0.0005	0.0004	0.0002	0.0002
<i>Classic CF</i>	UserkNN	0.1685	0.1370	0.0630	0.0528	0.0582	0.0477
	ItemkNN	0.1409	0.1165	0.0610	0.0507	0.0634	0.0524
	RP ³ β	0.1829	0.1520	0.0671	0.0559	0.0683	0.0565
	EASE ^R *	0.1661	0.1384	0.0655	0.0552	0.0710	<u>0.0567</u>
<i>Graph CF</i>	NGCF	0.1556	0.1320	0.0556	0.0452	0.0319	0.0246
	DGCF	0.1736	0.1477	0.0621	0.0505	0.0384	0.0295
	LightGCN	0.1826	<u>0.1545</u>	0.0629	0.0516	0.0419	0.0323
	SGL	—	—	0.0669	0.0552	0.0474	0.0372
	UltraGCN	0.1863	0.1580	<u>0.0672</u>	0.0553	<u>0.0688</u>	0.0561
	GFCF	<u>0.1849</u>	0.1518	0.0697	0.0571	0.0710	0.0584

Families	Models	Allrecipes		BookCrossing	
		Recall	nDCG	Recall	nDCG
<i>Reference</i>	MostPop	<u>0.0472</u>	<u>0.0242</u>	0.0352	0.0319
	Random	0.0024	0.0010	0.0013	0.0011
<i>Classic CF</i>	UserkNN	0.0339	0.0188	0.0871	0.0769
	ItemkNN	0.0326	0.0180	0.0779	0.0739
	RP ³ β	0.0170	0.0089	0.0941	<u>0.0821</u>
	EASE ^R	0.0351	0.0192	<u>0.0925</u>	0.0847
<i>Graph CF</i>	NGCF	0.0291	0.0144	0.0670	0.0546
	DGCF	0.0448	0.0234	0.0643	0.0543
	LightGCN	0.0459	0.0236	0.0803	0.0660
	SGL	0.0365	0.0192	0.0716	0.0600
	UltraGCN	0.0475	0.0248	0.0800	0.0651
	GFCF	0.0101	0.0051	0.0819	0.0712

*Results for EASE^R on Amazon Book are taken from [BARS Benchmark \[96\]](#).

Индуктивные и трансдуктивные методы

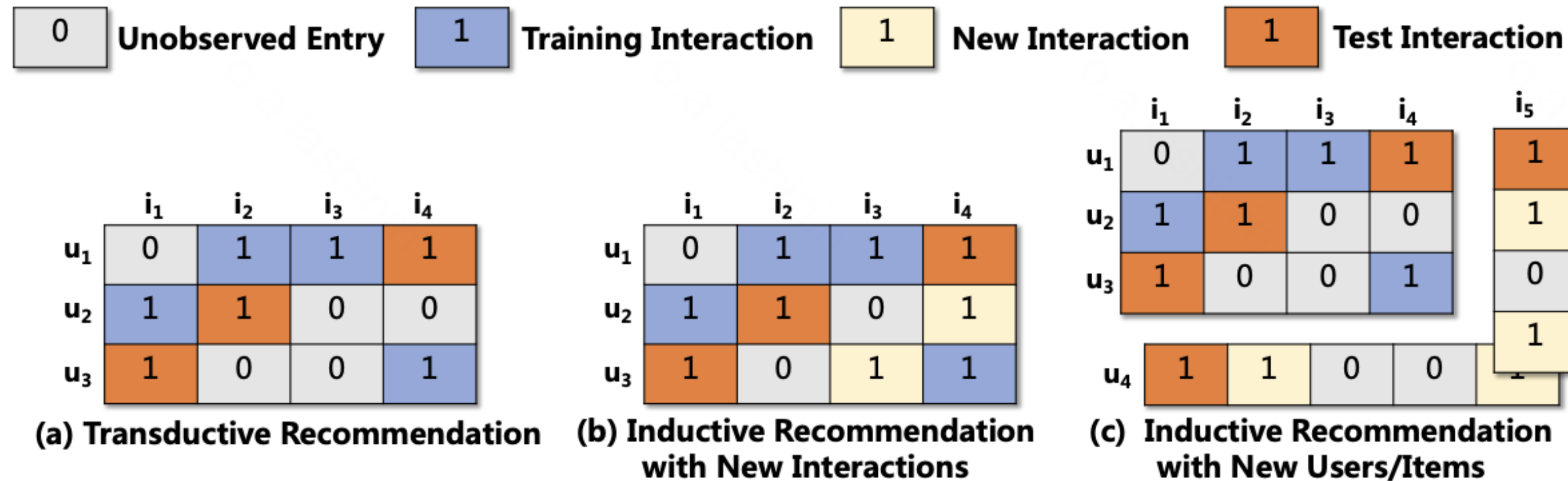


Figure 2: Traditional transductive recommendation scenario and two inductive scenarios proposed in this paper.

LightGCN трансдуктивен по юзерам и айтемам
GFCF трансдуктивен по айтемам, индуктивен по
 новым интеракциям и юзерам

LightGCN ИНДУКТИВНЫЙ

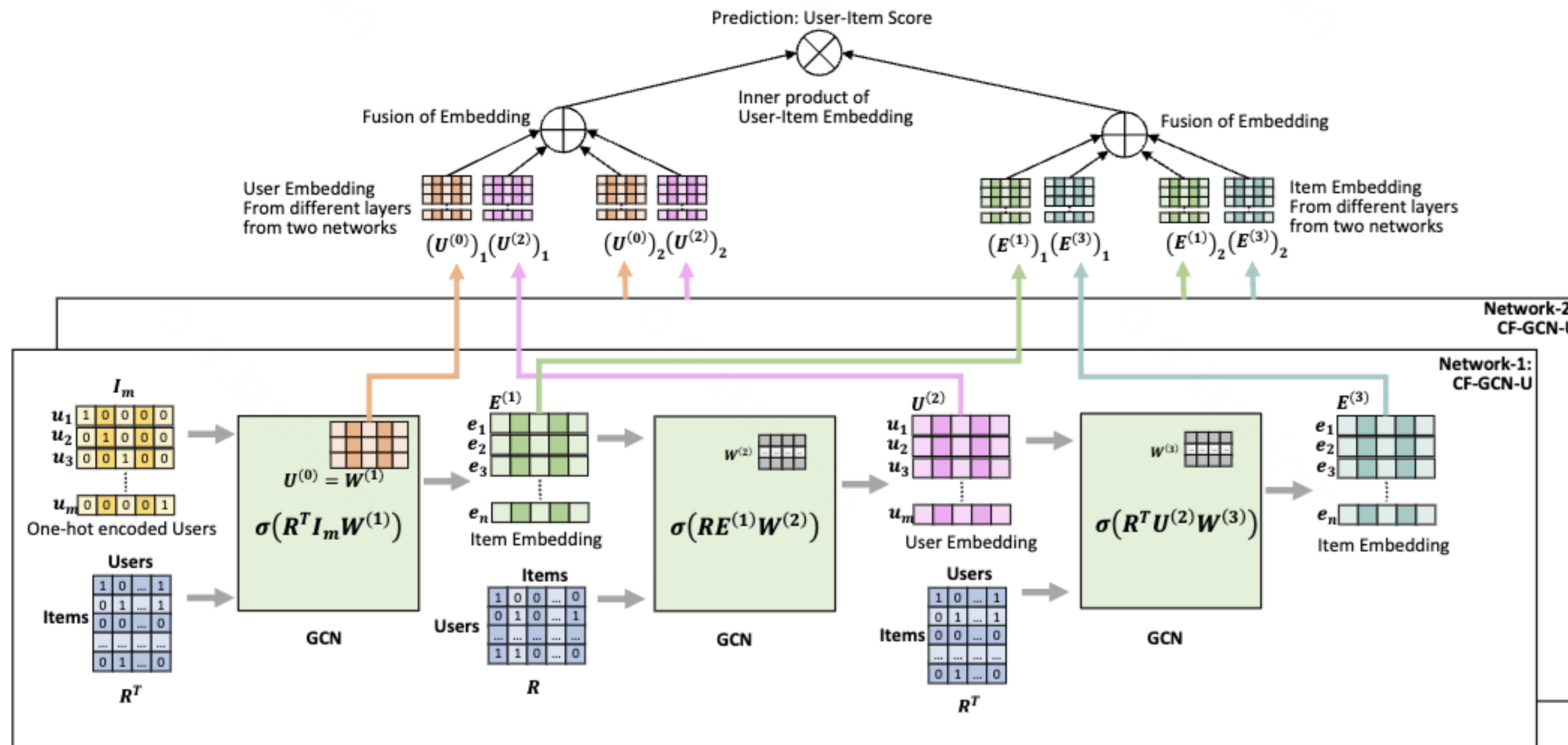
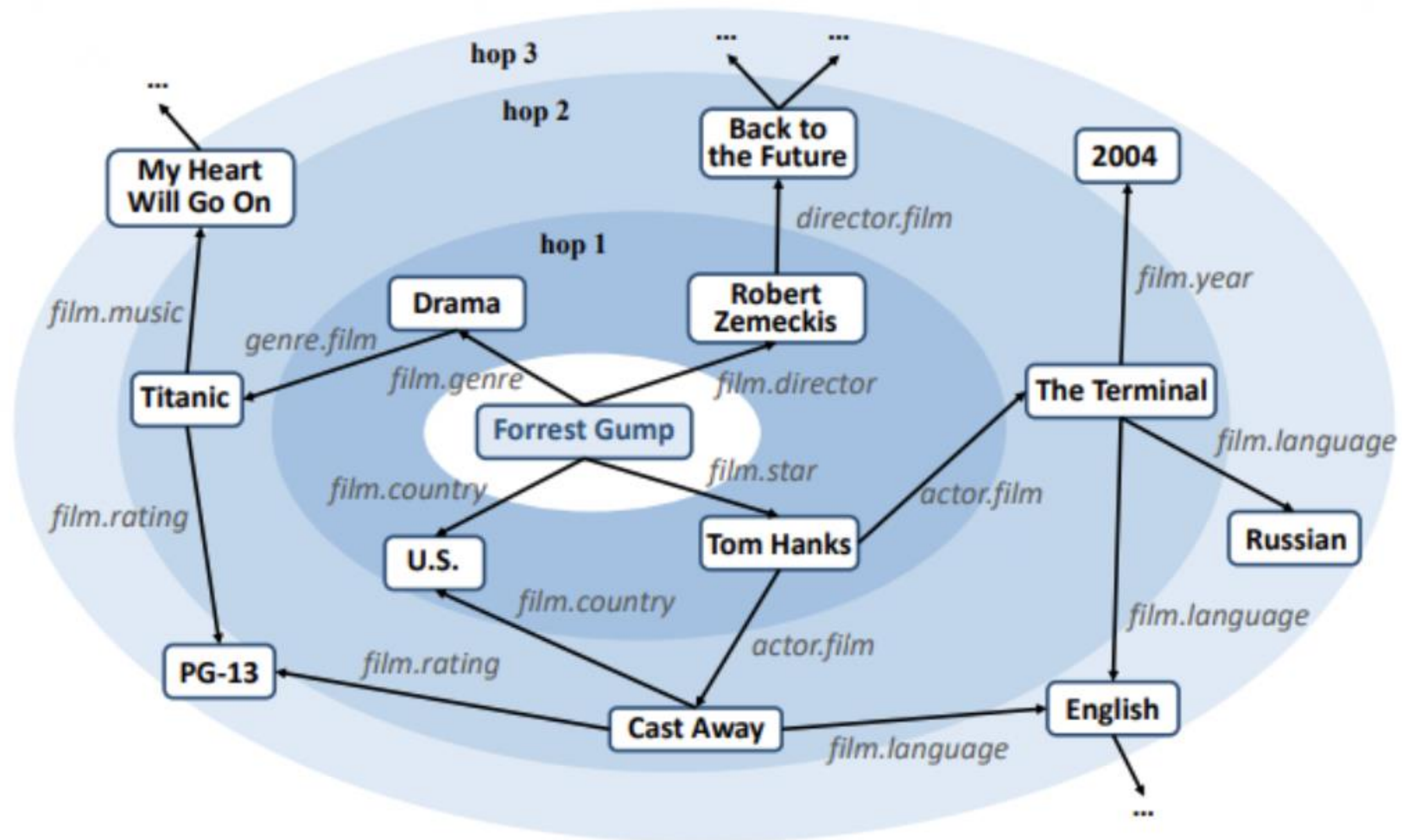
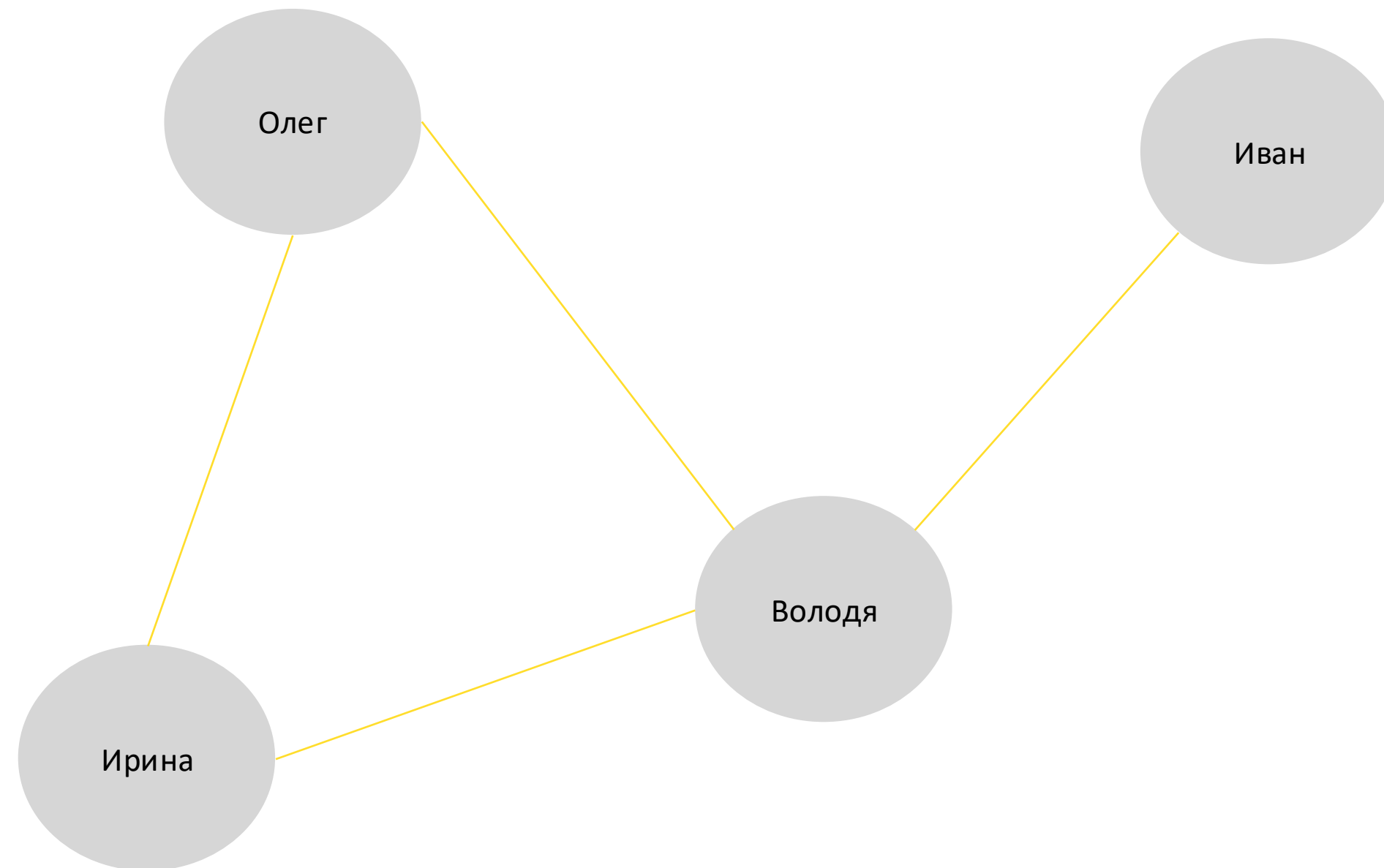


Figure 1: Graph Convolutional Network for Collaborating Filtering Architecture (CF-GCN-U) with Twin Networks. Each network is CF-GCN-U, using GCN layers with graphs R and R^T . We learn user embedding and derive item embedding. Fuse user and item embedding from different layers/networks. Removing layer weight matrices (except in the first layer) and non-linear activation function $\sigma(\cdot)$ result in the CF-LGCN-U network.

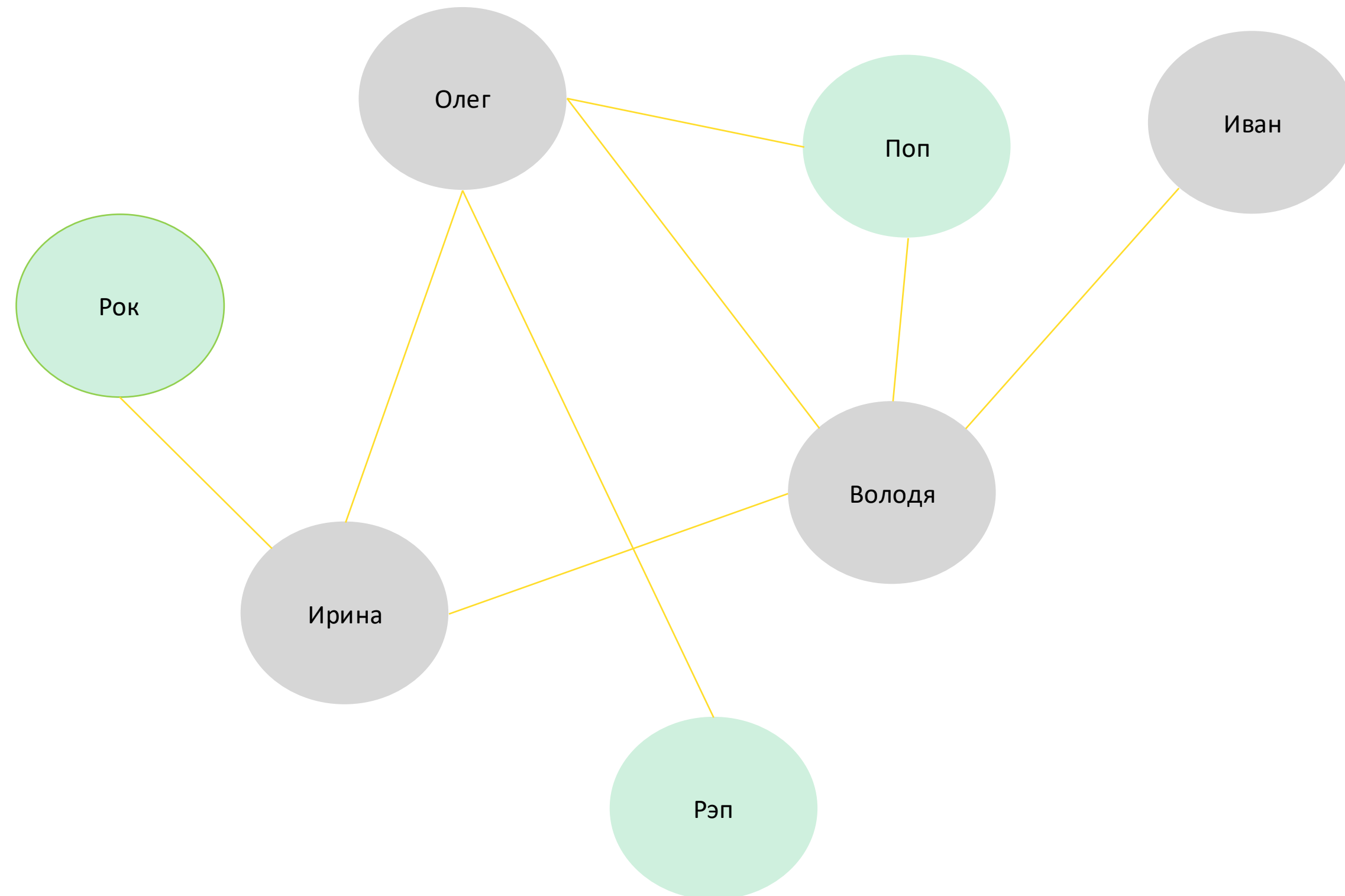
Про ResSys на графах знаний



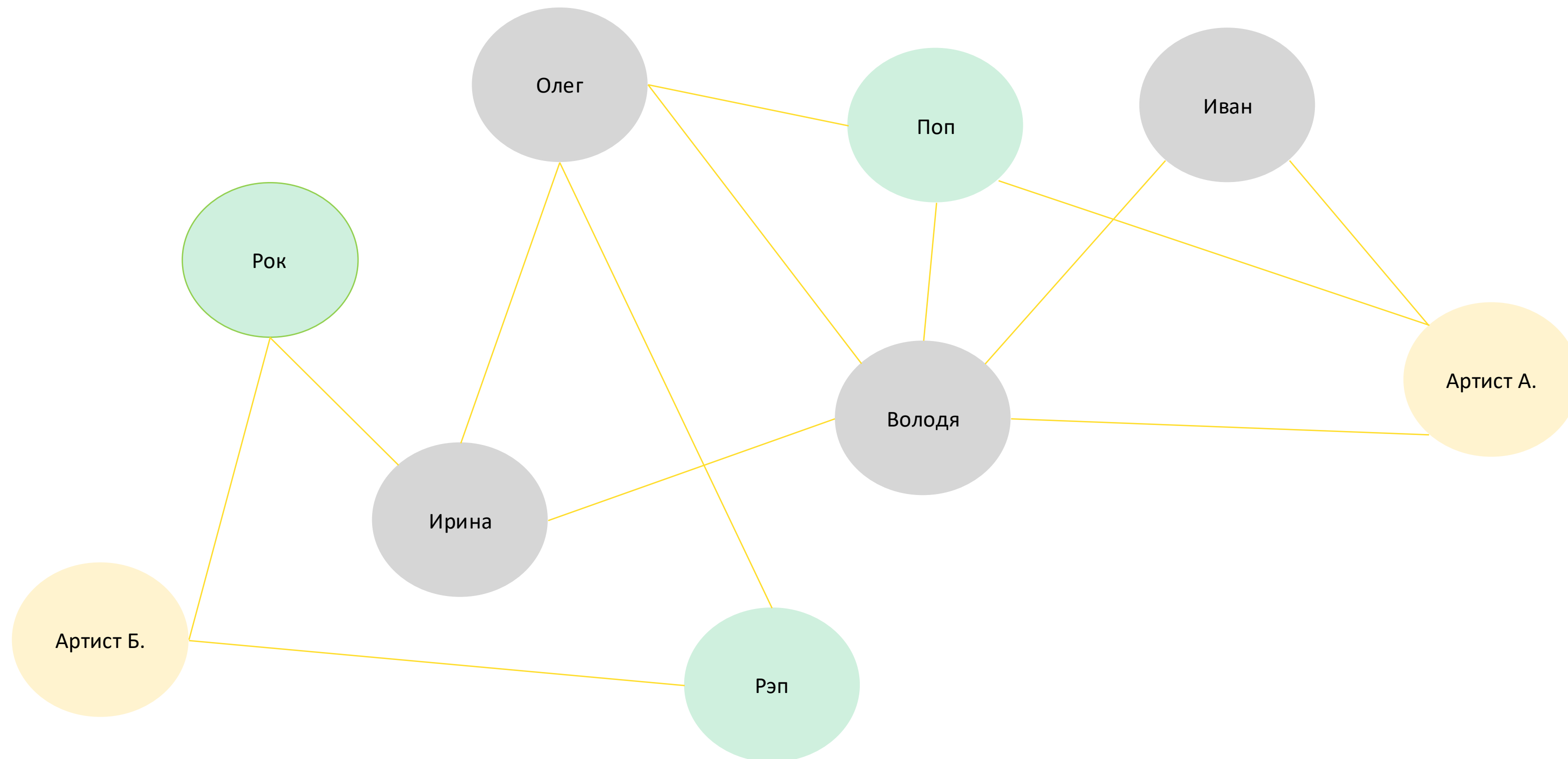
Граф знаний на примерах



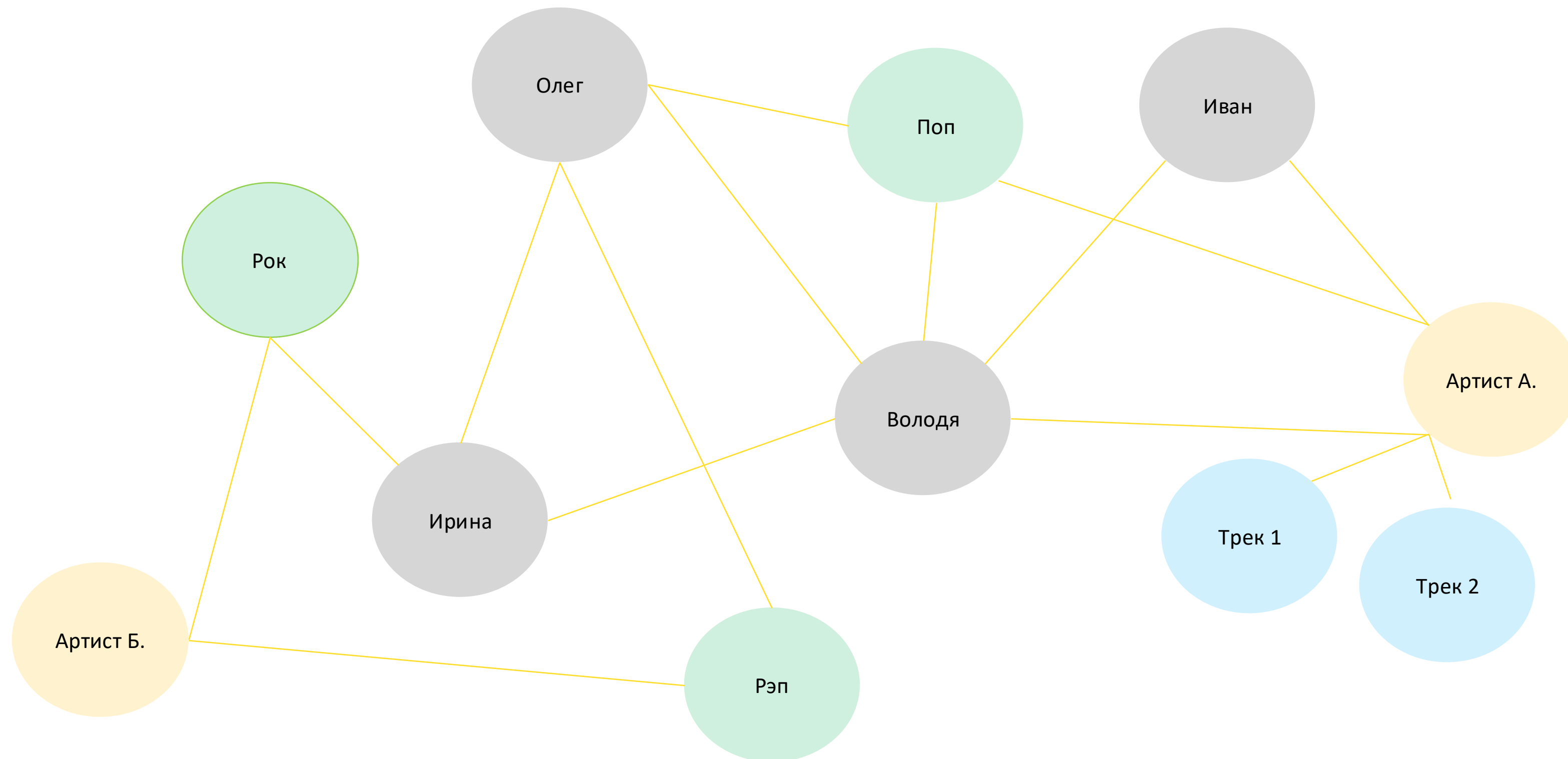
Граф знаний на примерах



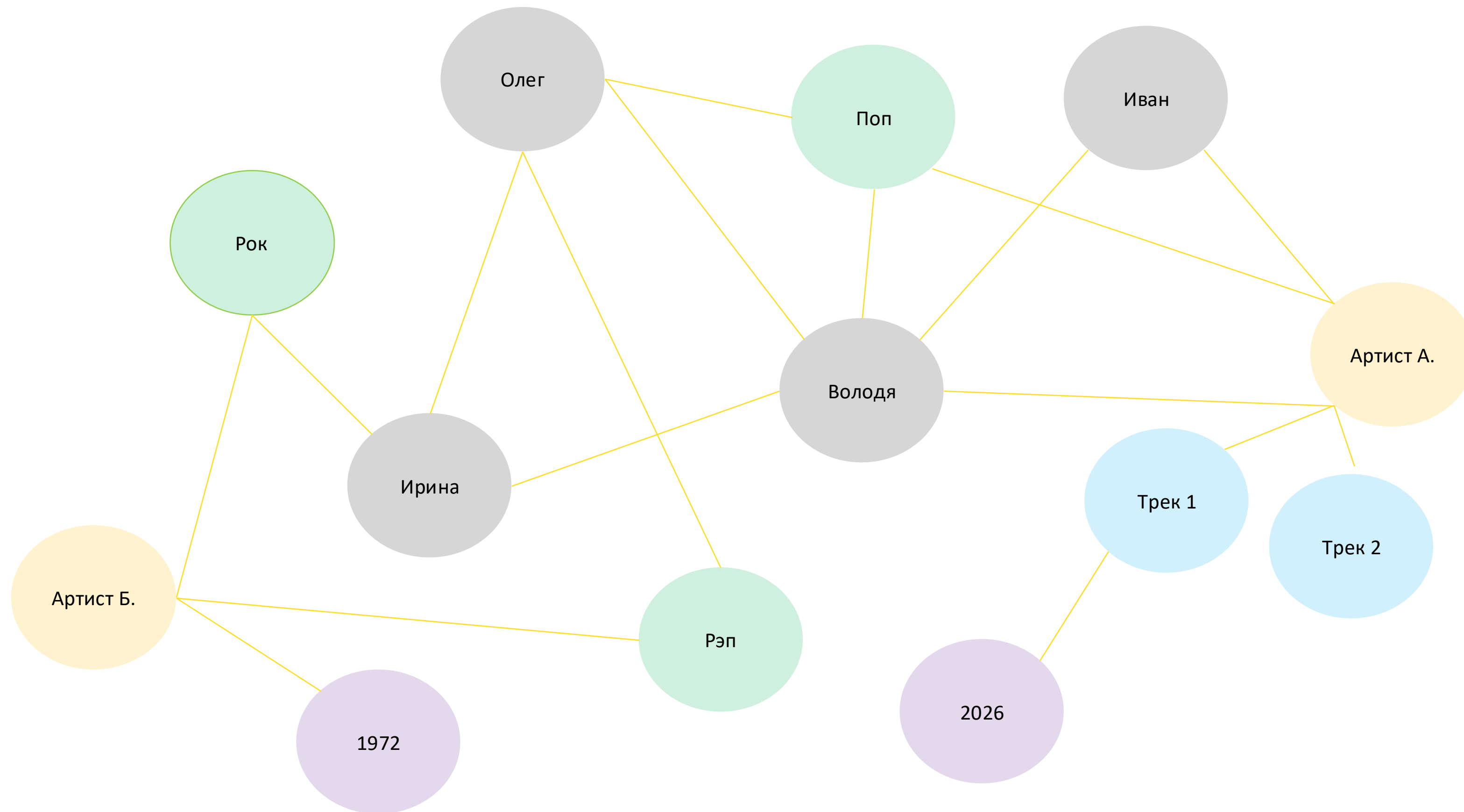
Граф знаний на примерах



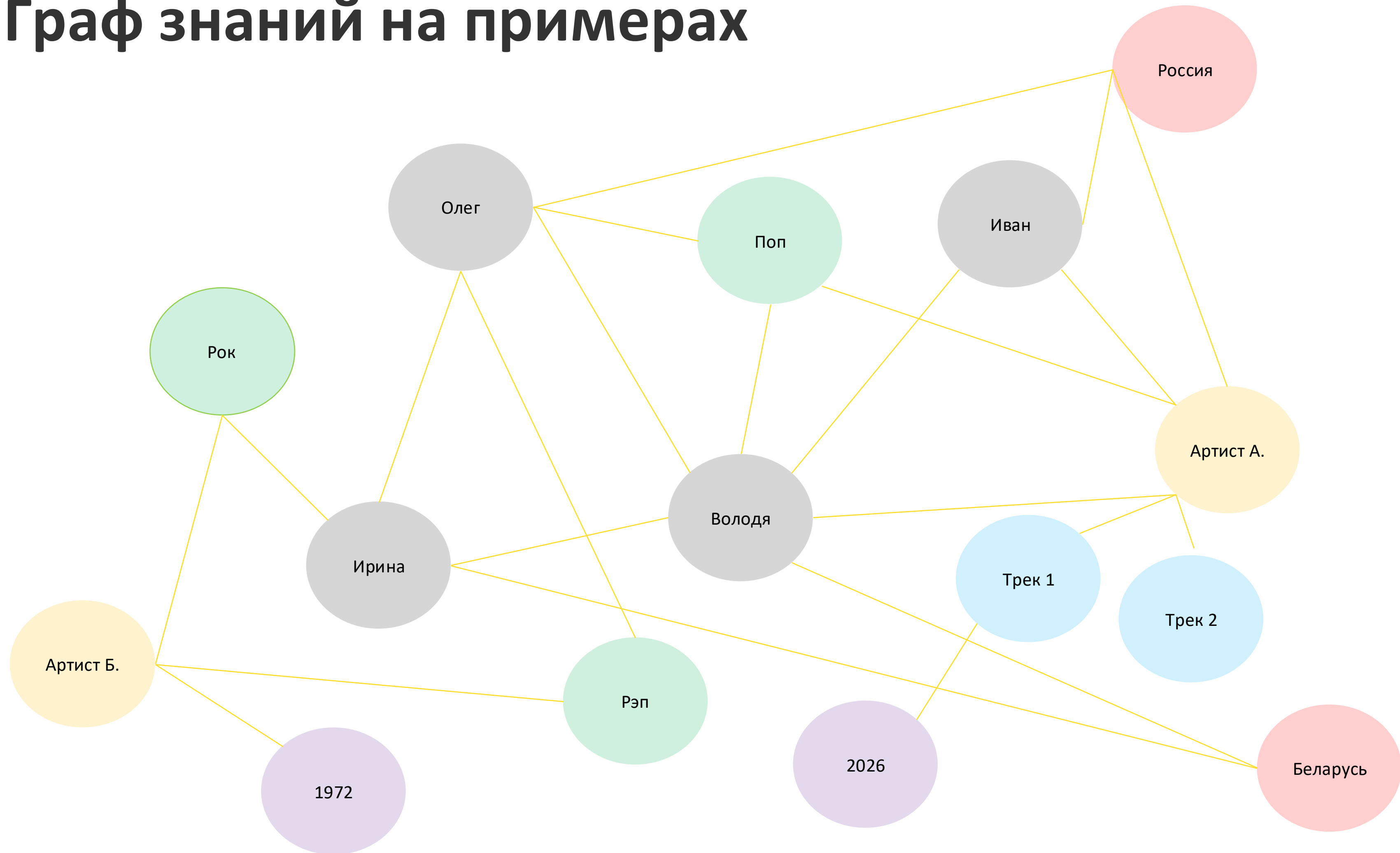
Граф знаний на примерах



Граф знаний на примерах



Граф знаний на примерах



Насколько вообще графы знаний могут работать?



Revisiting the performance evaluation of knowledge-aware recommender systems: are we making progress?

Marina Ananyeva^{1,2,*,†}, Oleg Lashinin^{2,*,†} and Maria Kuznetsova^{2,*,†}

Table 2
Performance comparison on three datasets. The best model in each semantic group is in **bold**, the models are sorted in ascending chronological order. The best model for each dataset and metric pair is in **green**, the second best model in **violet**, and the third best model in **blue**.

Method	MovieLens-1M		TTRS		Amazon-books	
	Recall@10	NDCG@10	Recall@10	NDCG@10	Recall@10	NDCG@10
Pop	0.072	0.126	0.233	0.134	0.012	0.011
ItemKNN [3]	0.163	0.126	0.322	0.240	0.149	0.149
EASE [5]	0.205	0.314	0.389	0.290	0.158	0.157
NGCF [11]	0.172	0.268	0.343	0.254	0.082	0.076
DGCF [28]	0.172	0.265	0.345	0.254	0.087	0.082
LightGCN [12]	0.173	0.269	0.352	0.262	0.088	0.083
SGL [13]	0.179	0.271	0.354	0.264	0.103	0.097
CKE [15]	0.165	0.263	0.349	0.257	0.077	0.072
CFKG [29]	0.154	0.241	0.324	0.240	0.067	0.062
RippleNet [20]	0.121	0.203	0.301	0.222	0.036	0.033
KGCN [21]	0.146	0.234	0.316	0.235	0.051	0.046
MKR [17]	0.139	0.224	0.331	0.246	0.047	0.043
KTUP [16]	0.159	0.251	0.331	0.244	0.061	0.056
KGAT [23]	0.169	0.273	0.342	0.254	0.076	0.071
KGNNLS [22]	0.146	0.235	0.317	0.234	0.052	0.047

Table 1
Graph-based recommender models with or without using side information. Non-graph baselines are in **bold** and target models baselines are underlined. A bipartite graph G_1 is defined as a set of triplets $\{(u, y_{ui}, i) | u \in U, i \in I\}$, where U, I are sets of users and objects, respectively vertices, and connections y_{ui} represent interactions. In the User-Item graph, in addition to interactions, vertices with external features of the item are added.

Model	Year	#citations*	Datasets	Baselines	Metrics	Graph	Motivation
Models with bipartite user-item graph							
NGCF [11]	2019	797	Gowalla, Yelp2018, Amazon-Books	MF, NeuMF, CMN , HOP-Rec, GC-MC, PinSage	Recall@20, NDCG @20	Bipartite	High-order embedding propagation
DGCF [28]	2020	91	Gowalla, Yelp2018, Amazon-Books	MF , GC-MC, <u>NGCF</u> , DisenGCN, MacridVAE	Recall@20, NDCG @20	Bipartite	Disentangled representation
LightGCN [12]	2020	492	Gowalla, Yelp2018, Amazon-Books	<u>NGCF</u> , Mult-VAE , GRMF	Recall@20, NDCG @20	Bipartite	Without complex operations and non-linearity
SGL [13]	2021	59	Yelp2018, Amazon-Books, Alibaba-iFashion	<u>NGCF</u> , LightGCN, Mult-VAE , DNN+SSL	Recall@20, NDCG @20	Bipartite	Self-supervision
KGE models							
CKE [15]	2016	941	MovieLens-1M IntentBooks	BPRMF , BPRMF+TransE, PRP, PER, LibFM , CMF , CTR BPRMF+SDAE	MAP@K, Recall@K K: 20, 40, 60, 80, 100	Item	Structural, textual, visual embeddings + CF
CFKG [29]	2018	181	Amazon	BPR , BPR_HFT , VBPR , DeppCoNN , <u>CKE</u> , JRL	Precision, Recall, HR, NDCG K:10	User-Item	Heterogenous user-item knowledge graph
RippleNet [20]	2018	471	MovieLens-1M Book-Crossing Bing-News	<u>CKE</u> , SHINE , DKN , PER, LibFM + TransR, Wide&Deep	Precision@K, Recall@K, F1@K K: 1, 2, 5, 10, 20, 50, 100	Item for each user	Preference propagation
KGCN [21]	2018	280	MovieLens-20M Book-Crossing Last.FM	SVD , LibFM , LibFM + TransE, PER, <u>CKE</u> , RippleNet	Recall@K K: 1, 2, 5, 10, 20, 50, 100	Item for each user	Graph Convolutional
MKR [17]	2019	214	MovieLens-1M Book-Crossing Last.FM Bing-News	PER, <u>CKE</u> , DKN , <u>RippleNet</u> , LibFM + TransR, Wide&Deep	Recall@K Precision@K K: 2, 5, 10, 20, 50	Item	Multi-task learning, cross&compress unit
Hybrid graph-based models							
KTUP [16]	2019	212	MovieLens-1M	FM , BPRMF , <u>CFKG</u> , <u>CKE</u> , CoFM	Recall@10, Precision@10 F1 score@10, Hit ratio@10, NDCG @10	Item	Graph completion, preference translation
KGAT [23]	2019	605	Amazon-Books Last-FM Yelp2018	FM , NFM , <u>CKE</u> , <u>CFKG</u> , MCRc, <u>RippleNet</u> , GC-MC	Recall@20, NDCG @20	Bipartite + item	Attention
KGNN-LS [22]	2019	205	MovieLens-20M, Book-Crossing, Last.FM, Dianping-Food	SVD , LibFM , LibFM+TransE, PER, <u>CKE</u> , <u>RippleNet</u>	Recall@K K: 2, 10, 50, 100	Item for each user	Label smoothing

*The data is current as of 2022-03-25.

Насколько вообще графы знаний могут работать?



Revisiting the performance evaluation of knowledge-aware recommender systems: are we making progress?

Marina Ananyeva^{1,2,*,†}, Oleg Lashinin^{2,*,†} and Maria Kuznetsova^{2,*,†}

Table 2
Performance comparison on three datasets. The best model in each semantic group is in **bold**, the models are sorted in ascending chronological order. The best model for each dataset and metric pair is in **green**, the second best model in **violet** and the third best model in **blue**.

Method	MovieLens-1M		TTRS		Amazon-books	
	Recall@10	NDCG@10	Recall@10	NDCG@10	Recall@10	NDCG@10
Pop	0.072	0.126	0.233	0.134	0.012	0.011
ItemKNN [3]	0.163	0.126	0.322	0.240	0.149	0.149
EASE [5]	0.205	0.314	0.389	0.290	0.158	0.157
NGCF [11]	0.172	0.268	0.343	0.254	0.082	0.076
DGCF [28]	0.172	0.265	0.345	0.254	0.087	0.082
LightGCN [12]	0.173	0.269	0.352	0.262	0.088	0.083
SGL [13]	0.179	0.271	0.354	0.264	0.103	0.097
CKE [15]	0.165	0.263	0.349	0.257	0.077	0.072
CFKG [29]	0.154	0.241	0.324	0.240	0.067	0.062
RippleNet [20]	0.121	0.203	0.301	0.222	0.036	0.033
KGCN [21]	0.146	0.234	0.316	0.235	0.051	0.046
MKR [17]	0.139	0.224	0.331	0.246	0.047	0.043
KTUP [16]	0.159	0.251	0.331	0.244	0.061	0.056
KGAT [23]	0.169	0.273	0.342	0.254	0.076	0.071
KGNNLS [22]	0.146	0.235	0.317	0.234	0.052	0.047

Table 3
Training time comparison on the MovieLens-1M dataset.

Model	#epochs	Best model					Overall tuning	
		Sec./epoch		Overall training time, min.	GPU RAM, GB		#hparams/greed size	Tuning time, h.
		Train	Eval		Train	Eval		
Pop	1	1,95	6,66	0,14	0,00	0,00	-	0,0
ItemKNN	1	1,68	12,20	0,23	0,00	0,00	24/24	0,5
EASE	1	1,65	8,68	0,17	0,00	0,00	6/6	0,7
NGCF	93	23,32	5,79	45,12	0,31	0,32	115/2400	24
DGCF	168	139,22	5,93	406,42	4,92	4,92	15/180	24
LightGCN	262	7,58	5,73	58,12	0,17	0,17	80/80	24
SGL	92	29,35	5,64	53,65	0,41	0,41	79/100	24
CKE	239	8,55	5,91	57,60	0,39	0,39	60/60	19,3
CFKG	83	3,98	10,05	19,41	0,17	0,17	60/60	15
RippleNet	10	30,47	9,06	6,59	0,99	1,00	83/100	24
KGCN	66	6,83	11,51	20,17	0,16	0,18	5/5	5
MKR	70	10,91	14,43	29,56	0,44	0,56	96/180	24
KTUP	212	8,64	13,06	76,67	0,18	0,18	101/240	24
KGAT	98	27,13	5,64	53,52	1,18	2,72	89/375	24
KGNNLS	66	12,82	11,69	26,96	0,19	0,19	5/5	5,1

"Overall training time, min." = ("Train sec./epoch" + "Eval sec./epoch") * "#epochs" / 60

"#hparams" - number of randomly iterated hyperparameters from the grid

"Tuning time" - time spent iterating hyperparameters from the grid (maximum - 24 hours)

PinSage

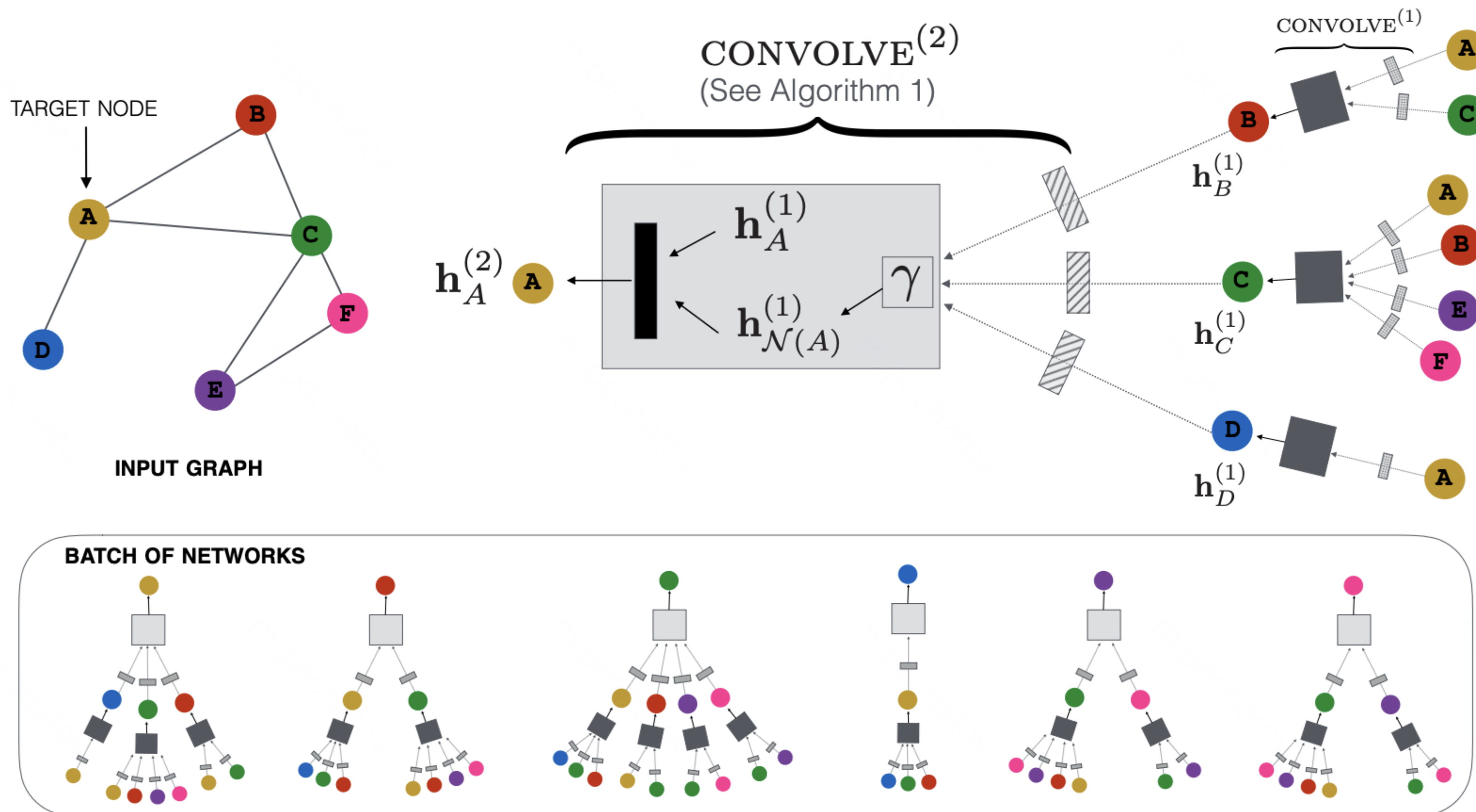
Graph convolutional neural networks for web-scale recommender systems

R Ying, R He, K Chen, P Eksombatchai... - Proceedings of the 24th ..., 2018 - dl.acm.org

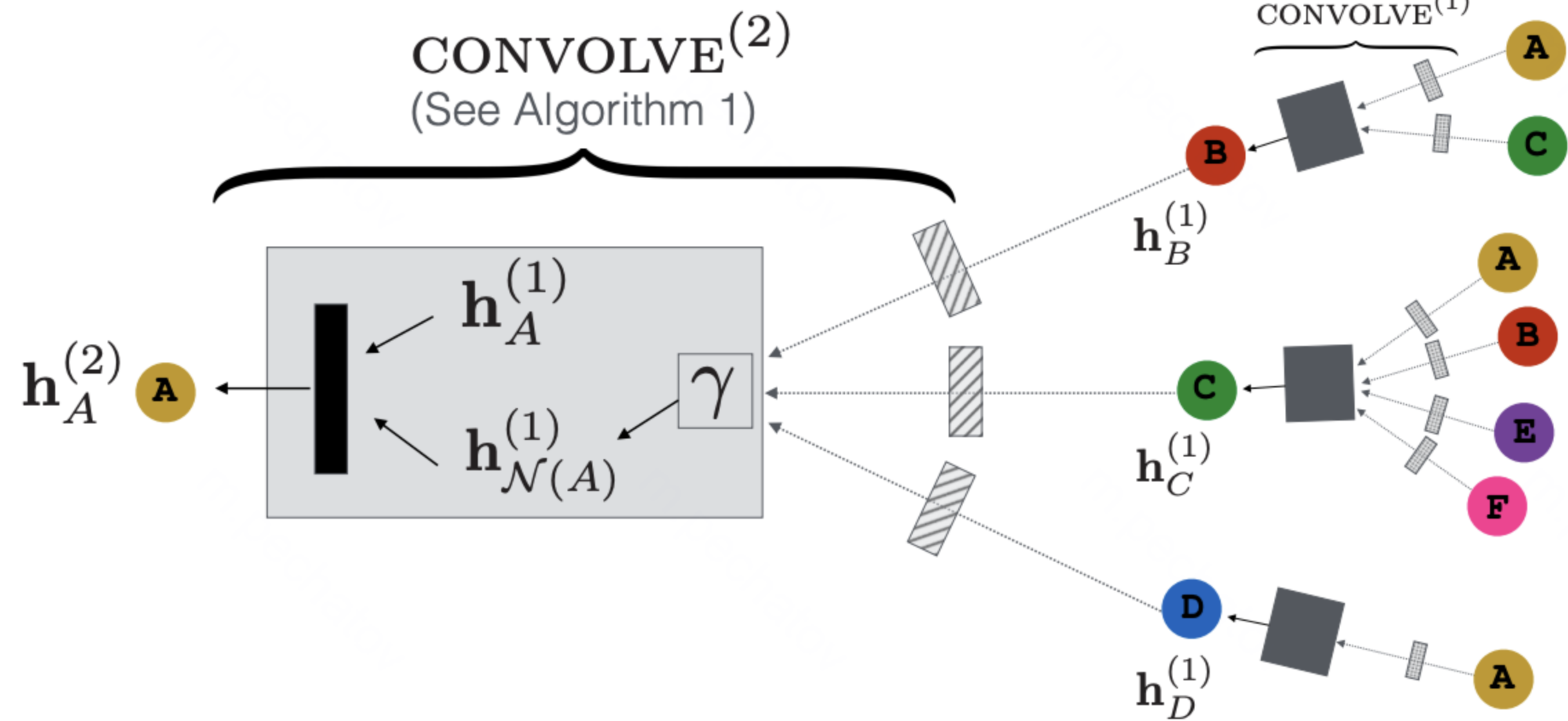
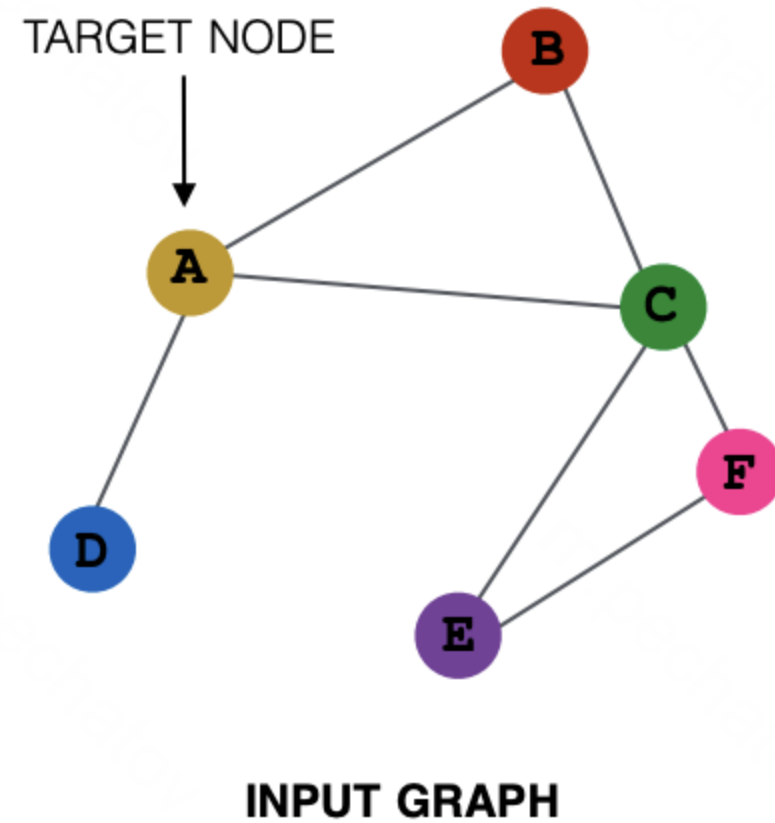
... Recent advancements in deep **neural networks** for **graph**-structured data ... **Graph Convolutional Network** (GCN) algorithm **PinSage**, which combines efficient random walks and **graph** ...

☆ Save 77 Cite Cited by 5302 Related articles All 7 versions

[PDF] acm.org



PinSage



- Работаем на item-item графе.
- Выбираем для каждого item топ-N релевантных айтемов как позитивы.
- Негативы – рандомные + hard

Позитивные и негативные кандидаты



Query

Positive Example

Random Negative

Hard Negative

$$J_{\mathcal{G}}(z_q z_i) = \mathbb{E}_{n_k \sim P_n(q)} \max\{0, z_q \cdot z_{n_k} - z_q \cdot z_i + \Delta\},$$

Якорный айтем

Отступ

Figure 2: Random negative examples and hard negative examples. Notice that the hard negative example is significantly more similar to the query, than the random negative example, though not as similar as the positive example.

PinSage

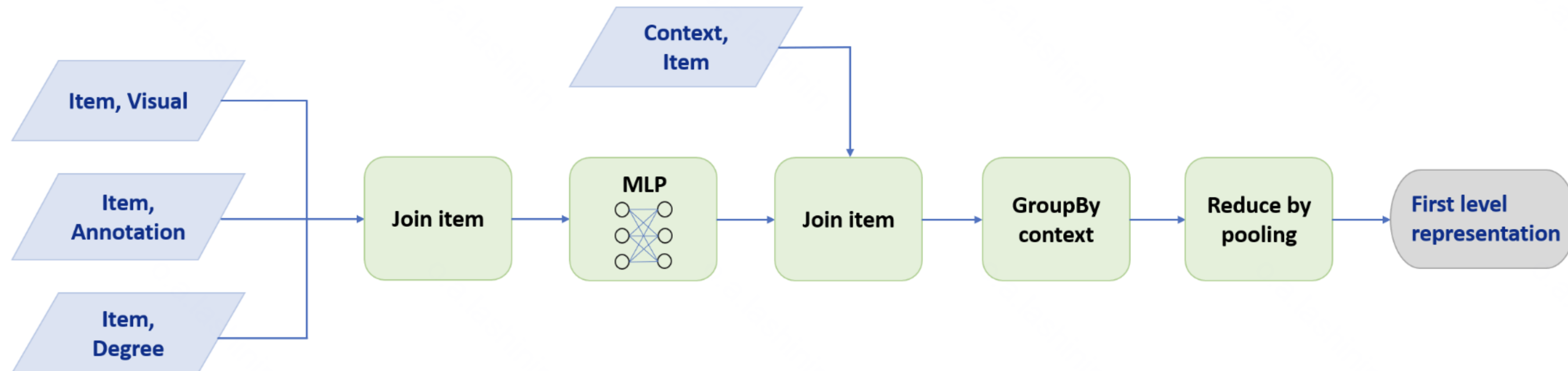
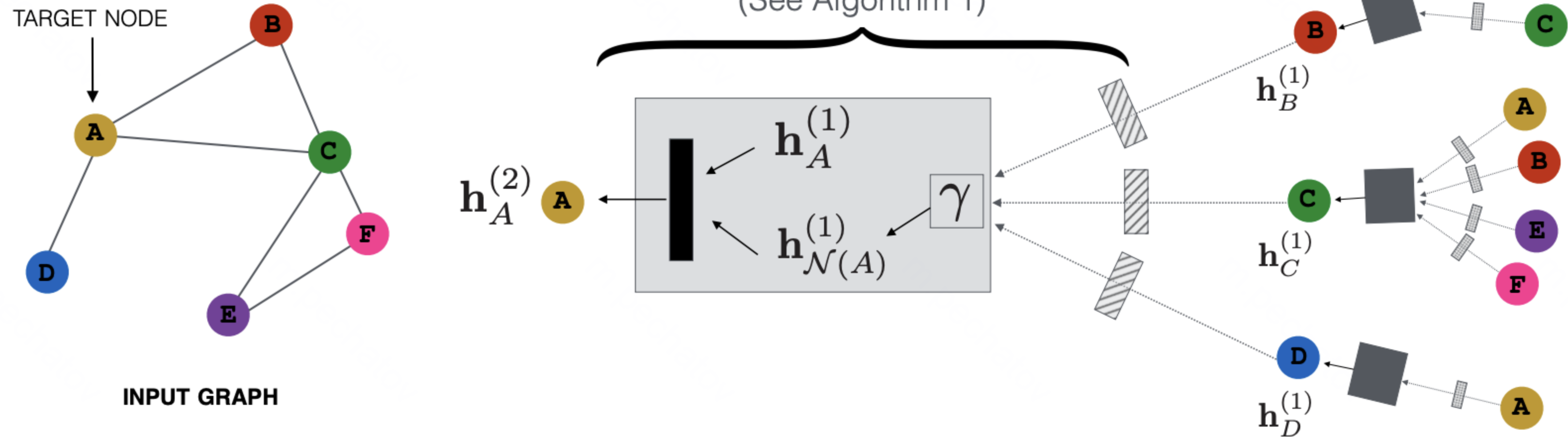


Figure 3: Node embedding data flow to compute the first layer representation using MapReduce. The second layer computation follows the same pipeline, except that the inputs are first layer representations, rather than raw item features.

Сравнение графов с трансформерами

	Графы	Трансформеры
Знают последовательность событий	-	+
Умеют «забывать» старое и фокусироваться на новом	Нужны более сложные хаки	Нативно умеют
Имеют сложную математику	Скорее да	Скорее нет
Можно инференсить по батчам	Скорее да, но с потерей информации (сэмплированием)	Точно да
Могут работать на новых юзерах и айтемах	Часть моделей да, но часть нет	На новых юзерах SASRec by design, на айтемах – тоже могут
Явно моделируют связи нод между собой (айтем – жанр)	Да	Нет
Запоминают информацию	Через сам граф и веса	Только через веса модели